Universidad Nacional Autónoma de México
Facultad de Ingeniería
División de Ingeniería Eléctrica
Departamento de Control y Robótica

CONTROL DISTRIBUIDO E INTEGRACIÓN SCADA

Documentación técnica del HMI

Control y Supervisión de Sistema Secuencial Electroneumático

Gateway OPC DA · núcleo · interfaz PySide6/QML · gemelo digital web



AUTOR

Daan Yael Maldonado Uriostigue

Planta	SLC 5/04 (1747-L541C) · 5 cilindros · RSLinx Classic
Software	Python 3.12/3.13 · PySide6 (Qt 6) · ZeroMQ · aiohttp · three.js
Versión	1.0 · 10 de octubre de 2026 · Ciudad Universitaria, CDMX

Resumen

Este documento describe, de punta a punta, el software de supervisión y mando desarrollado para la estación electroneumática del laboratorio: una planta de cinco cilindros neumáticos gobernada por un PLC Allen-Bradley SLC 5/04 con un programa Ladder derivado de un GRAFCET. El sistema sustituye al HMI v6.1 hecho en Tkinter y está pensado como alternativa moderna a las interfaces hechas con Delphi y otras herramientas RAD.

El software se divide en dos procesos que se comunican por ZeroMQ. El primero es un *gateway* en Python de 32 bits que habla OPC DA con RSLinx Classic. El segundo es un HMI en Python de 64 bits con PySide6 y QML. Los acompaña un gemelo digital web: un servidor aiohttp y una escena 3D con three.js. Un *puente* lleva los datos de la planta al servidor por WebSocket o por MQTT y trae de vuelta órdenes firmadas con HMAC-SHA256.

Cada capítulo explica qué hace una parte del código y por qué se hizo así:

- los protocolos de comunicación (DF1, OPC DA, ZeroMQ con msgpack, WebSocket, MQTT) y las bibliotecas elegidas;
- el modelo físico de los cilindros y de la cadena neumática con retorno por muelle;
- el estimador de carrera y el «modo suave», que predice la animación cuando el enlace serie es lento;
- el sistema de diseño gráfico (capas SVG a escala 1 mm = 1 unidad, botones, LEDs, iconos, tipografía Inter);
- las animaciones declarativas de Qt Quick y la integración con Qt Design Studio;
- las normas que orientan la interfaz y la planta (ISA-101, IEC 60073, IEC 60848, IEC 61131-3, ISO 15552, ISO 6432, ISO 1219, ISA-18.2);
- las pruebas automáticas y las mediciones de latencia hechas en el laboratorio.

El documento termina con una comparación razonada entre Python y Delphi/RAD para construir HMI innovadoras.

> **Cómo leer este documento** Las rutas de archivo (`core/tags.py`) son relativas a la raíz del repositorio del HMI. Las referencias `archivo:línea` señalan el lugar exacto del código en el estado del 10 de octubre de 2026 y pueden desplazarse algunas líneas con cambios posteriores. Los nombres del dominio del PLC se escriben tal cual aparecen en el Ladder (`SC1`, `EV_E1`, `STARTV`, `X0`…`X8`). Las cajas verdes «Por qué» explican decisiones de diseño; las rojas, reglas de seguridad que el código no debe romper.

Índice general

Resumen i

1. Introducción 1

1.1. Propósito del documento . . . 1

1.2. Contexto del proyecto . . . 1

1.3. Visión general del software . . . 2

1.4. Estructura del repositorio . . . 2

1.5. Modos de operación . . . 3

1.6. Cómo se ejecuta . . . 4

2. El sistema controlado 6

2.1. La planta electroneumática . . . 6

2.1.1. Cilindros de doble efecto 1A, 2A y 3A . . . 6

2.1.2. Cadena neumática 4A/5A . . . 6

2.2. El controlador . . . 8

2.3. Tabla de variables . . . 8

2.4. La secuencia en GRAFCET . . . 9

2.5. Del GRAFCET al Ladder . . . 11

2.5.1. Pendientes conocidos del programa . . . 12

3. Arquitectura del software 13

3.1. Los requisitos que la definen . . . 13

3.2. Por qué dos procesos y no dos hilos . . . 13

3.3. Paquetes y dependencias . . . 14

3.4. Contratos . . . 16

3.5. Un ciclo de punta a punta . . . 16

3.6. Comparación con el HMI v6.1 . . . 16

4. Protocolos de comunicación 19

4.1. DF1 y el enlace con el PLC . . . 19

4.2. OPC DA . . . 20

4.2.1. Qué es . . . 20
4.2.2. Caché o dispositivo . . . 20
4.2.3. OpenOPC y el requisito de 32 bits . . . 21
4.2.4. Escrituras . . . 22
4.3. ZeroMQ: el bus entre procesos . . . 22
4.3.1. Qué es . . . 22
4.3.2. Los dos patrones usados . . . 22
4.3.3. Por qué ZeroMQ . . . 23
4.3.4. Un detalle de Windows: la granularidad del reloj . . . 24
4.4. msgpack: el formato de los mensajes . . . 24
4.4.1. El snapshot . . . 24
4.4.2. Órdenes y respuestas . . . 25
4.5. WebSocket: el enlace saliente con el servidor . . . 25
4.6. MQTT: el transporte alternativo . . . 26
4.7. Seguridad del enlace: HMAC, firmas y contraseñas . . . 27
4.7.1. HMAC-SHA256 . . . 27
4.7.2. Órdenes firmadas . . . 27
4.7.3. Sobre sellado para MQTT . . . 28
4.7.4. Cuentas del servidor . . . 28
4.8. Resumen comparativo . . . 28

5. Bibliotecas y herramientas 30
5.1. Inventario . . . 30
5.2. Qt, PySide6 y QML . . . 31
5.2.1. Un solo QML para Qt 5 y Qt 6 . . . 31
5.3. pyzmq y msgpack . . . 32
5.4. OpenOPC y pywin32 . . . 32
5.5. aiohttp y paho-mqtt . . . 32
5.6. SQLite . . . 33
5.7. loguru . . . 33
5.8. three.js . . . 33
5.9. pytest y Playwright . . . 33
5.10. Herramientas de desarrollo . . . 34

6. Núcleo y gateway **35**

6.1. `core/tags.py`: la tabla de variables . . . 35
6.2. `gateway/servidor.py`: el bucle . . . 36
6.2.1. Un solo hilo con planificación por plazo . . . 36
6.2.2. Los siete pasos de cada ciclo . . . 37
6.2.3. Pulsos cronometrados en el gateway . . . 37
6.3. Los dos backends . . . 38
6.3.1. `BackendSim` . . . 38
6.3.2. `BackendOPC` . . . 38
6.4. `core/alarmas.py` . . . 38
6.5. `core/historial.py` . . . 39
6.6. `core/mando.py`: el árbitro de mando . . . 39
6.7. `core/latencia.py` . . . 40
6.8. `gateway/diagnostico_opc.py` . . . 40
6.9. Línea de órdenes del gateway . . . 41

7. Física y modelos **42**

7.1. La física de un cilindro neumático . . . 42
7.1.1. Fuerza . . . 42
7.1.2. Dinámica . . . 43
7.2. Modelo del simulador: válvula biestable y carrera . . . 43
7.3. La cadena 4A/5A . . . 44
7.4. El Ladder simulado . . . 45
7.5. El estimador de carrera (modo REAL) . . . 46
7.5.1. Reglas . . . 46
7.5.2. Calibración . . . 46
7.6. El modo suave . . . 47
7.6.1. El problema . . . 47
7.6.2. La idea . . . 47
7.6.3. Temporizador previsto . . . 48
7.6.4. Cinemática del vástago en modo suave . . . 49
7.6.5. El seguidor del navegador . . . 49

8. La interfaz gráfica **50**

8.1. Arranque: `ui/app.py` . . . 50
8.2. `Planta`: el puente entre Python y QML . . . 51
8.2.1. Cómo llegan los datos sin congelar la pantalla . . . 52
8.2.2. Lógica que vive en el cliente . . . 53
8.3. La pantalla . . . 53
8.3.1. Ventana y pantalla separadas . . . 53
8.3.2. Rejilla de 12 columnas . . . 53
8.3.3. Mandos y confirmación . . . 54
8.3.4. Temas oscuro y gris . . . 54
8.4. Ventana de conexión . . . 56
8.5. Visor de historial . . . 57
8.6. Herramientas de la interfaz . . . 57

9. Assets y sistema de diseño 59
9.1. Organización . . . 59
9.2. Cilindros vectoriales por capas . . . 59
9.2.1. Un milímetro, una unidad . . . 59
9.2.2. Las capas y su orden . . . 60
9.2.3. El muelle . . . 61
9.2.4. Por qué SVG por capas . . . 62
9.2.5. Restricciones de QtSvg . . . 62
9.2.6. Cómo se generaron . . . 62
9.3. Tokens de color y significado . . . 64
9.4. Botones, LEDs e iconos . . . 64
9.5. Logos y tipografía . . . 65
9.6. Cómo carga el HMI los assets . . . 66

10. Animaciones 67
10.1. Principios . . . 67
10.2. Las herramientas de Qt Quick . . . 67
10.2.1. Curvas de suavizado . . . 68
10.3. El vástago de 1A, 2A y 3A . . . 69
10.4. Cámaras, bobinas y sensores . . . 70
10.5. Cadena neumática 4A/5A . . . 70

10.6. GRAFCET . . . 71
10.7. Alarmas . . . 71
10.8. Botones y controles . . . 71
10.9. Tendencias . . . 71
10.10 Lo que no se usó, y por qué . . . 72
10.11 Rendimiento . . . 72

11. Integración con Qt Design Studio 73
11.1. Qué es y por qué se incorporó . . . 73
11.2. El problema: el diseñador no ejecuta Python . . . 73
11.2.1. 1. Separar la pantalla de la ventana . . . 73
11.2.2. 2. Datos de prueba generados, no escritos . . . 73
11.2.3. 3. La ruta de los assets . . . 74
11.2.4. 4. La rejilla en un componente aparte . . . 75
11.3. El proyecto `HMI.qmlproject` . . . 75
11.4. Cómo conviven el diseño visual y el código . . . 75
11.5. Comparación con el diseñador de formularios de Delphi . . . 76
11.6. Limitaciones conocidas . . . 76

12. Normas aplicables 78
12.1. ISA-101: interfaces hombre-máquina . . . 79
12.1.1. Qué dice . . . 79
12.1.2. Cómo se refleja . . . 79
12.1.3. Desvíos señalados . . . 79
12.2. IEC 60073: codificación por colores . . . 80
12.3. IEC 60204-1: pulsadores y paros . . . 80
12.4. ISA-18.2 / IEC 62682: gestión de alarmas . . . 81
12.4.1. Qué dice . . . 81
12.4.2. Qué implementa el código . . . 81
12.4.3. Qué falta . . . 81
12.5. IEC 60848: GRAFCET . . . 82
12.6. IEC 61131-3: lenguajes de programación . . . 82
12.7. ISO 1219: símbolos neumáticos . . . 82
12.8. ISO 15552 e ISO 6432: cilindros . . . 83

12.9. IEC 62443 y seguridad de las comunicaciones . . . 83
12.10. Contraste y accesibilidad (WCAG 2.x) . . . 84
12.11. Resumen de desvíos y recomendaciones . . . 84

13. Gemelo digital y enlace web 86
13.1. Arquitectura . . . 86
13.2. El puente . . . 86
13.3. El servidor . . . 88
13.3.1. Una cola por navegador . . . 88
13.3.2. Órdenes desde la web . . . 88
13.4. El frontend 3D . . . 89
13.4.1. La escena . . . 89
13.4.2. Cámara y controles . . . 89
13.4.3. Móvil . . . 90
13.5. Gemelo local . . . 90
13.6. Despliegue en la PC servidor . . . 90

14. Pruebas y latencia 92
14.1. Pruebas automáticas . . . 92
14.1.1. Estrategia . . . 92
14.1.2. Resultado . . . 92
14.1.3. Prueba de humo . . . 92
14.2. Latencia medida en el laboratorio . . . 94
14.2.1. Conclusión . . . 94
14.2.2. Lo que el software hace mientras tanto . . . 95
14.2.3. Mejoras de código y de Ladder propuestas . . . 95

15. Python frente a Delphi y las herramientas RAD 96
15.1. Qué es cada opción . . . 96
15.2. Comparación por criterios . . . 96
15.3. Lo que este proyecto obtuvo de Python . . . 98
15.4. Lo que Delphi habría hecho mejor . . . 98
15.5. Conclusión . . . 99

16. Conclusiones y trabajo futuro 100

16.1. Lo que se logró 100
16.2. Trabajo futuro 100

A. Tabla de variables completa **102**

B. Campos del snapshot **103**

C. Discrepancias detectadas entre documentación y código **104**

D. Glosario **106**

Bibliografía **108**

Índice de figuras

1.1. Vista general: un proceso habla OPC y todos los demás consumen su snapshot. 3

2.1. Cilindro ISO 15552 ø32×200 en sus cuatro estados 7
2.2. Minicilindro ISO 6432 ø20×50 con retorno por muelle 8
2.3. GRAFCET de nivel 2 de la estación 10
2.4. Escalón 0000 (GM) en notación Ladder 11

3.1. Procesos e hilos. El objeto COM vive en un solo hilo del gateway; la GUI nunca espera a la red. 14
3.2. Dependencias entre paquetes: todo depende de `core/`, y `core/` no depende de nada gráfico ni de COM. 15
3.3. Secuencia de un ciclo de lectura y de una orden 17

4.1. Pila de comunicación de punta a punta. 19

7.1. Modelo de tiempos de la cadena neumática 4A/5A 45
7.2. Extrapolación del acumulador de un TON 48

8.1. Mapa de componentes de la pantalla 54
8.2. Las dos variantes del tema, con la planta en marcha en modo SIM. 55
8.3. Ventana de conexión: modo, gateway, selector OPC de cuatro listas, envío al gemelo y consola de subprocesos. 56
8.4. Visor del historial SQLite. 57

9.1. Sistema de coordenadas del cilindro ISO 15552 ø32 60
9.2. Capas del cilindro ISO 15552 ø32 y su composición 61
9.3. Lámina de comprobación del autor: los dos cilindros en varias posiciones, compuestos a partir de las capas. 63
9.4. Botones industriales y LEDs del sistema de diseño 65
9.5. Iconos del sistema de diseño 65

10.1. Curvas de suavizado usadas por el HMI. 68
10.2. Animación de una carrera en modo REAL 70

11.1. Flujo de trabajo con Qt Design Studio y VS Code 76

13.1. Gemelo digital 3D en el navegador 86
13.2. Despliegue del gemelo: el laboratorio solo abre conexiones salientes. 87
13.3. El gemelo en un celular. 90

14.1. Lectura de grupo: dispositivo frente a caché 94

Índice de tablas

1.1. Modos de operación del gateway. 4

2.1. Hardware y software del controlador. 8
2.2. Resumen de la tabla de variables (Ladder del 18 de septiembre). 9
2.3. Mapa del programa Ladder (archivo LAD 2, 26 escalones). 11

3.1. Responsabilidad de cada paquete. 15
3.2. HMI v6.1 frente al HMI actual. 18

4.1. Alternativas consideradas para la comunicación gateway ↔ HMI. 23
4.2. Órdenes del protocolo ZeroMQ (`core/protocolo.py`). 25
4.3. Mensajes del WebSocket `/ws/planta`. 26
4.4. Topics MQTT del enlace (`core/enlace_mqtt.py`). 27
4.5. Resumen de los protocolos del sistema. 29

5.1. Bibliotecas del proyecto y sus versiones 30

6.1. Reglas de alarma (evaluadas en el gateway en cada ciclo). 39

7.1. Fuerzas teóricas a 6 bar 42
7.2. Tiempos de carrera del simulador (`CARRERAS_DEFECTO`). 44
7.3. Fases de cada cilindro en el cursor del ciclo 47

8.1. Argumentos de `ui/app.py`. 51
8.2. Componentes de la pantalla principal. 55

9.1. Tokens de color del sistema de diseño (variante oscura original y variante gris de `ui/qml/tema/Theme.qml`). 64

10.1. Animaciones de los controles. 72

11.1. Equivalencias entre Delphi VCL y Qt Design Studio 77

12.1. Normas aplicables y grado de aplicación. 78

12.2. Colores de IEC 60073 frente a los tokens del HMI. 80
12.3. Contraste WCAG de la paleta 84
12.4. Desvíos normativos y correcciones propuestas (ninguna aplicada; requieren decisión del autor). 85

14.1. Pruebas automáticas 93
14.2. Mediciones del 3 de octubre de 2026 contra el SLC real. 94

15.1. Python + Qt/QML frente a Delphi para HMI. 96

A.1. Tabla de variables del PLC. 102

B.1. Campos del snapshot (protocolo versión 2). 103

C.1. Discrepancias y observaciones (10 de octubre de 2026). 104

Listings

1.1. Carpetas principales del repositorio del HMI 2
1.2. Órdenes de ejecución habituales 4
4.1. Lectura de grupo de caché o de dispositivo (`gateway/backend_opc.py:248-257`) 21
4.2. Publicación del snapshot y de los eventos (`gateway/servidor.py:423-430`) 22
4.3. Estructura del snapshot (versión 2 del protocolo), mostrada como JSON 24
4.4. Forma canónica y firma de una orden (`core/remoto.py:61-71`) 27
6.1. Reglas de seguridad de escritura (`core/tags.py:203-222`) 35
6.2. Pseudocódigo del bucle por plazo 36
6.3. Pulso de 300 ms (`gateway/servidor.py:299-311`) 37
6.4. Esquema SQLite del historial (`core/historial.py:37-43`) 39
6.5. Reglas del árbitro para una escritura web (`core/mando.py:237-246`) 40
6.6. Subcomandos del diagnóstico OPC 40
6.7. Opciones de `python -m gateway` 41
7.1. Paso de la física con válvula biestable (`gateway/fisica.py:51-62`) 44
7.2. TON con acumulador flotante y OSR + CTU (`gateway/ladder_sim.py:103-119`) 45
7.3. Extrapolación del acumulador (`core/suavizado.py:145-154`) 48
8.1. Objetos de contexto del motor QML (`ui/app.py:208-216`) 50
8.2. Un binding típico: la etapa activa del GRAFCET y su barra de temporizador 51
8.3. Vaciado del SUB sin bloquear (`ui/cliente.py:589-604`) 52
8.4. Notificación agrupada de propiedades (`ui/cliente.py:513-521`) 52
8.5. Rejilla de la pantalla (`ui/qml/componentes/RejillaHmi.qml:18-29`) 53
9.1. Contenido de `assets/assets_cilindros/` 59
9.2. Vástago desplazado y muelle comprimido (`CapasCilindro.qml:56-72`) 60
9.3. Parámetros de los generadores (`gen_cilindros.py:18-21`, resumidos) 62
10.1. Las tres animaciones del vástago (`Cilindro.qml:50-57`) 69
10.2. Carrera calibrada y ajuste al extremo en REAL (`Cilindro.qml:65-76`) 69
10.3. Banda de alarmas: aparición y parpadeo (`BandaAlarmas.qml:14-16, 26-30`) 71
11.1. Fragmentos del archivo generado `ui/qml/dummydata/planta.qml` (líneas 1–21, 49–54, 84 y 91) 74

11.2. Proyecto de Qt Design Studio (`ui/qml/HMI.qmlproject`, resumido) 75
13.1. Envío por evento limitado por `hz` (`puente/enlace.py:271-282`) 87
13.2. Cola de salida por navegador (`servidor_web/app.py:94-112`) 88
13.3. Gemelo local 90

CAPÍTULO 1

Introducción

1.1 Propósito del documento

Este documento es la documentación técnica del código del HMI. Tiene dos lectores en mente: el autor y el profesor, que conocen el proceso y la ingeniería de control, y cualquier persona que retome el software y necesite entender por qué está hecho como está. No sustituye a las guías de uso de `docs/`. La instalación en el laboratorio está en `GUIA_INSTALACION_HMI.md`, el modo suave y el envío al gemelo en `GUIA_MODO_SUAVE_Y_GEMELO.md` y la edición visual en `GUIA_QT_DESIGN_STUDIO.docx`. Este documento explica el diseño que hay detrás de esas guías.

En concreto responde:

- qué hace cada paquete y cada módulo, y qué contrato respeta con los demás;
- qué protocolos usa cada eslabón de la cadena PLC → pantalla → navegador, qué viaja por ellos y con qué formato;
- qué bibliotecas se usaron, qué son y por qué esas y no otras;
- cómo se modeló la física de la planta y cómo se estima la posición de los vástagos sin sensores intermedios;
- cómo se construyeron y animaron los gráficos: assets SVG por capas, sistema de diseño y animaciones declarativas de Qt Quick;
- qué normas orientan la interfaz y cómo se reflejan en el código;
- cómo convive el código Python con la edición visual en Qt Design Studio;
- qué ventajas y desventajas tiene este enfoque frente a Delphi y otras herramientas RAD.

1.2 Contexto del proyecto

El proyecto pertenece a la asignatura *Control Distribuido e Integración SCADA* (Departamento de Control y Robótica, División de Ingeniería Eléctrica, Facultad de Ingeniería, UNAM). La planta es una estación electroneumática con cinco cilindros:

- tres de doble efecto (1A, 2A, 3A), mandados por el PLC con válvulas 5/2 biestables;
- dos de simple efecto con retorno por muelle (4A, 5A), que forman una cadena puramente neumática.

El control es un programa Ladder de 26 escalones, escrito en RSLogix 500 para un SLC 5/04 y derivado de un GRAFCET de nueve etapas (capítulo 2).

El enunciado original pide comunicar la planta con una interfaz por OPC DA, típicamente hecha en Delphi. Antes de este software, la interfaz funcional era el HMI v6.1 en Python y Tkinter (`old/hmi_estacion.py`), que ya hablaba OPC DA con RSLinx y se probó con el PLC físico. El nuevo HMI conserva todo lo que v6.1 hacía bien: tres modos de operación, reglas de seguridad, pulsos de 300 ms y lectura en grupo. Sobre esa base mejora tres aspectos:

- **La latencia.** Separa la lectura OPC de la pantalla en dos procesos y lee de la caché de RSLinx.
- **La presentación.** Usa QML con gráficos vectoriales por capas, animación con la duración de carrera calibrada y un sistema de diseño coherente con ISA-101.
- **El alcance.** Añade historial en SQLite, alarmas, árbitro de mando y un gemelo digital accesible desde cualquier navegador.

> **Arquitectura válida también para Delphi** La elección de Python en lugar de Delphi está pendiente del visto bueno del profesor. La arquitectura OPC (servidor `RSLinx OPC Server`, topic `PROYECTO1_TEST`, items `[PROYECTO1_TEST]I:1.0/3`...) es exactamente la misma para ambos lenguajes. El capítulo 15 discute la comparación con detalle.

1.3 Visión general del software

La figura 1.1 resume el sistema. RSLinx Classic actúa a la vez como driver del enlace con el PLC y como servidor OPC DA. El **gateway** es el único proceso que habla COM: lee la tabla de variables en grupo, aplica las reglas de seguridad, cronometra los pulsos y publica un *snapshot* del estado de la planta en cada ciclo. El **HMI** se suscribe a ese snapshot y manda órdenes de vuelta. El **puente** hace lo mismo que el HMI, pero reenvía los datos a un **servidor web** que los difunde a los navegadores del **gemelo digital**.

1.4 Estructura del repositorio

Listing 1.1: Carpetas principales del repositorio del HMI

```
HMI/
|-- core/          Python puro (sin Qt ni COM), compartido por gateway, HMI y puente
|                  tags · protocolo · alarmas · historial · calibracion · estimador
|                  suavizado · mando · remoto · enlace_mqtt · latencia
|-- gateway/       proceso OPC de 32 bits: python -m gateway --modo SIM|EMULATE|REAL
|                  ladder_sim · fisica · backend_sim · backend_opc · servidor · diagnostico_opc
|-- ui/            HMI PySide6/QML: app.py, cliente.py (Planta), conexion.py, historial.py,
|   |-- qml/       Main.qml, PantallaHmi.qml, VentanaConexion.qml, componentes/, tema/,
|   |              HMI.qmlproject + dummydata/ (Qt Design Studio)
|   `-- herramientas/  captura.py, gateway_falso.py, generar_dummydata.py
|-- puente/        enlace saliente gateway -> servidor web (WebSocket o MQTT), vincular.py
|-- digital_twin/  gemelo web: index.html (three.js), servidor_web/ (aiohttp), local.py,
|                  despliegue/ (Caddy, cloudflared, Mosquitto)
|-- assets/        sistema de diseño y gráficos SVG del autor (solo lectura para el código)
|-- tests/         pytest sin COM; las pruebas de ui/ corren con Qt offscreen
```

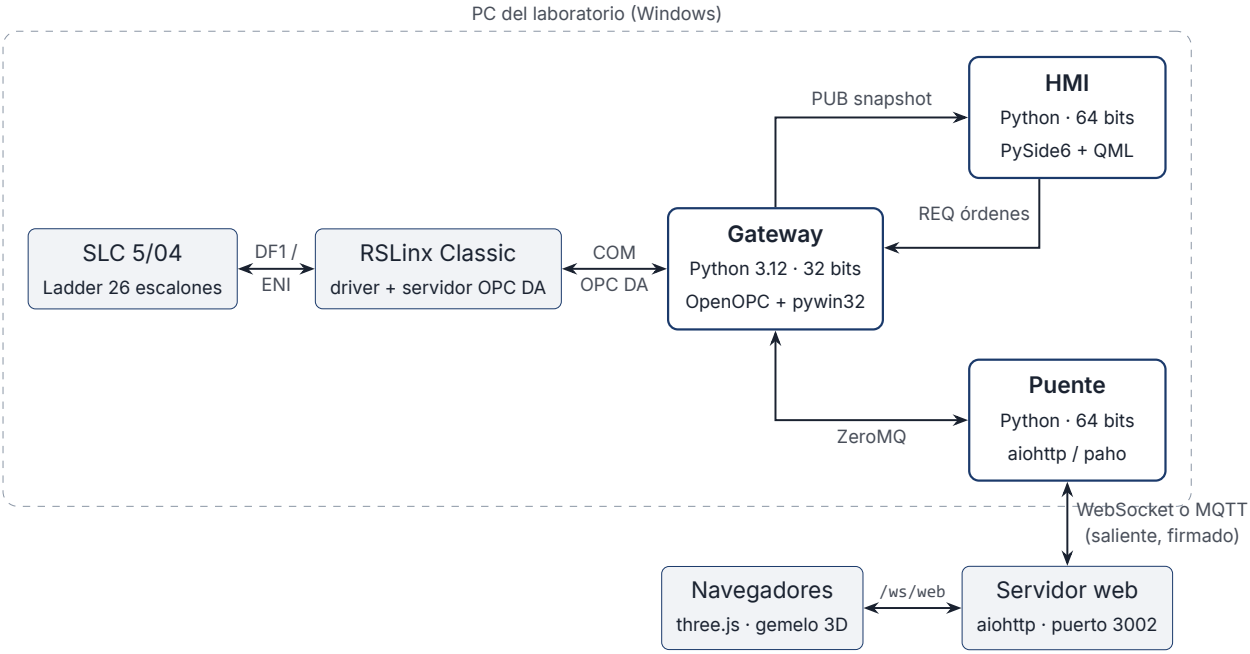


Figura 1.1: Vista general: un proceso habla OPC y todos los demás consumen su snapshot.

```
|-- lab/          lanzadores .bat para la PC del laboratorio
|-- mds/          documentación de contexto y contratos (ARQUITECTURA.md, ENLACE_WEB.md, ...)
|-- docs/         guías para el usuario final y esta documentación técnica
|-- bitacora/     una bitácora por sesión de trabajo
`-- old/          HMI v6.1 en Tkinter (referencia de solo lectura)
```

La regla que más pesa en esta organización está en `mds/REGLAS.md`: *ninguna dirección del PLC se escribe a mano fuera de* `core/tags.py`. Las direcciones son la interfaz entre Ladder, gateway, HMI, puente y gemelo, y una dirección duplicada o distinta entre componentes es el riesgo más alto del proyecto. Por eso todas salen de una sola tabla, que refleja la tabla de variables del Ladder del 18 de septiembre (apéndice A).

1.5 Modos de operación

El sistema hereda de v6.1 tres modos, que cambian el origen de los datos sin cambiar la pantalla:

Regla de seguridad Las reglas heredadas de v6.1 que ya funcionaban con el PLC físico no se negocian. `core/tags.py::validar_escritura` las codifica y el gateway la llama en *toda* escritura:

- el HMI nunca escribe en `O:2`, porque las salidas son autoridad exclusiva del Ladder;
- en modo REAL se bloquea toda escritura en `I:1`;
- los mandos START, STOP y RESET son pulsos de 300 ms (1 → 300 ms → 0) sobre `STARTV`, `STOPV` y `RESETV`, cronometrados en el gateway;

Modo	Origen de los datos	Cilindros	Escrituras permitidas
SIM	Ladder traducido a Python (`gateway/ladder_sim.py`) dentro del gateway. No necesita Windows, COM ni RSLinx.	Modelo físico del gateway (`gateway/fisica.py`); da la posición exacta.	Mandos, parámetros y, en modo manual, sensores simulados.
EMULATE	RSLogix Emulate 500 por RSLinx (driver `EMU500-1`).	No hay planta: el gateway mueve los cilindros según las bobinas leídas y escribe `SC/SE` en `I:1`.	Mandos, parámetros y entradas `I:1` simuladas.
REAL	SLC 5/04 físico por RSLinx (`AB_ETH-2` → 1761-NET-ENI → DF1).	Solo sensores de extremo: la posición intermedia se estima (capítulo 7).	Mandos y parámetros. **Toda escritura en `I:1` está bloqueada.**

Tabla 1.1: Modos de operación del gateway.

- solo se aceptan lecturas con calidad `Good`; tras cinco lecturas fallidas seguidas el gateway se declara desconectado;
- el objeto COM de OpenOPC se usa solo desde el hilo que lo creó;
- al escribir `T_ESPERA` se escriben los cinco preajustes `T4:0.PRE`…`T4:4.PRE` en una sola llamada.

1.6 Cómo se ejecuta

En la máquina de desarrollo solo existe el modo SIM. El intérprete es `.venv\Scripts\python.exe`: Python 3.13 de 64 bits con PySide6, pyzmq, msgpack, loguru, aiohttp, paho-mqtt y pytest.

Listing 1.2: Órdenes de ejecución habituales

```
.venv\Scripts\python.exe -m gateway --modo SIM                    # gateway simulado
.venv\Scripts\python.exe ui\app.py --dev --lanzar-gateway SIM     # HMI + gateway, recarga en caliente
.venv\Scripts\python.exe -m digital_twin.local --gateway SIM      # gemelo web sin internet
.venv\Scripts\python.exe -m gateway --modo SIM --simular-enlace   # imita el enlace lento del
    laboratorio
.venv\Scripts\python.exe -m pytest tests -q                       # pruebas automáticas
```

En el laboratorio se usan los lanzadores de `lab/`:

- `instalar.bat` crea `.venv32` (gateway, 32 bits) y `.venv` (HMI, 64 bits);
- `probar_opc.bat` diagnostica la conexión OPC;
- `gateway.bat` y `hmi.bat` arrancan el gateway y el HMI;
- `gemelo_local.bat` levanta el gemelo en la misma PC;
- `clave_puente.bat` gestiona la clave del puente.

La ventana de conexión del HMI (sección 8.4) también puede lanzar y vigilar todos esos procesos con un solo botón.

CAPÍTULO 2

El sistema controlado

El software no tiene sentido sin la planta y el programa del PLC que supervisa. Este capítulo resume ambos con el detalle que el código necesita: qué señales existen, qué hace cada etapa del GRAFCET y cómo se tradujo a Ladder. La fuente de verdad es la tabla del Ladder del 18 de septiembre (`docs/TABLA_DE_VARIABLES.md`, sección 3), que `core/tags.py` reproduce.

2.1 La planta electroneumática

2.1.1 Cilindros de doble efecto 1A, 2A y 3A

Los tres cilindros principales son de doble efecto: el aire empuja el émbolo tanto al salir como al regresar. Cada uno lo gobierna una electroválvula 5/2 *biestable*, con dos bobinas:

- `EV_En` lleva el carrete a la posición de salida;
- `EV_Cn` lo lleva a la de regreso.

Como el carrete conserva su posición al desenergizar las bobinas (tiene memoria mecánica), basta un pulso para invertir el cilindro. Por eso la física simulada modela el carrete y no solo la bobina (sección 7.2).

Cada cilindro lleva dos sensores magnéticos de ranura, que detectan el imán del émbolo:

- `SCn`: vástago retraído (*cerrado*);
- `SEn`: vástago extendido.

Son los *únicos* sensores de posición que ve el PLC. Lo que pasa entre los dos extremos no se mide, y esta limitación condiciona buena parte del diseño de la animación (capítulos 7 y 10).

2.1.2 Cadena neumática 4A/5A

Los cilindros 4A y 5A son de simple efecto: el aire los saca y un muelle los regresa. Forman una cadena *puramente neumática*, que el PLC no gobierna:

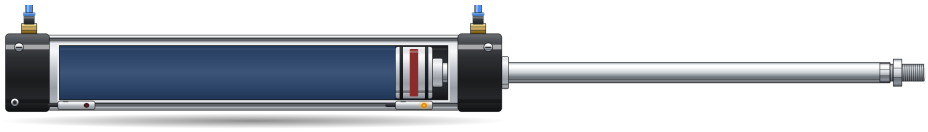
- una memoria neumática (EEV3/CEV3);
- temporizadores neumáticos;
- rodillos de final de carrera E4A, C4A, E5A y C5A;
- una válvula de simultaneidad.



(a) Reposo: vástago retraído, SC encendido.



(b) Saliendo: EV_E energizada, cámara trasera presurizada (azul al 28 %).



(c) Extendido: SE encendido.



(d) Regresando: EV_C energizada, cámara delantera presurizada.

Figura 2.1: Cilindro de doble efecto ISO 15552 ø32×200 compuesto con las capas SVG del HMI, en el mismo orden y con la misma geometría que `CapasCilindro.qml` (generado por `figuras/generar_figuras.py`).

La secuencia de la cadena es: sale 4A, tras un retardo sale 5A y luego ambos regresan por muelle.

El PLC no tiene sensores de 4A ni de 5A. Los «espera» con una ventana de tiempo, el temporizador `T4:5` (5 s), durante la etapa `X7`. El HMI, por tanto, no puede mostrar la posición real de estos cilindros: la estima con un modelo de tiempos (sección 7.3) y lo dice explícitamente en pantalla («sin sensores en el PLC · posición estimada»).

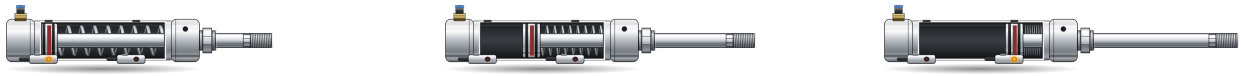


Figura 2.2: Minicilindro ISO 6432 ø20×50 de simple efecto en reposo, a media carrera y extendido. El muelle, dibujado relajado, se comprime escalándolo en x alrededor de su apoyo delantero.

2.2 El controlador

Elemento	Detalle
Procesador	SLC 5/04, 1747-L541C, 16 K palabras, OS401 serie C FRN 3-8. No tiene la instrucción ONS, por eso se usa OSR.
Chasis	1746-A4 (cuatro ranuras).
Ranura 1	1746-IB16: 16 entradas de 24 VCC (*sink*).
Ranura 2	1746-OB16: 16 salidas (*source*).
Ranura 3	1747-SDN, registrado por error en la configuración de E/S (conviene eliminarlo).
Programación	RSLogix 500; archivo principal LAD 2 con 26 escalones.
Emulación	RSLogix Emulate 500, estación 1, driver `EMU500-1` en RSLinx.
Comunicaciones	RSLinx Classic (Gateway, OEM o Single Node; la edición Lite no incluye OPC).
Enlace en el laboratorio	Ethernet, driver `AB_ETH-2`, hacia un 1761-NET-ENI en 192.168.1.80, que convierte a DF1 serie hacia el canal 0 del SLC (verificado el 3 de octubre de 2026).

Tabla 2.1: Hardware y software del controlador.

El HMI direcciona las variables por *dirección física* y no por símbolo. Las equivalencias son `I:1/n` ≡ `Local:1:I.Data.n` y `O:2/n` ≡ `Local:2:O.Data.n`. Esto permite usar la misma tabla con el emulador y con el PLC real, y es lo que exige la sintaxis de items de RSLinx: los bits de E/S llevan la palabra explícita (`I:1.0/3`, `O:2.0/4`).

2.3 Tabla de variables

La tabla completa, con los ItemID de OPC, está en el apéndice A. En resumen:

Las versiones anteriores del programa usaban `N7:0`…`N7:2` para los parámetros, los mandos en `B3:0/13`…`B3:0/15` y las salidas en `O:3`. Ese mapeo, que todavía trae v6.1, es obsoleto, y una de las razones para centralizar la tabla en `core/tags.py` fue precisamente que no volviera a colarse.

Grupo	Direcciones	Contenido
Entradas	`I:1/0`…`I:1/7`	`STOP_1` (pulsador NC: vale 1 en reposo), `START_1`, `SC1`, `SE1`, `SC2`, `SE2`, `SC3`, `SE3`
Salidas	`O:2/0`…`O:2/5`	`EV_E1`, `EV_C1`, `EV_E2`, `EV_C2`, `EV_E3`, `EV_C3`
Mandos y estado	`B3:0/0`…`B3:0/4`	`STARTV`, `STOPV`, `ACTIVE`, `END`, `RESETV`
Etapas	`B3:3/0`…`B3:3/8`	`X0`…`X8` (una sola activa)
Temporizadores	`T4:0`…`T4:5`	`T_ESPERA` (`T4:0.PRE`…`T4:4.PRE`) y `T_NEUM` (`T4:5.PRE`); base de tiempo de 0,01 s
Contador	`C5:0`	`N_CICLOS` (`C5:0.PRE`) y ciclos completados (`C5:0.ACC`)

Tabla 2.2: Resumen de la tabla de variables (Ladder del 18 de septiembre).

2.4 La secuencia en GRAFCET

El GRAFCET (norma IEC 60848) describe un automatismo secuencial con cinco elementos:

- **etapas**, que pueden estar activas o inactivas;
- **acciones**, asociadas a las etapas;
- **transiciones**, que unen las etapas;
- **receptividades**, la condición lógica de cada transición;
- **reglas de evolución**.

Una transición se franquea cuando todas sus etapas anteriores están activas y su receptividad es verdadera. Al franquearse, desactiva las etapas anteriores y activa las siguientes. La secuencia de la estación es lineal, con una divergencia en O al final para repetir o terminar (figura 2.3).

Sobre la secuencia actúan dos GRAFCET auxiliares, implementados como escalones independientes:

GM (marcha)
: $\text{ACTIVE} = (\text{START_1} + \text{STARTV} + \text{ACTIVE}) \cdot \text{STOP_1} \cdot \overline{\text{STOPV}}$. Es un enclavamiento clásico de marcha y paro. `STOP_1` es un pulsador NC, así que vale 1 en reposo y la serie se abre al pulsarlo.

GS (seguridad)
: $\text{S:1/15} + \text{END} + (\text{START_1} \cdot \overline{\text{STOP_1}}) + \text{RESETV} \Rightarrow$ `MOV 1` $\rightarrow$ `B3:3`, `RES C5:0`. En el primer escaneo tras pasar a RUN, al terminar los N ciclos, al pulsar Start y Stop a la vez, o con un RESET desde el HMI, la secuencia vuelve a X0 y el contador a cero.

Con STOP la etapa se *congela*: las transiciones exigen `ACTIVE`, pero las salidas no dependen de él. Al pulsar START la secuencia continúa donde estaba. Es un paro funcional, no de emergencia. Los TON siguen contando durante el paro, un detalle que el modo suave del HMI tiene en cuenta al extrapolar el acumulador (sección 7.6).

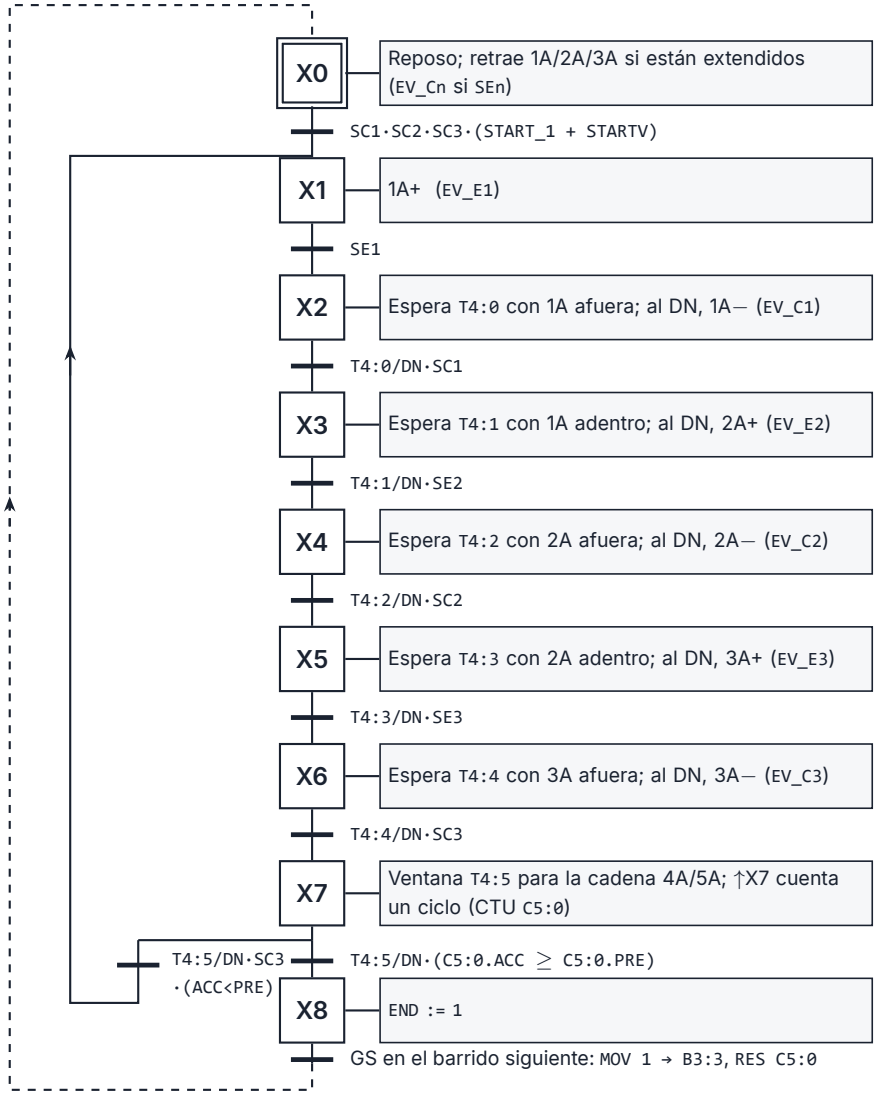


Figura 2.3: GRAFCET de nivel 2 de la estación (Ladder del 18 de septiembre). Todas las transiciones llevan además la condición `ACTIVE`. La flecha discontinua es el GRAFCET de seguridad (GS), que fuerza X0.

2.5 Del GRAFCET al Ladder

El Ladder implementa el GRAFCET con el método de *etapas como bits de memoria*. Cada etapa es un bit de `B3:3`. Cada transición es un escalón de la forma `XIC Xn · XIC ACTIVE · receptividad → OTU Xn, OTL Xn+1`, que desenclava la etapa actual y enclava la siguiente. Las acciones son escalones de salida que combinan etapas y bits `DN` de los temporizadores. Por ejemplo, para el cilindro 1A:

$$\text{EV_E1} = X1 + X2 \cdot \overline{\text{T4:0/DN}} \tag{2.1}$$

$$\text{EV_C1} = X2 \cdot \text{T4:0/DN} + X3 \cdot \overline{\text{T4:1/DN}} + X0 \cdot \text{SE1} + X7 \cdot \text{T4:5/DN} \cdot \text{SE1} \tag{2.2}$$

2A y 3A siguen la misma estructura. El bit `DN` separa en el tiempo las dos bobinas de una válvula, de modo que *nunca* se energizan a la vez. Esta propiedad la verifican las pruebas del simulador (`tests/test_ladder_sim.py`) y la demostración del gateway (`--ciclos-demo`, que cuenta conflictos EV_E·EV_C y falla si hay alguno).

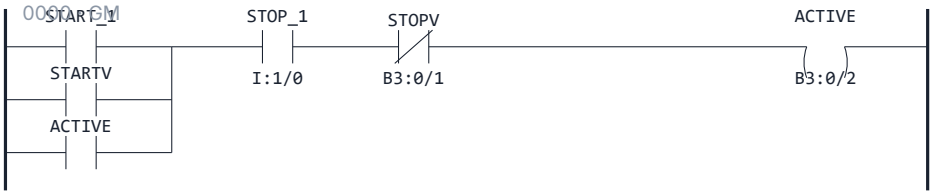


Figura 2.4: Escalón 0000 (GM): tres contactos NA en paralelo (`START_1`, `STARTV` y el propio `ACTIVE`, que enclava), en serie con `STOP_1` (NA sobre un pulsador NC) y `STOPV` (contacto NC, XIO), y la bobina `ACTIVE`.

Escalones	Bloque
0000	GM: `ACTIVE`
0001	GS: `MOV 1 → B3:3`, `RES C5:0`
0002 – 0008	Transiciones X0→X7 (`Xn·ACTIVE·receptividad → OTU Xn, OTL Xn+1`)
0009 – 0010	Divergencia en O tras X7 (comparaciones `LES`/`GEQ` de `C5:0`)
0011 – 0016	TON `T4:0`…`T4:5`
0017	`X7 → OSR B3:2/0 → CTU C5:0`
0018 – 0023	Salidas `EV_E1`…`EV_C3`
0024	`X8 → END`
0025	Fin de archivo

Tabla 2.3: Mapa del programa Ladder (archivo LAD 2, 26 escalones).

> **`END` dura un barrido** En el PLC, END $= X8$ y el GS fuerza X0 en el barrido siguiente, así que el bit `END` casi nunca se alcanza a leer por OPC: dura unos milisegundos y el sondeo es de cientos. El gateway no se fía de ese bit. Detecta el fin de los N ciclos por el paso X7 → X0 sin RESET ni START·$\overline{\text{STOP}}$, incrementa el contador `fines` del snapshot y emite el evento «Fin de N ciclos (END)» (`gateway/servidor.py`).

2.5.1 Pendientes conocidos del programa

La documentación de contexto registra tres puntos del Ladder por confirmar, que el software no puede corregir porque son del programa del PLC:

1. **Escalón 0001.** Estaba como `XIC I:1/0 · XIO I:1/1`, que es verdadero en reposo y deja la secuencia pegada en X0. Lo correcto es `XIC I:1/1 · XIO I:1/0` (ambos pulsados).
2. **Escalón 0002.** La rama del pulsador usaba `I:1/0` (Stop); debe ser `I:1/1` (Start) en paralelo con `STARTV`.
3. **Preajustes en cero.** En la impresión del 18 de septiembre estaban en 0, y con 0 el bit `DN` se enciende al instante. Deben cargarse `T4:0`…`T4:4.PRE` = 100, `T4:5.PRE` = 500 y `C5:0.PRE` = 3. El HMI puede escribirlos, pero conviene dejarlos en el archivo de datos.

Además, en el laboratorio (3 de octubre) el PLC tenía descargado un programa distinto: `B3` de una sola palabra, `T4` con cuatro temporizadores y preajustes en 0. El gateway lo detecta al conectar y lo reporta (sección 6.3.2).

CAPÍTULO 3

Arquitectura del software

3.1 Los requisitos que la definen

La arquitectura no salió de una preferencia estética. Salió de cinco restricciones concretas, verificadas al arrancar el proyecto (bitácora de la sesión 01, 23 de septiembre de 2026):

1. **OPC DA es COM de 32 bits.** RSLinx Classic expone sus datos por OPC Data Access, una tecnología basada en COM/DCOM de Microsoft. El proceso cliente debe ser de 32 bits y estar en la misma máquina que RSLinx. En Python eso significa un intérprete de 32 bits con `pywin32` y `OpenOPC`.
2. **PySide6 no publica ruedas de 32 bits.** Se verificó en PyPI el 23 de septiembre: ni PySide6 ni PyQt6 tienen paquetes `win32`. Un HMI con Qt 6 y QML necesita Python de 64 bits. Solo PyQt5 5.15 (Qt 5.15) tiene ruedas de 32 bits para Python 3.12.
3. **Una lectura lenta no puede congelar la pantalla.** En v6.1 la lectura OPC, el `sleep` y el refresco de Tkinter compartían el mismo ritmo, y el retardo sensor → pantalla era de 0,4 a 0,8 s. Con el enlace DF1 del laboratorio una lectura desde el dispositivo llegó a tardar 2,2 s.
4. **La misma pantalla para tres orígenes de datos.** SIM, EMULATE y REAL deben ser indistinguibles para la GUI, como ya ocurría en v6.1 con `SimBackend` y `DaBackend`.
5. **Acceso web sin abrir puertos en el laboratorio.** El gemelo digital debe verse desde fuera, pero la red de la universidad no permite conexiones entrantes a la PC del laboratorio.

La solución que satisface las cinco es la de la figura 3.1:

- un **gateway** de 32 bits que es el único que habla COM;
- un **HMI** de 64 bits que solo habla ZeroMQ;
- un **puente** que abre una conexión *saliente* hacia el servidor web.

3.2 Por qué dos procesos y no dos hilos

La alternativa obvia habría sido un solo programa con un hilo para OPC y otro para la GUI, como v6.1. No es posible por la restricción 2: el proceso que carga PySide6 es de 64 bits y no puede cargar el servidor COM de 32 bits de RSLinx en su espacio de memoria. Pero aun sin esa restricción, separar los procesos tiene ventajas propias:

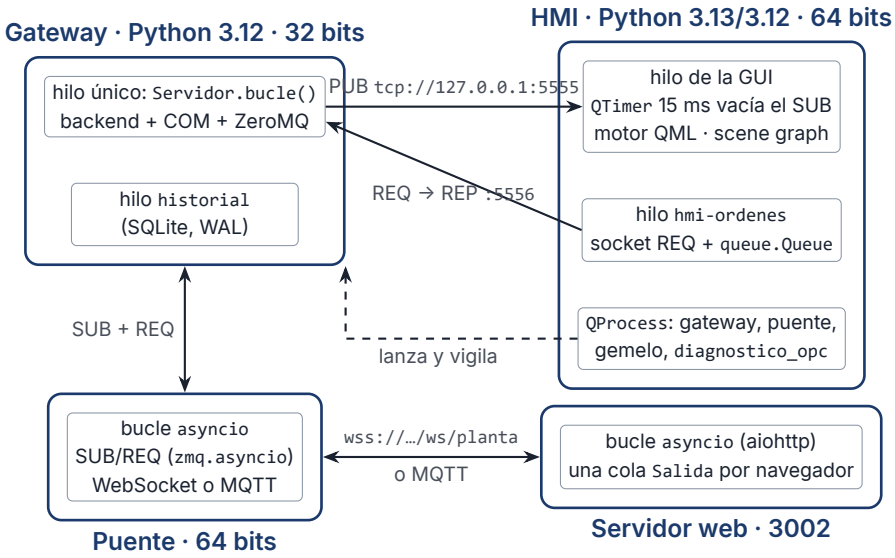


Figura 3.1: Procesos e hilos. El objeto COM vive en un solo hilo del gateway; la GUI nunca espera a la red.

Ventajas de separar el gateway del HMI

- **Aislamiento de fallos.** Si OpenOPC o RSLinx se cuelgan, la pantalla sigue viva y lo indica. Si el HMI se cierra, el gateway sigue leyendo y el gemelo web sigue recibiendo datos.
- **Varios clientes a la vez.** El HMI, el puente del gemelo y cualquier herramienta de diagnóstico se suscriben al mismo snapshot. Con OPC, cada cliente abriría sus propios grupos en RSLinx y multiplicaría el tráfico por el enlace serie, que es el cuello de botella medido (capítulo 14).
- **Un solo lugar para las reglas de seguridad.** Toda escritura, venga del HMI o de la web, pasa por `core/tags.py::validar_escritura` dentro del gateway y por el árbitro de mando.
- **Pruebas sin hardware.** El backend SIM implementa la misma interfaz que el backend OPC, así que todo lo que hay por encima (servidor, HMI, puente, gemelo) se prueba con pytest sin Windows ni COM.
- **Distribución.** El HMI puede correr en otra máquina apuntando a `--pub tcp://IP:5555`; basta con abrir el puerto.

3.3 Paquetes y dependencias

El código se reparte en paquetes con dependencias en un solo sentido (figura 3.2). `core/` es Python puro: no importa Qt ni COM, y lo comparten todos. Gracias a eso, las mismas reglas (tabla de tags, protocolo, alarmas, estimador, modo suave) corren en el gateway de 32 bits, en el HMI de 64 bits y en el puente.

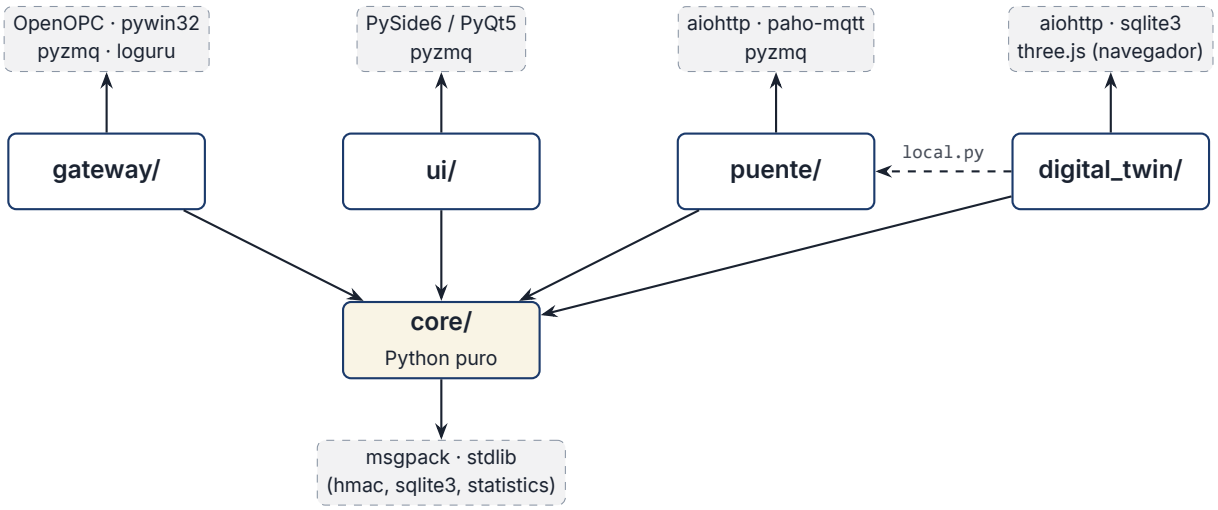


Figura 3.2: Dependencias entre paquetes: todo depende de `core/`, y `core/` no depende de nada gráfico ni de COM.

Paquete	Python	Responsabilidad
`core/`	3.12–3.13, 32 o 64 bits	Contratos (tags, protocolo), alarmas, historial SQLite, calibración, estimador de carrera, modo suave, árbitro de mando, firma HMAC, transporte MQTT, estadísticas de latencia.
`gateway/`	3.12, 32 bits (OPC)	Bucle por plazo, backends SIM y OPC, Ladder simulado, física, pulsos, publicación del snapshot, diagnóstico OPC.
`ui/`	3.12+, 64 bits	Objetos Qt (`Planta`, `Conexion`, `Historial`), QML de la pantalla, ventana de conexión, herramientas de captura y datos de diseño.
`puente/`	3.12+, 64 bits	Enlace saliente gateway ↔ servidor web: reenvío de snapshots, verificación de órdenes firmadas, modo suave para la web, vinculación de la clave.
`digital_twin/`	3.12+, 64 bits	Servidor aiohttp (cuentas, roles, auditoría, difusión), frontend 3D, gemelo local en un proceso, archivos de despliegue.

Tabla 3.1: Responsabilidad de cada paquete.

3.4 Contratos

Tres archivos fijan las interfaces entre módulos. Cambiarlos exige actualizar primero la documentación (`mds/ARQUITECTURA.md`) y la bitácora, porque varias sesiones de trabajo los usan en paralelo:

`core/tags.py`
: La tabla de variables: direcciones, grupos de lectura, pulsos, parámetros virtuales (`T_ESPERA` escribe cinco direcciones), reglas de escritura, definición de etapas y cilindros. Es la única fuente de direcciones.

`core/protocolo.py`
: El formato de los mensajes ZeroMQ: snapshot, evento, órdenes y respuestas; endpoints por defecto; versión del protocolo (2); tiempo de espera de una orden (`TIMEOUT_CMD_MS` = 2000 ms).

`mds/ARQUITECTURA.md`
: El contrato de comportamiento: qué hace el bucle del gateway en cada ciclo, qué propiedades expone `Planta` a QML, qué hace la ventana de conexión, cómo se integra Qt Design Studio.

3.5 Un ciclo de punta a punta

La figura 3.3 sigue un ciclo del gateway y una orden del operador. Lo importante es que *nada espera a nada que no necesite*:

- el gateway publica en cada ciclo aunque no haya cambios;
- el HMI aplica el último snapshot en su próximo cuadro;
- las órdenes viajan por un canal independiente con respuesta.

3.6 Comparación con el HMI v6.1

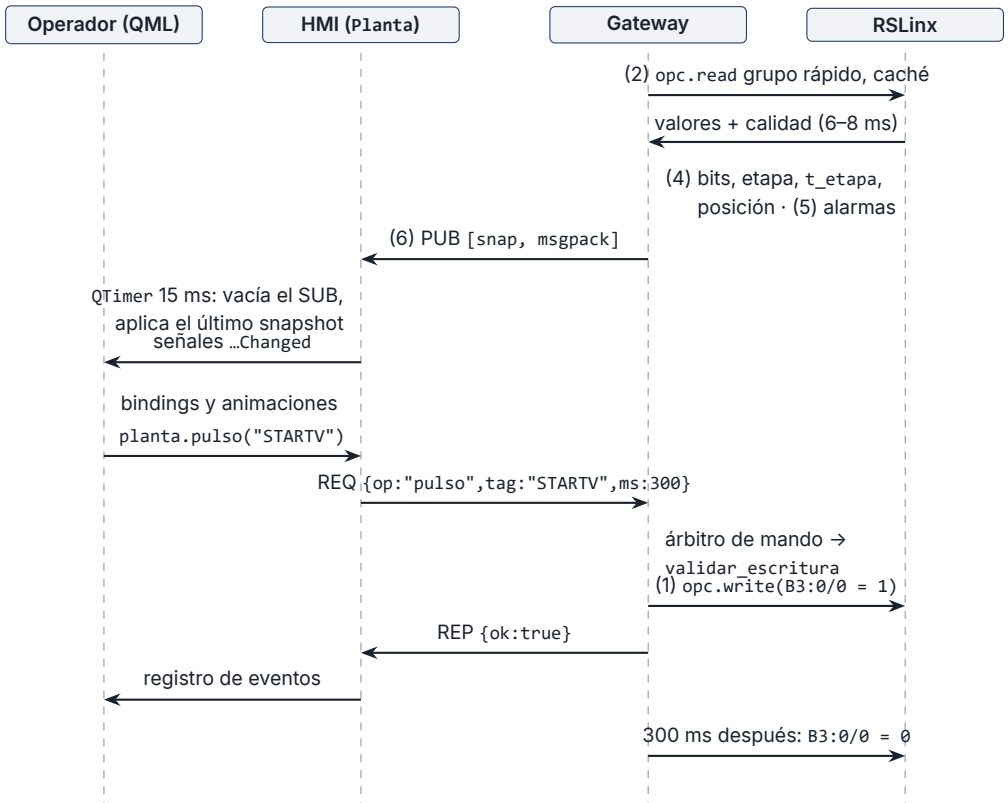


Figura 3.3: Secuencia de un ciclo de lectura y de una orden de START. Los números corresponden a los pasos del bucle del gateway (sección 6.2).

Aspecto	v6.1 (Tkinter, `old/`)	HMI actual
Procesos	Uno: GUI + worker COM en un hilo.	Gateway de 32 bits + HMI de 64 bits + puente, por ZeroMQ.
Ritmo de lectura	Lectura + `sleep(150 ms)`; periodo real de 350–500 ms.	Planificador por plazo (50 ms en SIM, 100 ms en OPC), sin `sleep` tras leer.
Fuente OPC	Siempre el dispositivo (`sync=True`).	Caché de RSLinx por defecto (6–8 ms frente a 200–2238 ms medidos).
Pulsos	Cronometrados en Tkinter, de duración variable.	Cronometrados en el gateway: 1 inmediato, 0 a los 300 ms.
Refresco de pantalla	Tick de Tkinter cada 150 ms.	`QTimer` de 15 ms y scene graph de Qt Quick con animación entre muestras.
Mapeo	Viejo (Start en `I:1.0/0`, salidas en `O:3`, parámetros en `N7`).	Tabla del 18 de septiembre en un solo archivo.
Posición de los vástagos	Igual a la del simulador; en REAL, sin estimación.	Estimador con carreras calibradas (mediana de 5) y modo suave predictivo.
Cadena 4A/5A	No se representaba.	Modelo de tiempos y cuenta regresiva de `T4:5`.
Persistencia	Ninguna.	SQLite: eventos, alarmas, escrituras; visor y exportación CSV.
Alarmas	No.	Cinco reglas con dos niveles y banda roja.
Acceso remoto	No.	Gemelo 3D web con cuentas, roles, auditoría y órdenes firmadas.
Pruebas	Manuales.	229 pruebas pytest (sección 14.1).

Tabla 3.2: HMI v6.1 frente al HMI actual.

CAPÍTULO 4

Protocolos de comunicación

Entre un sensor de la planta y un píxel en el navegador de un celular hay siete eslabones de comunicación. Cada uno usa un protocolo distinto, elegido por una razón concreta. La figura 4.1 los muestra en orden y este capítulo los recorre uno por uno: qué es cada protocolo, qué viaja por él en este sistema y por qué se eligió.



Figura 4.1: Pila de comunicación de punta a punta.

4.1 DF1 y el enlace con el PLC

Qué es. DF1 es el protocolo serie de Allen-Bradley para la familia SLC y PLC-5. En modo *full duplex*, cada trama lleva delimitadores, la carga en formato PCCC (comandos de lectura y escritura de archivos de datos: `N7`, `B3`, `T4`…) y un control de error CRC-16. El receptor confirma cada trama con `ACK` o la rechaza con `NAK`. Si no llega confirmación dentro del tiempo de espera (por defecto 1 s), el emisor la repite.

Cómo está en el laboratorio. La PC no habla DF1 directamente. RSLinx usa el driver Ethernet `AB_ETH-2` hacia un módulo 1761-NET-ENI (192.168.1.80), que convierte a DF1 serie hacia el canal 0 del SLC 5/04. Se verificó el 3 de octubre de 2026 con el topic `PROYECTO1_TEST`, *Polled Messages* a 1000 ms y un *ping* al ENI de 1 ms.

Por qué importa al software. A 19 200 bps, con 10 bits por byte (arranque, 8 de datos y parada), cada byte tarda

$$t_{\text{byte}} = \frac{10 \text{ bits}}{19\,200 \text{ bit/s}} \approx 0{,}52 \text{ ms}. \tag{4.1}$$

Una transacción de lectura (petición, respuesta y ACK) ronda los 35 a 90 bytes, es decir 20–50 ms de cable. Sin embargo, en el laboratorio se midieron lecturas individuales desde el dispositivo de 404, 1214, 1422, 2416 y 2441 ms. Son escalones de casi exactamente 1 s, la firma de los *reintentos* DF1 por tiempo de espera del ACK. La conclusión del diagnóstico (`docs/DIAGNOSTICO_CONEXION_PLC.md`) es que la latencia que queda es del enlace serie ENI $\leftrightarrow$ canal 0, no del software. Hay que revisar en RSLogix el *Channel Status* (ACK timeouts, NAKs, retries), la configuración del canal (DF1 full duplex, 19 200, CRC), que la velocidad del ENI coincida y que ninguna otra PC tenga RSLinx contra el mismo ENI.

4.2 OPC DA

4.2.1 Qué es

OPC *Data Access* (OPC DA) es una especificación de la OPC Foundation, de finales de los noventa, para que cualquier programa de Windows lea y escriba variables de cualquier controlador sin conocer su protocolo nativo. Está construida sobre COM/DCOM, el modelo de objetos binarios de Microsoft. El fabricante entrega un *servidor OPC*, aquí RSLinx Classic, que habla el protocolo del PLC. Los clientes ven una jerarquía de tres niveles:

Servidor
: `RSLinx OPC Server`, o `RSLinx Local OPC Server` / `RSLinx Remote OPC Server` según la instalación. El gateway usa el que liste `probar_opc --servidores`.

Grupo
: Un conjunto de items que el cliente lee junto, con su propia tasa de refresco (*update rate*). El gateway crea tres: `rapido`, `lento` y, solo al conectar, `validar`.

Item
: Una variable, con un ItemID de la forma `[TOPIC]dirección`, por ejemplo `[PROYECTO1_TEST]I:1.0/3` o `[PROYECTO1_TEST]T4:5.PRE`. El *topic* se configura en RSLinx (DDE/OPC → Topic Configuration) y asocia un nombre a un driver y una estación.

Cada valor leído llega con una **calidad** (`Good`, `Bad`, `Uncertain`) y una marca de tiempo. El gateway solo acepta valores con calidad `Good`: un item que pasa a `Bad` conserva su último valor bueno pero se descuenta en el contador `calidad` del snapshot («Good 22/24»).

4.2.2 Caché o dispositivo

OPC DA permite leer de dos fuentes:

- **dispositivo** (`source="device"`): el servidor va al PLC en cada lectura y el cliente espera la respuesta;
- **caché** (`source="cache"`): el servidor devuelve lo último que trajo con su propio sondeo, que mantiene al ritmo `update` del grupo.

v6.1 leía siempre del dispositivo. En el laboratorio eso dejaba al gateway bloqueado entre 200 y 2238 ms por lectura: 19 snapshots en 16 s. Con la caché, la lectura tarda 6–8 ms y se obtienen unos 9 snapshots por segundo (124 en 14 s). Desde la sesión 10 la caché es la fuente por defecto, y se puede cambiar en caliente con la orden `fuente`.

Listing 4.1: Lectura de grupo de caché o de dispositivo (`gateway/backend_opc.py:248-257`)

```
        # OpenOPC: `source` decide de dónde lee el servidor. "device" fuerza el viaje al PLC
        # en cada llamada; "cache" devuelve lo último que RSLinx trajo con su propio sondeo,
        # cuyo ritmo fija `update` (ms). `sync=False` NO es leer de caché: es AsyncIO.Read,
        # que también va al dispositivo. Por eso ambas variantes son síncronas.
        if self.fuente == "dispositivo":
            crudos = self.opc.read(self._items_rapidos, group=GRUPO_RAPIDO,
                                   sync=True, source="device")
        else:
            crudos = self.opc.read(self._items_rapidos, group=GRUPO_RAPIDO,
                                   update=self.periodo_ms, sync=True, source="cache")
```

Trampa de RSLinx: un item inexistente contamina el paquete RSLinx junta en un solo paquete DF1 los items del mismo archivo de datos (`B3`, `T4`…). Si *uno* no existe en el PLC (por ejemplo `B3:3` cuando `B3` tiene una sola palabra), el paquete entero vuelve `Bad`, también los items que sí existen. Además ocurre para *todos* los clientes de RSLinx mientras el item siga en algún grupo, y el grupo de un gateway terminado a la fuerza sigue vivo unos 6 minutos.
Por eso `BackendOPC.conectar()` valida la tabla al conectar:

1. lee los 24 items una vez desde el dispositivo en el grupo `validar` y lo elimina;
2. prueba uno a uno, hasta dos veces, los que no salieron `Good`;
3. quita del sondeo los elementos que el PLC no tiene y los reporta como alarma («El PLC no tiene B3:3, T4:4…»).

Si ningún item sale `Good`, la conexión falla y el servidor reintenta.

4.2.3 OpenOPC y el requisito de 32 bits

OpenOPC (paquete `OpenOPC-Python3x`) es un cliente OPC DA para Python. Por debajo usa la interfaz de automatización `OPC DA Automation Wrapper` (`gbda_aut.dll`), registrada en `C:\Windows\SysWOW64`, que `pywin32` invoca por COM. Como el servidor de RSLinx y esa DLL son de 32 bits, el intérprete también tiene que serlo. Hay otras dos condiciones:

- el objeto COM solo puede usarse desde el hilo que lo creó; el gateway llama a `pythoncom.CoInitialize()` en su único hilo antes de crear el cliente;

- OpenOPC se importa *dentro* de `conectar()`, no al principio del módulo, para que el modo SIM funcione en cualquier sistema sin COM.

4.2.4 Escrituras

Las escrituras usan `opc.write([(item, valor), …])` y comparan cada estado devuelto con `"Success"`. El backend aplica tres capas de protección:

- rechaza `O:` siempre;
- rechaza `I:` en REAL;
- rechaza los elementos que la validación encontró faltantes.

`T_ESPERA` escribe sus cinco preajustes en una sola llamada. En EMULATE, los sensores simulados se escriben primero bit a bit; si RSLinx rechaza la escritura de bits sueltos, se hace lectura-modificación-escritura de la palabra `I:1.0` con máscara, igual que en v6.1.

4.3 ZeroMQ: el bus entre procesos

4.3.1 Qué es

ZeroMQ (ØMQ, biblioteca `libzmq`; en Python, `pyzmq`) es una biblioteca de mensajería *sin broker*. Ofrece «sockets» de alto nivel que transportan mensajes completos, no flujos de bytes, y que implementan patrones de comunicación: publicación/suscripción, petición/respuesta, colas y tuberías. Los sockets se reconectan solos, encolan mensajes mientras el otro extremo no está y funcionan igual sobre TCP, IPC o memoria (`inproc://`).

4.3.2 Los dos patrones usados

PUB/SUB para el estado
: El gateway hace `bind` de un socket PUB en `tcp://127.0.0.1:5555` y publica, en cada ciclo, un mensaje de dos partes `[b"snap", msgpack(snapshot)]` y uno `[b"evt", msgpack(evento)]` por cada evento. Cada suscriptor (HMI, puente) se suscribe a los prefijos `snap` y `evt`. El publicador no sabe cuántos suscriptores hay ni los espera: si uno es lento, ZeroMQ descarta lo que exceda su marca de agua (`SNDHWM`/`RCVHWM` = 1000). El HMI se queda con el último snapshot y con todos los eventos.

REQ/REP para las órdenes
: El gateway hace `bind` de un socket REP en `:5556`. Cada cliente usa un socket REQ con `REQ_RELAXED` y `REQ_CORRELATE`, que permiten reintentar tras un tiempo de espera sin que el socket quede bloqueado en el estado «esperando respuesta», y `RCVTIMEO` = `SNDTIMEO` = 2000 ms. Cada orden recibe una respuesta `{ok, msg, t_ms}`.

Listing 4.2: Publicación del snapshot y de los eventos (`gateway/servidor.py:423-430`)

```
        # (6) publicar
        self.seq += 1
        snap["seq"] = self.seq
        self.snapshot = snap
        self.pub.send_multipart([protocolo.TOPIC_SNAP, protocolo.empaquetar(snap)])
        eventos_ciclo, self._eventos = self._eventos, []
        for ev in eventos_ciclo:
            self.pub.send_multipart([protocolo.TOPIC_EVT, protocolo.empaquetar(ev)])
```

Por qué 2000 ms de espera para una orden Hasta la sesión 10 el tiempo de espera era de 500 ms. En el laboratorio, una escritura OPC por el enlace DF1 tardó entre 100 y 1200 ms, y la orden podía esperar además a que terminara la lectura en curso. Con 500 ms el HMI daba por perdidas órdenes que el PLC sí recibía: un pulso se confirmó `ok` a los 929 ms y el bit apareció en el snapshot a los 1043 ms. El hilo de órdenes del HMI es independiente de la pantalla, así que esperar más no la congela.

4.3.3 Por qué ZeroMQ

Alternativa	Qué ofrece	Por qué no
Sockets TCP crudos	Control total.	Habría que escribir el enmarcado de mensajes, la reconexión, el multiplexado de varios clientes y el patrón publicador/suscriptor.
Qt Remote Objects / D-Bus	Integración con Qt.	El gateway no usa Qt (corre en 32 bits sin PySide6), y `core/` debe seguir siendo Python puro.
gRPC / HTTP	Esquemas formales, ecosistema.	Más dependencias, peor ajuste a la difusión periódica de estado; gRPC no tiene ruedas oficiales para Windows de 32 bits.
MQTT local	Publicación/suscripción con broker.	Requiere un proceso broker más en la PC del laboratorio. Se usa solo donde aporta: entre el laboratorio y el servidor web, a través de internet.
Memoria compartida	Latencia mínima.	Sincronización manual, sin red, sin varios lectores sencillos y frágil entre 32 y 64 bits.
ZeroMQ	PUB/SUB y REQ/REP listos, sin broker, reconexión automática, ruedas para `win32` y `win_amd64`.	—

Tabla 4.1: Alternativas consideradas para la comunicación gateway ↔ HMI.

4.3.4 Un detalle de Windows: la granularidad del reloj

En Windows, `zmq.Poller.poll(timeout)` redondea la espera al tic del sistema, unos 15,6 ms. Con un periodo objetivo de 50 ms eso introduce hasta un tercio de periodo de error. El gateway lo corrige de dos formas (`gateway/servidor.py`):

- mientras corre, activa `timeBeginPeriod(1)` de `winmm.dll`;
- el tramo final antes de cada plazo (los últimos ∼32 ms) lo espera con `poll(0)` y `time.sleep(0.001)`.

Con eso el percentil 95 del periodo queda en el periodo nominal más 0,5 ms.

4.4 msgpack: el formato de los mensajes

Qué es. MessagePack es un formato de serialización binario con el mismo modelo de datos que JSON: mapas, listas, cadenas, números, booleanos y nulo. Es más compacto y más rápido de codificar. Un entero pequeño ocupa un byte; un booleano, un byte.

Por qué. El snapshot se publica entre 10 y 20 veces por segundo y contiene decenas de valores. msgpack lo empaqueta sin definir un esquema aparte (como exigiría Protocol Buffers), su implementación en C tiene ruedas para Windows de 32 y de 64 bits, y se lee igual en Python que en cualquier otro lenguaje. El protocolo usa `msgpack.packb(obj, use_bin_type=True)` y `msgpack.unpackb(datos, raw=False)`, de modo que las cadenas viajan como UTF-8 y vuelven como `str`.

4.4.1 El snapshot

Listing 4.3: Estructura del snapshot (versión 2 del protocolo), mostrada como JSON

```
{ "v": 2, "seq": 1532, "t": 1760054373.21, "modo": "REAL", "conectado": true,
  "fuente": "cache", "calidad": {"good": 24, "total": 24},
  "bits":  {"STOP_1": true, "SC1": false, "SE1": true, "EV_E1": true, "ACTIVE": true, "X2": true, ...},
  "ints":  {"T0_ACC": 37, "T0_PRE": 100, "T0_DN": 0, ..., "C_ACC": 1, "C_PRE": 3},
  "palabras": {"I:1.0": 69, "O:2.0": 1, "B3:0": 4, "B3:3": 4},
  "etapa": 2, "t_etapa": 0.41,
  "pos": null, "pos_neum": [0.0, 0.0],
  "lat": {"lectura_ms": 6.8, "lectura_max_ms": 9.1, "ciclo_ms": 7.5,
          "periodo_ms": 101.2, "p95_ms": 8.4, "fuente": "cache"},
  "alarmas": [ {"id": "calidad", "nivel": "aviso", "msg": "...", "t_ini": ..., "activa": true} ],
  "escrituras": {"pendientes": 0, "ultima": {"tag": "STARTV", "valor": 1, "ok": true, "quien": "HMI
    local"}},
  "fines": 0,
  "mando": {"remoto_habilitado": false, "titular": {...}, "vence_en_s": 52, "enlace_web": true, ...} }
```

`pos` vale `null` en REAL porque el gateway no conoce la posición intermedia. El HMI y el puente la estiman (capítulo 7). En SIM y EMULATE la da el modelo físico del gateway.

4.4.2 Órdenes y respuestas

Orden (`op`)	Campos	Efecto
`pulso`	`tag`, `ms`=300	Escribe 1 en `STARTV`, `STOPV` o `RESETV` y programa el 0.
`escribir`	`tag`, `valor`	Escribe un parámetro (`T_ESPERA`, `T_NEUM`, `N_CICLOS`); fuerza la relectura del grupo lento.
`entrada`	`tag`, `valor`	Simula un pulsador o un sensor (solo SIM y EMULATE).
`fisica`	`auto`	Física automática o sensores manuales (solo local, no en REAL).
`fuente`	`fuente`	Cambia la fuente OPC entre caché y dispositivo.
`ping`, `estado`	—	Comprobación; `estado` devuelve el snapshot completo.
`mando`	`accion`, `habilitar`	Pedir o liberar el mando; habilitar el mando remoto (solo local).
`presencia`	`conectados`	Lista de usuarios web conectados (la manda el puente).

Tabla 4.2: Órdenes del protocolo ZeroMQ (`core/protocolo.py`).

Cualquier orden puede llevar un campo `quien = {usuario, rol, origen: "web"}`. Sin él, la orden es del HMI local, que tiene el nivel más alto en el árbitro de mando (sección 6.6).

4.5 WebSocket: el enlace saliente con el servidor

Qué es. WebSocket (RFC 6455) es un protocolo que empieza como una petición HTTP con la cabecera `Upgrade: websocket` y, si el servidor responde `101 Switching Protocols`, convierte la conexión TCP en un canal bidireccional de mensajes. Atraviesa los mismos proxies y cortafuegos que HTTPS (puerto 443, `wss://`).

Por qué. La PC del laboratorio no admite conexiones entrantes. El puente abre una conexión *saliente* hacia `wss://digitaltwinscada.smartsensorslab.com/ws/planta`, como haría un navegador, y por ella viajan en ambos sentidos los snapshots, los eventos y las órdenes. El servidor queda detrás de Caddy o de un túnel de Cloudflare (sección 13.6).

Autenticación. El puente manda `Authorization: Bearer <clave>` con la clave compartida de `datos/puente.clave` (32 bytes aleatorios en base64url). El servidor la compara con `hmac.compare_digest`, que tarda lo mismo con cualquier clave y no revela por tiempo cuántos caracteres acertó. Si no coincide, responde 401 y lo registra en la auditoría. Solo hay una planta a la vez: un puente nuevo reemplaza al anterior, que se cierra con el código 4000.

Ambos extremos usan además el latido de protocolo de aiohttp (`heartbeat=20` s) y un tamaño máximo de mensaje de 4 MiB. Si la conexión cae, el puente reintenta tras 1, 2, 4, 8 y 15 s.

Medición de la latencia web. Con cada `pong`, el puente calcula $\text{rtt} = (t_{\text{monótono,ahora}} - t_{\text{eco}}) \cdot 1000$ y lo añade al snapshot como `lat_web_ms`. El navegador muestra una píldora «PLC · enlace · web» con las tres latencias.

Sentido	Tipo	Contenido
puente → servidor	`hola`	`{v, planta, modo, en_linea, t}` al conectar y al cambiar de modo.
	`snap`	`{s: snapshot}` en cuanto llega del gateway, como máximo a `--hz` (20 por defecto) y como mínimo uno por segundo.
	`evt`	`{e: evento}`, con cola de hasta 300.
	`resultado`	`{id, ok, msg, t, mando}`, respuesta a una orden.
	`ping`	`{t}` cada 3 s; el `pong` da el tiempo de ida y vuelta `lat_web_ms`.
servidor → puente	`orden`	Orden firmada (sección 4.7).
	`presencia`	Usuarios conectados; el puente la reenvía al gateway cada 5 s y sirve de latido del enlace.
	`pong`	`{t (eco), t_srv}`.

Tabla 4.3: Mensajes del WebSocket `/ws/planta`.

4.6 MQTT: el transporte alternativo

Qué es. MQTT (OASIS, versión 3.1.1) es un protocolo ligero de publicación y suscripción con *broker*. Los clientes publican mensajes en *topics* jerárquicos (`scada/planta/snap`) y se suscriben a ellos. Cada publicación lleva una calidad de servicio:

- QoS 0: como mucho una vez;
- QoS 1: al menos una vez, con confirmación;
- QoS 2: exactamente una vez.

Un mensaje *retenido* lo guarda el broker y se entrega a quien se suscriba después. El *last will* es un mensaje que el broker publica en nombre del cliente si este se desconecta sin despedirse.

Por qué, y por qué no por defecto. El autor dispone de un broker propio (`broker.smartsensorslab.com`) compartido con otro proyecto. MQTT se implementó en la sesión 07, a petición expresa, como transporte *completo y alternativo* (también para órdenes, con respuesta). Está apagado por defecto: el WebSocket sigue siendo el canal principal porque lleva órdenes con respuesta directa, presencia y colas por navegador sin depender de un tercero. Se activa con `python -m puente --mqtt mqtt+wss://usuario:clave@broker…`.

La URL del broker acepta cuatro esquemas:

- `mqtt://` (TCP, puerto 1883);
- `mqtts://` (TLS, 8883);
- `mqtt+ws://` (WebSocket, 80);
- `mqtt+wss://` (WebSocket con TLS, 443).

El último es el que atraviesa un túnel de Cloudflare, que solo deja pasar HTTP/WebSocket. El cliente es paho-mqtt 2 con `CallbackAPIVersion.VERSION2`, MQTT 3.1.1, sesión limpia, *keepalive* de 20 s y

Topic (`scada/<planta>/…`)	Publica	QoS	Retenido	Uso
`snap`	puente	0	no	Snapshot (se descarta si llega viejo).
`evt`	puente	1	no	Eventos.
`estado`	puente	1	**sí**	`hola`; también es el *last will* (`en_linea:false`).
`resultado`	puente	1	no	Respuesta a una orden.
`ping`	puente	0	no	Medición de ida y vuelta.
`orden`	servidor	1	no	Orden firmada.
`presencia`	servidor	0	**sí**	Usuarios conectados.
`pong`	servidor	0	no	Eco del `ping`.

Tabla 4.4: Topics MQTT del enlace (`core/enlace_mqtt.py`).

reconexión de 1 a 30 s. Sus llamadas de vuelta se pasan al bucle `asyncio` con `call_soon_threadsafe`.

4.7 Seguridad del enlace: HMAC, firmas y contraseñas

4.7.1 HMAC-SHA256

Un HMAC (RFC 2104) es un código de autenticación de mensajes: una huella que solo puede calcular quien conoce una clave secreta K. Con una función hash H (aquí SHA-256, de bloque $B = 64$ bytes),

$$\mathrm{HMAC}(K, m) = H\Big((K' \oplus \mathit{opad}) \,\|\, H\big((K' \oplus \mathit{ipad}) \,\|\, m\big)\Big), \tag{4.2}$$

donde K' es la clave rellenada a B bytes, $\mathit{ipad} = \texttt{0x36}^B$, $\mathit{opad} = \texttt{0x5C}^B$ y $\|$ es concatenación. Quien recibe $(m, \mathrm{HMAC}(K, m))$ y conoce K comprueba que el mensaje no se alteró y que lo generó alguien con la clave.

4.7.2 Órdenes firmadas

Toda orden que entra desde la web la firma el *servidor* y la verifica el *puente* antes de pasarla al gateway. Así, aunque alguien se hiciera pasar por el servidor ante el puente, no podría inyectar órdenes sin la clave:

Listing 4.4: Forma canónica y firma de una orden (`core/remoto.py:61-71`)

```python
def _canonico(orden):
    cuerpo = {"id": orden.get("id"), "t": orden.get("t"), "quien": orden.get("quien"),
              "cmd": orden.get("cmd")}
    return json.dumps(cuerpo, sort_keys=True, separators=(",", ":"), ensure_ascii=True).encode("ascii")


def firmar(clave, orden):
    """Devuelve la orden con el campo "firma" (no modifica la original)."""
    o = dict(orden)
    o["firma"] = hmac.new(clave.encode("utf-8"), _canonico(o), hashlib.sha256).hexdigest()
    return o
```

La forma canónica (claves ordenadas, sin espacios, solo ASCII) garantiza que firmante y verificador calculen el HMAC sobre exactamente los mismos bytes. El usuario y su rol (`quien`) van *dentro* de lo firmado, así que nadie puede subirse el rol por el camino. El verificador (`core.remoto.Verificador`) comprueba, en orden:

1. la estructura de la orden;
2. la firma, con `hmac.compare_digest`;
3. la frescura: $|t_{\text{ahora}} - t| \leq 30\,\text{s}$;
4. que el `id` (16 hexadecimales aleatorios) no se haya visto entre los últimos 2000;
5. que `quien.origen` sea `"web"`.

Los pasos 3 y 4 impiden el *ataque de repetición*: reenviar una orden válida capturada.

4.7.3 Sobre sellado para MQTT

En MQTT, el broker es un tercero y cualquiera con acceso al topic podría publicar. Por eso *todo* mensaje MQTT va sellado: `firma_hex(64 caracteres) + "\n" + json`, con la firma HMAC-SHA256(K, json). Los sobres con firma ajena se descartan. Las órdenes llevan además su firma propia, que es la que protege contra la repetición.

4.7.4 Cuentas del servidor

- **Contraseñas.** PBKDF2-HMAC-SHA256 (RFC 8018) con 240 000 iteraciones y sal aleatoria de 16 bytes. Cada verificación tarda unos 0,1 s a propósito, y se calcula fuera del bucle `asyncio`. Si el usuario no existe se calcula igual el hash, para que el tiempo de respuesta no revele qué usuarios existen.
- **Sesiones.** Token aleatorio de 32 bytes, del que la base guarda solo el SHA-256. Viaja en la cookie `gemelo_sesion`, que es `HttpOnly`, `SameSite=Strict`, `Secure` bajo HTTPS y dura 12 h.
- **Defensas.**
  - Cinco intentos fallidos en 300 s por IP dan 429.
  - Se comprueba la cabecera `Origin`.
  - Cabeceras `nosniff`, `SameOrigin` y `Referrer-Policy`.
  - Límite de 4 órdenes por segundo por usuario.
- **Roles.** `usuario` (solo mira) < `operador` (mandos) < `ingeniero` (parámetros, auditoría) < `admin` (cuentas, clave del puente). El HMI local está por encima de todos.

4.8 Resumen comparativo

Protocolo	Patrón	Formato	Por qué en este eslabón
DF1	maestro/esclavo serie	PCCC binario	Es el puerto nativo del SLC 5/04; no hay interfaz DH+ en el laboratorio.
OPC DA	cliente/servidor COM	variantes COM	Es lo que ofrece RSLinx Classic y lo que pide el enunciado.
ZeroMQ	PUB/SUB + REQ/REP	msgpack	Varios clientes de un mismo estado, sin broker, entre procesos de 32 y 64 bits.
WebSocket	bidireccional	JSON	Atraviesa NAT y cortafuegos como HTTPS; conexión saliente desde el laboratorio.
MQTT	PUB/SUB con broker	JSON sellado	Broker ya disponible; alternativa al servidor directo.
HTTPS	petición/respuesta	JSON	Inicio de sesión, cuentas, auditoría.

Tabla 4.5: Resumen de los protocolos del sistema.

CAPÍTULO 5

Bibliotecas y herramientas

Este capítulo describe cada biblioteca externa del proyecto: qué es, para qué se usa aquí y por qué se eligió frente a las alternativas. El criterio general fue siempre el mismo. Antes de añadir una dependencia al gateway se comprueba que tenga rueda (*wheel*) para Windows de 32 bits y Python 3.12, porque el gateway del laboratorio corre ahí (`mds/REGLAS.md`, sección 5). Las versiones se fijan con rangos en dos archivos de requisitos, uno por proceso: `requirements-gateway.txt` y `requirements-hmi.txt`.

5.1 Inventario

Biblioteca	Rango	Instalada	32 bits	Dónde y para qué
Python	3.12–3.13	3.13.7	sí	Lenguaje de todo el sistema (3.12 de 32 bits en el gateway del laboratorio).
PySide6	`>=6.8,<7`	6.11.2	**no**	HMI: Qt 6, motor QML, Qt Quick, QtSvg; también QtPdf en herramientas.
PyQt5 (alternativa)	5.15.11	5.15.11	sí	HMI con Qt 5.15 si solo hay Python de 32 bits (`.venv-qt5`).
pyzmq	`>=26,<27`	26.4.0	sí	Bus gateway ↔ HMI/puente (libzmq 4.3.5).
msgpack	`>=1.0,<2`	1.2.2	sí	Serialización del snapshot y las órdenes.
loguru	`>=0.7,<0.8`	0.7.3	pura	Registro del gateway en consola.
OpenOPC-Python3x	`>=1.3`	(lab.)	sí	Cliente OPC DA del gateway.
pywin32	`>=306`	(lab.)	sí	COM (`pythoncom`), base de OpenOPC.
aiohttp	`>=3.9,<4`	3.14.3	—	Servidor web del gemelo y cliente WebSocket del puente.
paho-mqtt	`>=2.0,<3`	2.1.0	pura	Transporte MQTT opcional.
sqlite3	stdlib	3.50.4	sí	Historial del gateway y auditoría del servidor.
pytest	desarrollo	9.1.1	—	Pruebas automáticas.
Playwright	desarrollo	1.63.0	—	Prueba del frontend 3D en Chromium sin ventana.
three.js	r128	r128	—	Escena 3D del gemelo en el navegador.
Inter	4.001	—	—	Tipografía del HMI y de este documento (licencia OFL 1.1).

Tabla 5.1: Bibliotecas del proyecto. «Instalada» es la versión del entorno de desarrollo (`.venv`, 10 de octubre de 2026); «32 bits» indica si hay rueda `win32` para Python 3.12.

5.2 Qt, PySide6 y QML

Qué es. Qt es un marco de aplicaciones en C++ con más de treinta años de desarrollo, usado en automoción, aviónica, equipos médicos y HMI industriales. PySide6 es el enlace oficial de Python para Qt 6, mantenido por The Qt Company con licencia LGPLv3. **QML** es el lenguaje declarativo de Qt para interfaces. Una pantalla se describe como un árbol de objetos con propiedades, y las propiedades se *enlazan* (*bindings*) con expresiones que se reevalúan solas cuando cambian sus dependencias. **Qt Quick** es la biblioteca de elementos visuales de QML. Se dibuja con un *scene graph* acelerado por GPU (Direct3D 11 en Windows con Qt 6) en un hilo de render propio.

Para qué. Toda la interfaz del HMI es QML (`ui/qml/`). Python solo expone tres objetos al motor: `Planta`, `Conexion` e `Historial`. Los gráficos vectoriales (SVG) se dibujan con QtSvg.

Por qué Qt/QML y no Tkinter, wxPython o una web local

- **Animación declarativa.** En QML, `Behavior on x { NumberAnimation { duration: 150 } }` basta para que un vástago se deslice en vez de saltar. En Tkinter (v6.1) cada animación es un temporizador y un redibujado manual del Canvas.
- **Separación entre lógica y presentación.** El QML no sabe de ZeroMQ ni de OPC: se enlaza a propiedades de `Planta`. La pantalla se puede rediseñar sin tocar Python, incluso en un editor visual (Qt Design Studio, capítulo 11).
- **Rendimiento.** El scene graph dibuja en la GPU y solo actualiza lo que cambió. La GUI de Python apenas trabaja: aplica un snapshot y emite señales.
- **Calidad gráfica.** Antialiasing, SVG nítido a cualquier escala, tipografía con *hinting*, opacidades y degradados nativos.
- **Frente a una web local:** sin navegador ni servidor HTTP en la PC del laboratorio; acceso directo a procesos (`QProcess`), archivos y SQLite. La web se reservó para el gemelo remoto, donde sí aporta.

5.2.1 Un solo QML para Qt 5 y Qt 6

Como PySide6 no tiene ruedas de 32 bits, el plan B si el laboratorio solo admitiera Python de 32 bits es PyQt5 5.15 (Qt 5.15). Para no mantener dos interfaces, desde el 24 de septiembre el QML es compatible con ambas versiones:

- imports con versión (`QtQuick 2.15`, `QtQuick.Controls 2.15`, `QtQuick.Layouts 1.15`, `QtQuick.Shapes 1.15`);
- ninguna API exclusiva de Qt 6: sin `MultiEffect`, `topLeftRadius`, `Qt.alpha`, operadores `??` ni `?.`;
- las cifras tabulares (`font.features`) y los pesos de fuente en `Canvas` pasan por funciones de `Estilo.qml` que comprueban si existen.

El Python de `ui/` importa Qt únicamente desde `ui/qt_compat.py`, que elige PySide6 si existe y si

no PyQt5 (`HMI_QT=PyQt5` fuerza el segundo). Ese módulo unifica `Signal`/`pyqtSignal`, `Slot`/`pyqtSlot`, `Property`/`pyqtProperty`, los enums de `QProcess` (que en PySide6 están anidados) y la forma de instalar interceptores de URL y ganchos de excepciones.

> **Qué se quedó fuera por esta compatibilidad** El plan inicial del HMI mencionaba `MultiEffect` para resplandores y sombras, `ParticleSystem` y pyqtgraph para las tendencias. Ninguno se usa. `MultiEffect` solo existe en Qt 6.5+. Las tendencias se dibujan con `Canvas` de QML, que no añade dependencias y funciona igual en ambas versiones (capítulo 10).

5.3 pyzmq y msgpack

pyzmq es el enlace oficial de Python a `libzmq` (capítulo 4). Incluye la libzmq compilada en la rueda, para `win32` y `win_amd64`, de modo que no hay que instalar nada más. También ofrece `zmq.asyncio`, que usan el puente y el gemelo local para integrar los sockets en el bucle de `asyncio`.

msgpack serializa el snapshot. Su extensión en C acelera la codificación; si no está disponible cae a una implementación pura. Frente a JSON reduce el tamaño y el tiempo de codificación. Frente a Protocol Buffers evita mantener un esquema compilado: el snapshot es un diccionario de Python y se publica tal cual.

5.4 OpenOPC y pywin32

pywin32 da acceso a la API de Windows y a COM desde Python (`win32com`, `pythoncom`). **OpenOPC-Python3x** es la adaptación a Python 3 del cliente OpenOPC original. Ofrece `client()`, `connect()`, `servers()`, `list()`, `read()` y `write()` sobre la interfaz de automatización OPC (`gbda_aut.dll`). Era la base de v6.1, que ya funcionaba contra el PLC físico, y se conservó por esa razón: es probada, pequeña y no necesita licencias. Como se usa a través de COM, impone las dos condiciones vistas en la sección 4.2: Python de 32 bits y un solo hilo.

5.5 aiohttp y paho-mqtt

aiohttp es un marco HTTP asíncrono para `asyncio`: servidor y cliente, con WebSocket nativo en ambos lados. El servidor del gemelo es una sola aplicación aiohttp que sirve el frontend, la API REST y los dos WebSocket (`/ws/web` y `/ws/planta`). El puente usa el cliente WebSocket de la misma biblioteca.

> **Por qué aiohttp y no Django, FastAPI o websockets** El registro inicial del gemelo hablaba de Django. En la sesión 03 se eligió aiohttp porque el prototipo del autor ya lo usaba y porque reúne en *un solo proceso*, sin dependencias adicionales, todo lo que hace falta:
>
> - HTTP y WebSocket en el mismo servidor;
> - colas por navegador;

- difusión del snapshot.

Django necesitaría Channels y un servidor ASGI, y para varias instancias, Redis. FastAPI añade Starlette, uvicorn y pydantic. La biblioteca `websockets` no sirve HTTP con comodidad.

paho-mqtt es el cliente MQTT de la Fundación Eclipse, en Python puro. Se usa en su versión 2 (`CallbackAPIVersion.VERSION2`). Corre su propio hilo de red (`loop_start`), y sus llamadas de vuelta se trasladan al bucle de `asyncio` con `call_soon_threadsafe`. Se importa solo dentro de `arrancar()`, así que quien no usa MQTT no necesita tenerlo instalado.

5.6 SQLite

`sqlite3` viene con Python. Es una base de datos SQL completa en un solo archivo, sin servidor. El gateway escribe el historial en `datos/historial.sqlite` desde un hilo propio, en modo WAL (*Write-Ahead Logging*), que permite al visor del HMI leer mientras el gateway escribe. El servidor del gemelo guarda en otra base las cuentas, las sesiones y la auditoría. No se eligió un servidor de base de datos (PostgreSQL, InfluxDB) porque el volumen es pequeño (eventos, alarmas y escrituras, no la serie temporal completa) y un archivo se copia, se respalda y se abre con cualquier herramienta.

5.7 loguru

loguru simplifica el registro del gateway: un solo objeto `logger`, formato con hora y nivel en color y sin configuración. Es Python puro, así que no tiene problemas de 32 bits.

5.8 three.js

three.js es la biblioteca de JavaScript más usada para gráficos 3D con WebGL. El gemelo usa la versión r128, servida desde `digital_twin/servidor_web/estatico/three.min.js` con respaldo en un CDN. Construye la escena con geometrías primitivas (cilindros, cajas, tubos sobre curvas Catmull-Rom), materiales físicos (`MeshStandardMaterial`), sombras suaves y *tone mapping* ACES. No carga modelos externos, así que la escena se genera por código y pesa poco (capítulo 13).

5.9 pytest y Playwright

pytest ejecuta las 229 pruebas del proyecto (sección 14.1). Las de `ui/` crean un `QCoreApplication` con la plataforma `offscreen`, sin ventana. **Playwright** maneja un Chromium sin ventana para comprobar que el frontend 3D carga, inicia sesión y reacciona a los snapshots.

5.10 Herramientas de desarrollo

- **VS Code** con la extensión «Qt Qml» y el servidor de lenguaje `pyside6-qmlls`: edición de QML con autocompletado.
- **Recarga en caliente** propia (`ui/app.py --dev`): un `QFileSystemWatcher` recarga el QML al guardar sin perder la conexión ZeroMQ.
- **Qt Design Studio 4.8.3** (instalado en `E:\Qt`): edición visual de `PantallaHmi.qml` (capítulo 11).
- **RSLogix 500, Emulate 500 y RSLinx Classic** en una VM de Windows 7: programa del PLC y modo EMULATE.
- **MiKTeX/LuaLaTeX**: este documento.

CAPÍTULO 6

Núcleo y gateway

Este capítulo documenta los paquetes `core/` y `gateway/`: la tabla de variables, el bucle del gateway, los dos backends, las alarmas, el historial y el árbitro de mando. La física y la estimación de posición, que también viven en `core/` y `gateway/`, tienen su propio capítulo (7).

6.1 `core/tags.py`: la tabla de variables

`core/tags.py` es la única fuente de direcciones del sistema. No tiene dependencias externas. Contiene:

- **Mapas de bits** por palabra: `BITS_I1` (`STOP_1`=0 … `SE3`=7), `BITS_O2` (`EV_E1`=0 … `EV_C3`=5), `BITS_B30` (`STARTV`=0 … `RESETV`=4) y `BITS_B33` (`X0`…`X8`).
- **Grupos de lectura.**
  - `ITEMS_RAPIDOS` tiene 22 items: las cuatro palabras `I:1.0`, `O:2.0`, `B3:0` y `B3:3`, más ACC, PRE y DN de los seis temporizadores.
  - `ITEMS_LENTOS` tiene `C5:0.ACC` y `C5:0.PRE`, que se leen cada `PERIODO_LENTO_CICLOS` = 10 ciclos.

  Las palabras se leen enteras y se descomponen en bits en el gateway: una palabra cuesta lo mismo que un bit en el enlace y trae dieciséis.
- **Escritura.** `PULSOS` (`STARTV`, `STOPV`, `RESETV`), `PARAMETROS` (tags virtuales: `T_ESPERA` → cinco PRE, `T_NEUM` → `T4:5.PRE`, `N_CICLOS` → `C5:0.PRE`), `PRESETS_DEFECTO` (100, 500, 3), `PULSO_MS` = 300 y `LIMITE_PRE` = 32 767.
- **Secuencia.** `ETAPAS` (número, texto y temporizador de cada etapa, que el HMI usa para dibujar el GRAFCET), `TIMER_DE_ETAPA` = {2:0, 3:1, 4:2, 5:3, 6:4, 7:5} y `CILINDROS` (bobinas y sensores de cada cilindro).
- **Funciones.** Codificar y decodificar palabras con máscara `0xFFFF`, normalizar `I:1/3` a `('I:1.0', 3)`, construir el ItemID `[TOPIC]dir`, y la más importante, `validar_escritura`.

Listing 6.1: Reglas de seguridad de escritura (`core/tags.py:203-222`)

```
def validar_escritura(tag, modo, valor=None):
    """(ok, motivo). Reglas de seguridad de mds/REGLAS.md sección 6."""
    try:
        dirs = direcciones_de_escritura(tag)
    except KeyError as e:
        return False, str(e)
```

```
    for d in dirs:
        if d.startswith(PREFIJOS_PROHIBIDOS):
            return False, "prohibido escribir salidas (%s)" % d
        if d.startswith("I:") and modo == "REAL":
            return False, "en modo REAL no se escriben entradas I:1"
    if tag in PARAMETROS:
        if valor is None:
            return False, "parámetro sin valor"
        v = int(valor)
        if not (0 <= v <= LIMITE_PRE):
            return False, "valor fuera de rango 0..%d" % LIMITE_PRE
        if tag == "N_CICLOS" and v < 1:
            return False, "N_CICLOS debe ser >= 1"
    return True, ""
```

`etapa_activa(bits)` devuelve la etapa solo si hay *exactamente una* activa. Durante una transición que el sondeo atrapa a medias (dos bits o ninguno), devuelve `None` en lugar de inventar.

6.2 `gateway/servidor.py`: el bucle

6.2.1 Un solo hilo con planificación por plazo

El objeto COM solo puede usarse desde el hilo que lo creó. Por eso el backend, los sockets ZeroMQ y la lógica del ciclo viven en un *único* hilo. El historial se escribe en otro hilo, pero solo recibe lotes por una cola. La planificación es por **plazo** (*deadline*): el ciclo n debe empezar en $t_0 + n \cdot T$, y entre plazos el hilo atiende órdenes con `zmq.Poller`, con un tiempo de espera igual a lo que falta para el siguiente plazo. No hay `sleep` después de leer. Si una lectura tarda más que el periodo, el siguiente plazo se reprograma a «ahora» (sin ráfagas para recuperar ciclos perdidos) y el periodo efectivo se reporta en `lat.periodo_ms`.

Listing 6.2: Pseudocódigo del bucle por plazo

```
siguiente ← ahora
mientras corriendo:
    ciclo()                                  # pasos (1)…(7)
    siguiente ← siguiente + periodo
    si siguiente < ahora: siguiente ← ahora  # va tarde: no se recuperan ciclos
    mientras ahora < siguiente:              # atender órdenes sin pasarse del plazo
        espera ← siguiente − ahora
        si espera > 32 ms: poll(espera − 32 ms) y procesar órdenes
        si no:             poll(0); sleep(1 ms) # tramo fino (granularidad de Windows)
        vencer flancos de bajada de pulsos
```

El periodo por defecto es de 50 ms en SIM y de 100 ms con OPC. En el laboratorio se midió que pedirle a RSLinx un refresco más rápido *empeora* el resultado, porque satura el enlace serie (capítulo 14).

6.2.2 Los siete pasos de cada ciclo

1. **Escrituras pendientes**, incluidos los flancos de bajada de los pulsos cuyo plazo venció.
2. **Lectura del grupo rápido** o, si no hay conexión, intento de reconexión. Cada fallo suma uno a un contador. Con `FALLOS_PARA_DESCONECTAR` = 5 fallos seguidos, el gateway se declara desconectado, publica la alarma y reintenta tras 1, 2, 4 y 8 s.
3. **Lectura del grupo lento** cada 10 ciclos, tras conectar, tras escribir un parámetro y al entrar en X0 o X7.
4. **Construcción del snapshot.**
   - Decodifica las palabras en bits y calcula `etapa`.
   - `t_etapa` es el tiempo, medido con `perf_counter`, desde que el gateway vio el cambio de etapa.
   - Toma `pos` del backend. Si el backend no da `pos_neum`, la calcula con el modelo de la cadena.
   - Detecta «Marcha», «Paro» y el fin de los N ciclos.
   - Avanza el árbitro de mando (`tick`).
5. **Alarmas.** Evalúa las reglas de `core/alarmas.py`.
6. **Publicación** del snapshot y de los eventos del ciclo (listado 4.2).
7. **Historial.** Pasa el lote del ciclo al hilo de SQLite.

6.2.3 Pulsos cronometrados en el gateway

Listing 6.3: Pulso de 300 ms (`gateway/servidor.py:299-311`)

```
    def _orden_pulso(self, tag, ms):
        if tag not in tags.PULSOS:
            return self._rechazo(tag, 1, "no es un pulso: %s" % tag)
        ok, msg = tags.validar_escritura(tag, self.modo, 1)
        if not ok:
            return self._rechazo(tag, 1, msg)
        ok, msg = self._escribir_pares(tag, [(tags.PULSOS[tag], 1)])
        self._registrar_escritura(tag, 1, ok, msg)
        if not ok:
            return protocolo.respuesta(False, msg)
        self._pulsos[tag] = time.perf_counter() + ms / 1000.0
        self._evento("Pulso %s (%d ms) · %s" % (tag, ms, etiqueta(self._quien)), origen=etiqueta(self.
    _quien))
        return protocolo.respuesta(True, "pulso %s %d ms" % (tag, ms))
```

El 1 se escribe en cuanto llega la orden y el 0 se agenda a $t + 300$ ms. Como el bucle vence los flancos de bajada entre plazos, la duración real del pulso no depende de la GUI ni del periodo de lectura. En v6.1, en cambio, el pulso se cronometraba en Tkinter y se alargaba de forma variable.

6.3 Los dos backends

Ambos backends implementan la misma interfaz (`conectar`, `leer_rapido`, `leer_lento`, `escribir`, `cerrar`, `cambiar_fisica`, `cambiar_fuente` y las propiedades `modo`, `fuente`, `conectado`, `pos`, `pos_neum`). Las lecturas devuelven `(valores, calidad, t_ms)`. El servidor no sabe cuál tiene.

6.3.1 `BackendSim`

Ejecuta en el mismo proceso el Ladder traducido a Python y el modelo físico. En cada `leer_rapido`:

1. avanza el tiempo real transcurrido (medido con `perf_counter`, con un tope de 1 s) en barridos de hasta 5 ms;
2. en cada barrido mueve la física con las bobinas del barrido anterior;
3. copia los sensores a `I:1.0` y ejecuta un `scan` del Ladder.

Con la opción `--simular-enlace`, la clase `EnlaceLento` interpone una caché por archivo del PLC (`I`, `O`, `B3`, `T4`, `C5`) que se refresca a intervalos aleatorios con la distribución medida en el laboratorio:

- 62 %: entre 0,15 y 0,45 s;
- 28 %: entre 0,9 y 1,4 s;
- 10 %: entre 1,9 y 2,4 s.

En ese modo `pos` vale `None`, como en REAL. Así se prueba el modo suave sin el PLC.

6.3.2 `BackendOPC`

Habla con RSLinx mediante OpenOPC. Se describió en la sección 4.2: lectura de caché o de dispositivo, validación de direcciones al conectar, protección de escrituras y escritura de sensores en EMULATE. Cada valor se acepta solo si su calidad es `Good` y no es nulo. Las palabras se enmascaran con `& 0xFFFF`, porque RSLinx las entrega como enteros con signo: un `I:1.0` con el bit 15 encendido llegaría negativo.

En EMULATE, el backend lee las bobinas de `O:2.0`, mueve el modelo físico con el tiempo transcurrido y escribe en `I:1.0` solo los sensores que cambiaron.

6.4 `core/alarmas.py`

Mientras el gateway está desconectado, las reglas que dependen de datos (calidad, paro físico, etapa excedida) no se evalúan y conservan su estado: no tiene sentido declarar «etapa excedida» con datos viejos. Cada alarma es un diccionario `{id, nivel, msg, t_ini, activa}`. Al activarse emite un evento con su nivel y al limpiarse, un evento `info` «Normalizada: …». Las dos prioridades (alarma en rojo, aviso en ámbar) y la banda que solo aparece con alarmas activas siguen la filosofía de ISA-101 e ISA-18.2. El capítulo 12 discute hasta dónde llega esa conformidad.

id	nivel	se activa cuando	se limpia cuando
`desconexion`	alarma	`conectado == False` (cinco lecturas fallidas)	vuelve la lectura
`calidad`	aviso	algún item sin calidad `Good`	todos `Good`
`stop_fisico`	aviso	`STOP_1` = 0 (pulsador de paro oprimido)	`STOP_1` = 1
`etapa_excedida`	alarma	Con `ACTIVE`: etapa con TON y $t_{\text{etapa}} > 1{,}5 \cdot \text{PRE} \cdot 0{,}01 + 1$ s; X1, o X0 sin los tres cilindros retraídos, con $t_{\text{etapa}} > 5$ s	cambia la etapa
`latencia`	aviso	$p_{95} > 2 \cdot T$ durante 10 ciclos seguidos	baja del umbral

Tabla 6.1: Reglas de alarma (evaluadas en el gateway en cada ciclo).

6.5 `core/historial.py`

Listing 6.4: Esquema SQLite del historial (`core/historial.py:37-43`)

```
CREATE TABLE IF NOT EXISTS eventos    (t REAL, nivel TEXT, msg TEXT, origen TEXT);
CREATE TABLE IF NOT EXISTS alarmas    (id TEXT, nivel TEXT, msg TEXT, t_ini REAL, t_fin REAL);
CREATE TABLE IF NOT EXISTS escrituras (t REAL, tag TEXT, valor TEXT, ok INTEGER, msg TEXT);
CREATE INDEX IF NOT EXISTS ix_eventos_t  ON eventos (t);
CREATE INDEX IF NOT EXISTS ix_alarmas_id ON alarmas (id, t_ini);
```

La clase `Historial` acumula un lote por ciclo y lo pasa por una `queue.Queue` a un hilo propio, que es el dueño de la conexión. Ese hilo abre la base con `PRAGMA journal_mode=WAL` y `synchronous=NORMAL` e inserta cada lote en una transacción. Así el bucle del gateway nunca espera al disco. Si un lote falla, se descarta y el gateway sigue. Una alarma activa se inserta con `t_fin` nulo, y al limpiarse se actualiza la fila. `LectorHistorial` abre la base en solo lectura (`file:…?mode=ro`). Lo usa el visor del HMI, con filtros, límite de 500 filas y exportación a CSV en UTF-8 con BOM, que Excel abre con los acentos correctos.

6.6 `core/mando.py`: el árbitro de mando

Con el gemelo web, varias personas pueden querer mandar la planta a la vez. El árbitro decide quién puede escribir, y lo consulta el gateway *antes* de ejecutar cualquier orden.

- **Roles.** `usuario` (0) < `operador` (1) < `ingeniero` (2) < `admin` (3) < HMI local (10). Una orden sin `quien` es local.
- **Nivel mínimo por orden.** `ping` y `estado`: 0; `pulso` y `mando`: 1; `escribir`: 2. `entrada`, `fisica`, `fuente` y `presencia` son solo locales.
- **Mando exclusivo y temporal.** Una sola persona lo tiene. Vence a los 120 s sin órdenes si es web y a los 60 s si es local. Se concede si está libre, vencido, ya es tuyo, o si tu nivel es mayor que el de quien lo tiene.

- **El mando remoto está deshabilitado por defecto.** Solo el HMI local lo habilita, y al deshabilitarlo se revoca el mando web.
- **`STOPV` siempre pasa** para operador o superior, aunque no tenga el mando y aunque el remoto esté deshabilitado: parar nunca espera.
- **Enlace vigente.** Si el puente pasa 15 s sin dar noticias (presencia), el gateway marca el enlace web como caído y revoca el mando web.

Listing 6.5: Reglas del árbitro para una escritura web (`core/mando.py:237-246`)

```
        if op == "pulso" and cmd.get("tag") == "STOPV":
            return True, "", quien                      # parar nunca espera
        if not self.remoto_habilitado:
            return False, "el mando remoto está deshabilitado en el HMI local", quien
        if not self._vigente():
            return False, "pide el mando antes de escribir (está libre)", quien
        if not _mismo(self.titular, quien):
            return False, "el mando lo tiene %s" % etiqueta(self.titular), quien
        self._dar(quien)                                # renueva
        return True, "", quien
```

6.7 `core/latencia.py`

`EstadisticasLatencia` guarda en una ventana de 200 muestras el tiempo de cada lectura de grupo, medido con `time.perf_counter()`, el tiempo de ciclo y el periodo efectivo. El resumen que va al snapshot incluye la media de los últimos 20 periodos, el máximo de la ventana y el percentil 95, calculado por interpolación lineal:

$$k = \frac{(n-1)\,p}{100}, \qquad i = \lfloor k \rfloor, \qquad P_p = x_{(i)} + \big(x_{(i+1)} - x_{(i)}\big)(k - i), \tag{6.1}$$

con $x_{(i)}$ las muestras ordenadas. El HMI grafica esa serie en el panel «Latencia».

6.8 `gateway/diagnostico_opc.py`

El HMI es de 64 bits y no puede hablar COM, pero la ventana de conexión necesita descubrir servidores, topics e items, como hacía el selector de v6.1. La solución es una herramienta de línea de órdenes que corre con el Python de 32 bits y responde en JSON:

Listing 6.6: Subcomandos del diagnóstico OPC

```
python -m gateway.diagnostico_opc --json --servidores          {"ok": true, "servidores": [...]}
python -m gateway.diagnostico_opc --json --topics              {"ok": true, "topics": [...]}
python -m gateway.diagnostico_opc --json --archivos            {"ok": true, "ruta": "<topic>.Online",
    ...}
python -m gateway.diagnostico_opc --json --items RUTA          {"ok": true, "items": [...]}
python -m gateway.diagnostico_opc --json --leer ITEM           {"ok": true, "valor": 69, "calidad": "
    Good"}
```

La herramienta también mide tiempos de lectura desde el dispositivo y desde la caché, y prueba la escritura de un preajuste reescribiendo su mismo valor. Nunca escribe en `O:2`, `I:1` ni en los bits de mando.

6.9 Línea de órdenes del gateway

Listing 6.7: Opciones de `python -m gateway`

```
  --modo SIM|EMULATE|REAL     origen de los datos (defecto SIM)
  --servidor, --topic         servidor OPC y topic de RSLinx (defecto PROYECTO1_TEST)
  --periodo MS                periodo objetivo (50 en SIM, 100 en OPC)
  --fuente cache|dispositivo  fuente de lectura OPC (defecto cache)
  --pub, --cmd                endpoints ZeroMQ (tcp://127.0.0.1:5555 y :5556)
  --datos CARPETA             carpeta del historial; --sin-historial lo desactiva
  --remoto                    arranca con el mando web habilitado
  --ciclos-demo N             SIM: escribe N_CICLOS=N, pulsa START, sale al terminar
  --simular-enlace            SIM: imita el enlace lento del laboratorio
  --log NIVEL                 nivel de loguru
```

`--ciclos-demo` funciona como prueba de humo del simulador:

- cuenta las muestras con las dos bobinas de una válvula activas a la vez;
- imprime los tiempos por etapa;
- sale con código 0 solo si terminó los N ciclos sin conflictos.

CAPÍTULO 7

Física y modelos

El HMI tiene que dibujar cinco vástagos moviéndose, pero el PLC solo sabe dónde están en los extremos, y de 4A y 5A no sabe nada. Este capítulo explica la física real de un cilindro neumático, por qué el software *no* la simula completa, y los cuatro modelos que sí usa:

- el modelo de tiempos del simulador (`gateway/fisica.py`);
- el Ladder simulado (`gateway/ladder_sim.py`);
- el estimador de carrera para el modo REAL (`core/estimador.py`);
- el modo suave, que predice la animación cuando el enlace es lento (`core/suavizado.py`).

7.1 La física de un cilindro neumático

7.1.1 Fuerza

En un cilindro de doble efecto, el aire a presión manométrica p actúa sobre el área del émbolo $A = \pi D^2/4$ al salir, y sobre el área anular $A - \pi d^2/4$ al regresar (d es el diámetro del vástago). La fuerza teórica es $F = p\,A$, y la útil es algo menor por la fricción de las juntas. Para los cilindros dibujados en el HMI, suponiendo una presión de trabajo de 6 bar:

Cilindro	D / d (mm)	A (mm^2)	A_{anular} (mm^2)	F_{salida} (N)	F_{regreso} (N)
ISO 15552 ø32 (1A, 2A, 3A)	32 / 12	804,2	691,2	482,5	414,7
ISO 6432 ø20 (4A, 5A)	20 / 8	314,2	—	188,5 − F_{muelle}	muelle

Tabla 7.1: Fuerzas teóricas a $p = 0{,}6$ MPa (6 bar manométricos). Son valores de referencia calculados para este documento; el software no los usa. PENDIENTE: confirmar la presión real del tablero y los diámetros reales de los cilindros del laboratorio.

El consumo de aire de una carrera doble del ø32×200 es el volumen barrido, $V = A\,s + A_{\text{anular}}\,s = 0{,}161\,\text{L} + 0{,}138\,\text{L}$, multiplicado por la relación de compresión $(p + p_{\text{atm}})/p_{\text{atm}} = 7{,}01/1{,}013 \approx 6{,}92$. Resultan unos $2{,}07\,\text{L}$ de aire libre por ciclo y por cilindro.

7.1.2 Dinámica

Un modelo dinámico completo de un cilindro de doble efecto acopla la mecánica del émbolo con la termodinámica de las dos cámaras:

$$m\,\ddot{x} = p_1 A_1 - p_2 A_2 - F_{\text{fr}}(\dot{x}) - F_L, \tag{7.1}$$

$$\dot{p}_i = \frac{\kappa}{V_i(x)}\Big(R\,T\,\dot{m}_i \mp p_i A_i\,\dot{x}\Big), \qquad V_1 = V_{01} + A_1 x, \quad V_2 = V_{02} + A_2(s - x), \tag{7.2}$$

donde $\dot{m}_i$ es el caudal másico que entra o sale de cada cámara por la válvula. Ese caudal depende de la caracterización de la válvula según ISO 6358: conductancia sónica C y relación crítica de presiones b. En un cilindro de simple efecto como 4A y 5A, el término de la cámara 2 se sustituye por la fuerza del muelle, $k\,(x_0 + x)$.

En la práctica, con estrangulación del escape (*meter-out*), el perfil de velocidad tiene tres fases: una aceleración corta, una fase larga de velocidad casi constante limitada por el caudal de escape y una amortiguación al final de carrera. El tiempo de carrera es, por tanto, el parámetro que mejor resume el comportamiento.

Por qué el software usa un modelo de tiempos y no la dinámica completa

- **No hay datos para ajustarla.** Se desconocen la masa móvil, la carga, la fricción, C y b de las válvulas y los volúmenes muertos. Un modelo con diez parámetros inventados no es más fiel que uno con dos medidos.
- **No hay sensores para validarla.** Solo existen SC y SE. Lo único verificable es *cuánto tarda* el vástago en ir de un extremo al otro, y eso es justo lo que el modelo de tiempos usa y lo que el estimador mide.
- **Es lo que ve el PLC.** El GRAFCET avanza por eventos de sensores y por temporizadores. Un modelo que reproduce los tiempos de llegada a los sensores reproduce exactamente la secuencia que el PLC ejecuta.
- **Velocidad casi constante.** Con escape estrangulado, la fase dominante tiene velocidad casi constante, y una rampa lineal es una buena aproximación de primer orden.

7.2 Modelo del simulador: válvula biestable y carrera

`gateway/fisica.py` modela cada cilindro de doble efecto con dos estados: la posición normalizada $p \in [0,1]$ (0 = retraído, 1 = extendido) y la posición del carrete de la válvula 5/2 biestable, $c \in \{-1, 0, +1\}$. El carrete tiene memoria: cambia solo cuando una bobina está energizada sin la otra, y el vástago termina su carrera aunque la bobina se apague (corrección de la sesión 14).

$$c \leftarrow \begin{cases} +1 & \text{si EV_E} \wedge \overline{\text{EV_C}} \\ -1 & \text{si EV_C} \wedge \overline{\text{EV_E}} \\ c & \text{en otro caso} \end{cases} \qquad p \leftarrow \begin{cases} \min\big(1,\ p + \Delta t / t_{\text{ext}}\big) & c = +1 \\ \max\big(0,\ p - \Delta t / t_{\text{ret}}\big) & c = -1 \end{cases} \tag{7.3}$$

$$\text{SC} = (p \le 0{,}02), \qquad \text{SE} = (p \ge 0{,}98). \tag{7.4}$$

La velocidad es constante: $1/t_{\text{ext}}$ o $1/t_{\text{ret}}$ carreras por segundo. Los tiempos por defecto son distintos por cilindro para que la calibración del HMI tenga algo que medir:

Cilindro	t_{ext} (s)	t_{ret} (s)	velocidad media de salida (ø32×200)
1A	0,80	0,75	$0{,}25\,\text{m/s}$
2A	0,90	0,85	$0{,}22\,\text{m/s}$
3A	0,70	0,70	$0{,}29\,\text{m/s}$

Tabla 7.2: Tiempos de carrera del simulador (`CARRERAS_DEFECTO`).

Listing 7.1: Paso de la física con válvula biestable (`gateway/fisica.py:51-62`)

```
        dt = max(0.0, float(dt))
        for k, (e, c) in enumerate(ev):
            if e and not c:
                self.carrete[k] = 1
            elif c and not e:
                self.carrete[k] = -1
            t_ext, t_ret = self.carreras[k]
            if self.carrete[k] > 0:
                self.pos[k] = min(1.0, self.pos[k] + dt / t_ext)
            elif self.carrete[k] < 0:
                self.pos[k] = max(0.0, self.pos[k] - dt / t_ret)
        return self.sensores()
```

7.3 La cadena 4A/5A

Para 4A y 5A no hay bobinas ni sensores en el PLC, así que el modelo es puramente temporal y se ancla en la etapa X7. Con $t = t_{\text{etapa}}$ y la rampa

$$r(t; t_0, d) = \begin{cases} 0 & t \le t_0 \\ \text{mín}\big(1, (t - t_0)/d\big) & t > t_0 \end{cases} \tag{7.5}$$

las posiciones son

$$p_4 = r\big(\text{mín}(t, t_{\text{ret}});\ 0,\ t_{4,\text{ext}}\big) \cdot f(t), \tag{7.6}$$

$$p_5 = r\big(\text{mín}(t, t_{\text{ret}});\ t_{5,\text{ini}},\ t_{5,\text{ext}}\big) \cdot f(t), \tag{7.7}$$

$$f(t) = 1 - r(t;\ t_{\text{ret}},\ d_{\text{ret}}), \tag{7.8}$$

con $t_{4,\text{ext}} = 0{,}6$ s, $t_{5,\text{ini}} = 1{,}2$ s, $t_{5,\text{ext}} = 0{,}6$ s, $t_{\text{ret}} = 2{,}8$ s y $d_{\text{ret}} = 0{,}7$ s. El factor f representa el retorno por muelle de ambos a la vez. Fuera de X7 los dos valen 0. El ciclo modelado dura 3,5 s, dentro de la ventana de 5 s de `T4:5`: la ventana del PLC tiene un margen de 1,5 s (43 %), en línea con la recomendación del informe del proyecto de medir la cadena real y añadir un 20–30 %.

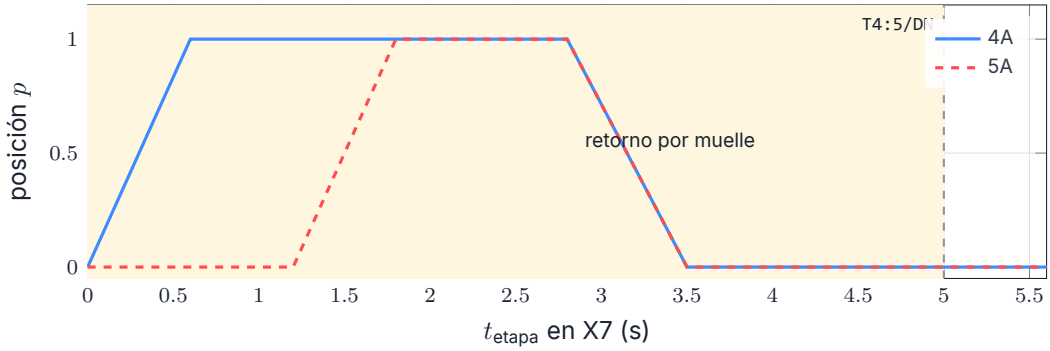


Figura 7.1: Modelo de tiempos de la cadena neumática durante X7. La zona sombreada es la ventana de `T4:5` (5 s).

7.4 El Ladder simulado

`gateway/ladder_sim.py` traduce el programa del PLC a Python, escalón por escalón y en el mismo orden. En cada llamada a `scan(dt)`:

1. **0000 GM**: ACTIVE $=$ (START_1 $\vee$ STARTV $\vee$ ACTIVE) $\wedge$ STOP_1 $\wedge$ $\overline{\text{STOPV}}$.
2. **0001 GS**: con primer barrido, `END`, Start y Stop a la vez, o `RESETV`: X0 = 1 y el resto 0; `C5:0.ACC` = 0.
3. **0002–0010**: tabla de transiciones (origen, destino, receptividad), con la divergencia de X7 según `C_ACC` < `C_PRE`. El simulador franquea como mucho una transición por barrido.
4. **0011–0016**: los TON, con acumulador *flotante* en segundos. El ACC entero se expone como $\lfloor \text{acum}/0{,}01 \rfloor$ acotado al PRE.
5. **0017**: flanco de subida de X7 (el OSR) incrementa `C5:0.ACC`.
6. **0018–0023**: ecuaciones de salida, que no dependen de `ACTIVE`.
7. **0024**: END $= X8$.

Listing 7.2: TON con acumulador flotante y OSR + CTU (`gateway/ladder_sim.py:103-119`)

```
        # 0011..0016 TON: T4:0..T4:4 con X2..X6, T4:5 con X7 (base 0.01 s)
        for n in range(tags.N_TIMERS):
            etapa_en = n + 2 if n < 5 else 7
            if self.X(etapa_en):
                pre_s = i[_T(n, "PRE")] * tags.BASE_TIEMPO_S
                # Como en el SLC, el ACC deja de crecer al llegar al PRE.
                self.acum[n] = min(self.acum[n] + dt, max(pre_s, self.acum[n]))
                i[_T(n, "ACC")] = min(i[_T(n, "PRE")], int(self.acum[n] / tags.BASE_TIEMPO_S + 1e-9))
                b[_T(n, "DN")] = self.acum[n] + 1e-9 >= pre_s
            else:
                self._reset_ton(n)

        # 0017 X7 → OSR B3:2/0 → CTU C5:0
        x7 = self.X(7)
```

```
        if x7 and not self.osr_x7:
            i["C_ACC"] = min(tags.LIMITE_PRE, i["C_ACC"] + 1)
        self.osr_x7 = x7
```

Por qué un acumulador flotante El simulador de v6.1 sumaba al ACC `int(round(dt·100))` en cada barrido. Con barridos de menos de 5 ms, el redondeo da 0 y el temporizador se congela. El acumulador flotante suma el tiempo exacto y solo convierte a centésimas al exponer el valor. El `1e-9` evita que errores de redondeo binario retrasen el bit `DN` un barrido.

7.5 El estimador de carrera (modo REAL)

En REAL el gateway publica `pos = null`. El HMI (y el puente para la web) estima la posición con `EstimadorCarrera`, que además *mide* los tiempos reales de carrera y los guarda en una calibración.

7.5.1 Reglas

1. **Inicio de carrera.** Un flanco de subida de `EV_En` con `SEn` = 0 inicia una salida: objetivo 1, duración $T = \text{calibración}(n, \text{ext})$, $t_0 = t$, y p_0 igual a la posición estimada actual. El caso de `EV_Cn` y `SCn` es simétrico.
2. **Durante la carrera**, interpolación lineal:

$$\hat{p}(t) = p_0 + (p^* - p_0) \cdot \text{clamp}\left(\frac{t - t_0}{T}, 0, 1\right). \tag{7.9}$$

3. **Llegada del sensor de destino** (`SE` al salir, `SC` al regresar). Se fija $\hat{p} = p^*$, se marca `confirmado` y, si la carrera era completa, se registra $(t - t_0) \cdot 1000$ ms en la calibración. Si el sensor llega antes de que la estimación termine, el HMI salta al extremo; si llega después, el vástago espera en el extremo hasta que el sensor lo confirme.
4. **Solo carreras completas.** Se mide únicamente si el sensor de origen estaba activo en el snapshot actual o en el anterior. Una inversión a media carrera se anima desde la posición estimada, pero no se registra.
5. **Tiempo límite.** Si en 10 s no llega el sensor, se cancela la medición sin registrar.
6. **En reposo mandan los sensores**: SE · $\overline{\text{SC}} \Rightarrow 1$, SC · $\overline{\text{SE}} \Rightarrow 0$.

7.5.2 Calibración

`Calibracion` guarda, por cilindro y dirección, las últimas mediciones válidas (entre 100 y 10 000 ms) y usa como duración la **mediana** de las cinco más recientes (800 ms si no hay ninguna). La mediana, y no la media, hace que una medición anómala, como un pulso de Stop a media carrera o un retardo del enlace, no desplace la duración. Se persiste en `datos/calibracion.json` con escritura atómica: archivo temporal, `fsync` y `os.replace`. Un corte de luz deja el archivo viejo o el nuevo, nunca uno a medias.

$$T_{n,\text{dir}} = \text{mediana}\left(\{\, t_k : 100 \le t_k \le 10\,000 \text{ ms} \,\}_{\text{últimas } 5}\right) \tag{7.10}$$

7.6 El modo suave

7.6.1 El problema

En el laboratorio, la caché de RSLinx se refresca cada 0,3 s típicamente, con picos de 1–2 s, y cada archivo del PLC (`I`, `O`, `B3`, `T4`) puede llegar con distinta antigüedad. Con el estimador clásico pasan tres cosas:

- el vástago salta de extremo a extremo;
- la etapa avanza a saltos;
- si `T_ESPERA` es corto, carreras completas ocurren entre dos lecturas y nunca se ven. Con `T_ESPERA` = 0,2 s, el estimador perdía de 1 a 3 de cada 3 carreras.

7.6.2 La idea

La secuencia es *fija y conocida*. Si se sabe en qué punto del ciclo está la planta, se sabe también qué tiene que pasar después. `Suavizador` mantiene un **cursor del ciclo**

$$\iota = 2 \cdot \text{etapa} + \text{DN} \qquad (\iota \in \{0, \ldots, 15\}), \tag{7.11}$$

que combina toda la evidencia disponible y *solo avanza* dentro de un ciclo:

- la etapa leída en `B3:3`;
- las bobinas de `O:2`: por ejemplo, `EV_E2` encendida implica X3 con `T4:1/DN`;
- los acumuladores de `T4`.

Una lectura vieja de `B3:3` no deshace lo que ya mostró `O:2`. El cursor solo retrocede cuando la etapa *leída* vuelve atrás: X7 → X1 (ciclo nuevo) o cualquier etapa → X0 (RESET o fin).

Cilindro	empieza a salir	afuera confirmado	empieza a regresar	adentro confirmado
1A	2 (X1)	4 (X2)	5 (X2·DN)	6 (X3)
2A	7 (X3·DN)	8 (X4)	9 (X4·DN)	10 (X5)
3A	11 (X5·DN)	12 (X6)	13 (X6·DN)	14 (X7)

Tabla 7.3: Fases de cada cilindro en el cursor del ciclo (`FASES` de `core/suavizado.py`). Se derivan de las ecuaciones de salida del Ladder.

7.6.3 Temporizador previsto

Los TON cuentan a 100 centésimas por segundo, también durante un STOP. Para cada observación (ACC_k, t_k) el suavizador guarda $m = \max_k(\text{ACC}_k - 100\,t_k)$ y extrapola:

$$\widehat{\text{ACC}}(t) = \min\big(\text{PRE},\ \max(0,\ m + 100\,t)\big). \tag{7.12}$$

Tomar el máximo hace que la estimación nunca retroceda y siga a la observación *más adelantada*, aunque lleguen lecturas viejas después. Al entrar en la etapa se ancla con $\text{ACC} = 0$ en $t - t_{\text{etapa}}$. Cuando $\widehat{\text{ACC}} \geq \text{PRE}$, el cursor pasa a DN. Así la bobina que el Ladder enciende con ese DN se da por encendida sin esperar a leerla. En la planta simulada, la salida de 3A, que arranca con el DN de `T4:3`, se ve 0,1–0,26 s después de la real, frente a hasta 1–2 s esperando la lectura. Lo que el suavizador *nunca* hace es adelantar una transición del GRAFCET que no se haya leído, porque esas dependen de sensores.

Listing 7.3: Extrapolación del acumulador (`core/suavizado.py:145-154`)

```python
    def _acc_est(self, timer, t):
        m = self._m.get(timer)
        acc = 0.0 if m is None else max(0.0, m + 100.0 * t)
        pre = self._pre(timer)
        return min(float(pre), acc) if pre is not None else acc

    def _observar_timer(self, timer, acc, t):
        v = float(acc) - 100.0 * t
        if self._m.get(timer) is None or v > self._m[timer]:
            self._m[timer] = v
```

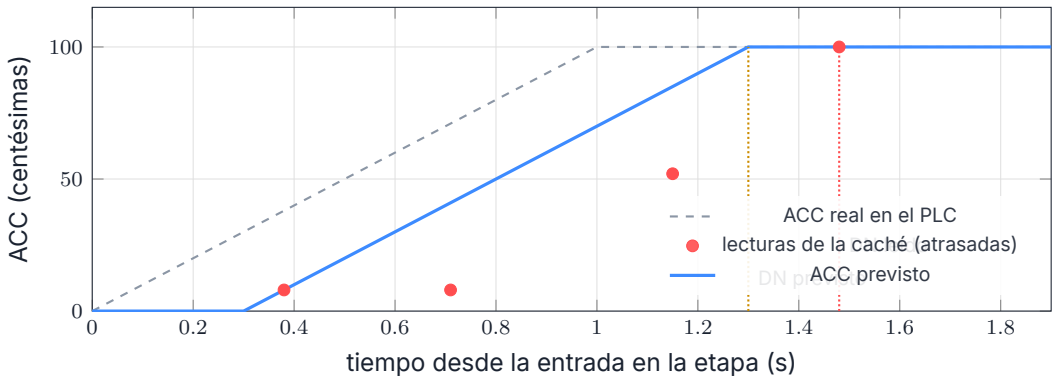


Figura 7.2: Ilustración de la extrapolación del acumulador de un TON con PRE = 100. La etapa se leyó 0,30 s después de empezar y las lecturas del acumulador llegan con retraso variable. La previsión se ancla en la entrada a la etapa, conserva la observación más adelantada ($m = \max(\text{ACC}_k - 100\,t_k)$) y avanza a 100/s sin retroceder: el DN se da por hecho antes de leerlo. Los datos son ilustrativos.

7.6.4 Cinemática del vástago en modo suave

Con $\Delta t = \text{mín}(\Delta t_{\text{máx}},\ t - t_{\text{anterior}})$ y la meta p^* que dicta el cursor:

$$T = \text{clamp}\left(\text{calibración}(n, \text{dir}),\ 0{,}25,\ 2{,}5\right)\ \text{s}, \qquad v = 1/T, \tag{7.13}$$

$$v \leftarrow \text{máx}\left(v,\ |p^* - p|/0{,}25\right) \quad \text{si el extremo ya está confirmado,} \tag{7.14}$$

$$p \leftarrow \begin{cases} p^* & |p^* - p| \le v\,\Delta t \\ p + \text{sgn}(p^* - p)\, v\,\Delta t & \text{en otro caso.} \end{cases} \tag{7.15}$$

Las reglas adicionales son:

- el vástago nunca se mueve más de una carrera por T; con cuadros de 15 ms, nunca más de 0,06 de carrera por cuadro;
- si el cursor pasa por una fase de regreso sin que el vástago haya llegado afuera, la carrera perdida se *repone* en $\text{mín}(T_{\text{ext}}, 0{,}35\ \text{s})$;
- si el sensor del extremo contrario sigue activo más de $3\ \text{s} + T$, manda el sensor y se marca `discrepancia`; el HMI muestra «sin confirmación».

7.6.5 El seguidor del navegador

El puente aplica el mismo suavizador y manda en el snapshot un bloque `suave` con posición, velocidad prevista v (con signo, en carreras por segundo) y objetivo de cada cilindro. El navegador no espera al siguiente snapshot: predice con la velocidad y absorbe cada corrección con un filtro exponencial de constante $\tau = 60$ ms:

$$\hat{p} = \text{clamp}\left(p + v\ \text{mín}(2, t - t_{\text{snap}}),\ \text{saturado en el objetivo}\right), \tag{7.16}$$

$$p_{\text{vista}} \leftarrow p_{\text{vista}} + (\hat{p} - p_{\text{vista}})\left(1 - e^{-h/\tau}\right), \tag{7.17}$$

con h el tiempo desde el cuadro anterior (clase `Seguidor`, `digital_twin/index.html`). Sin bloque `suave`, el navegador usa `Interp`: dibuja con un retraso de $1{,}5\,\bar{\Delta}t$ (acotado entre 40 y 300 ms) e interpola entre las dos últimas muestras, o extrapola como mucho 120 ms. $\bar{\Delta}t$ es la media móvil exponencial del periodo, $\bar{\Delta}t \leftarrow 0{,}8\,\bar{\Delta}t + 0{,}2\,d$.

> **El modo suave no baja la latencia** El PLC se sigue leyendo cuando el enlace lo permite. Lo que cambia es cómo se dibuja lo que llega tarde. Con el modo suave, `pos`, `posNeum`, `etapa`, `tEtapa` y los ACC que muestra el HMI son *previstos*. Los bits, las palabras, los eventos y las alarmas siguen siendo exactamente lo leído. El modo se activa con el botón «Suave» del sinóptico y se recuerda en `datos/preferencias_hmi.json`.

CAPÍTULO 8

La interfaz gráfica

8.1 Arranque: `ui/app.py`

`ui/app.py` prepara Qt, crea los objetos de Python que verá QML y carga la ventana. El orden importa:

1. `preparar_qt()`:
   - instala un manejador que manda todo aviso de Qt/QML a la consola y lo cuenta (la prueba de humo falla si hay errores de QML);
   - instala un gancho de excepciones;
   - fija el estilo de controles (`Basic` en Qt 6, `Default` en Qt 5);
   - crea la `QGuiApplication`.
2. `cargar_fuentes()` registra las cuatro Inter (`Regular`, `Medium`, `SemiBold`, `Bold`) con `QFontDatabase.addApplicationFont`. Tiene que ocurrir *antes* de crear el motor QML, que resuelve las familias al cargar.
3. Crea `Planta`, `Historial` y `Conexion`, y lee los datos institucionales de `mds/data.md`.
4. Crea el motor (`QQmlApplicationEngine`), instala el interceptor del tema, pone seis propiedades de contexto y carga `ui/qml/Main.qml`.
5. Conecta `Planta` al gateway (`planta.conectar(pub, cmd)`): el HMI siempre se suscribe al arrancar, de modo que si ya hay un gateway corriendo los datos aparecen solos.

Listing 8.1: Objetos de contexto del motor QML (`ui/app.py:208-216`)

```
        ctx = self.motor.rootContext()
        ctx.setContextProperty("planta", self.planta)
        ctx.setContextProperty("historial", self.historial)
        ctx.setContextProperty("conexion", self.conexion)
        ctx.setContextProperty("rutaAssets", QUrl.fromLocalFile(str(RUTA_ASSETS) + "/").toString())
        ctx.setContextProperty("datosInstitucion", self.datos)
        ctx.setContextProperty("configVentana", {"ancho": self.args.ancho, "alto": self.args.alto,
                                                 "pantallaCompleta": bool(self.args.pantalla_completa)
    })
        self.motor.load(QUrl.fromLocalFile(str(RUTA_MAIN)))
```

Argumento	Efecto
`--dev`	Recarga en caliente: un `QFileSystemWatcher` vigila `ui/qml/**` y el sistema de diseño. Un guardado espera 250 ms (para agrupar), limpia la caché de componentes y recarga `Main.qml` conservando la geometría de la ventana y la conexión ZeroMQ.
`--lanzar-gateway MODO`	Arranca `python -m gateway --modo MODO` como subproceso y lo cierra al salir.
`--gateway-falso`	Arranca el publicador sintético de `ui/herramientas/gateway_falso.py`.
`--pub`, `--cmd`	Endpoints de otro gateway (otra máquina, otros puertos).
`--ancho`, `--alto`, `--pantalla-completa`	Tamaño de la ventana (defecto 1600×900).
`--arrancar`, `--autocerrar S`	Pulso de START a los 1,5 s; salir a los S segundos con un resumen (pruebas).
`--sin-conexion`	No abrir sola la ventana de conexión.

Tabla 8.1: Argumentos de `ui/app.py`.

8.2 `Planta`: el puente entre Python y QML

`Planta` (`ui/cliente.py`) es un `QObject` con unas cuarenta propiedades notificables de solo lectura (`conectado`, `modo`, `etapa`, `tEtapa`, `bits`, `ints`, `palabras`, `pos`, `posNeum`, `carreras`, `alarmas`, `latenciaMs`, `parametros`, `mando`, `suave`, `tema`…), un modelo de eventos y un conjunto de slots invocables desde QML:

- órdenes: `pulso`, `escribir`, `entrada`, `fisicaAuto`, `fuenteOpc`;
- conexión: `conectar`, `desconectar`;
- mando: `remotoHabilitar`, `mandoTomar`, `mandoLiberar`;
- preferencias: `reiniciarCalibracion`, `fijarModoSuave`, `fijarTema`.

Un componente QML se enlaza a ellas directamente:

Listing 8.2: Un binding típico: la etapa activa del GRAFCET y su barra de temporizador

```
EtapaGrafcet {
    activa: planta.etapa === modelData.numero
    conTemporizador: modelData.timer >= 0
    progreso: modelData.timer >= 0 && (planta.ints[modelData.tagPre] || 0) > 0
              ? (planta.ints[modelData.tagAcc] || 0) / planta.ints[modelData.tagPre] : 0
}
```

Cuando `Planta` emite `etapaChanged` o `intsChanged`, el motor reevalúa esas expresiones y redibuja solo lo afectado. No hay código que «actualice la pantalla».

8.2.1 Cómo llegan los datos sin congelar la pantalla

- **Estado (SUB).** Un `QTimer` de precisión de 15 ms, en el hilo de la GUI, vacía el socket SUB con `recv_multipart(zmq.NOBLOCK)` hasta que no queda nada. Se queda con el *último* snapshot y con *todos* los eventos. Como nunca bloquea, una ráfaga de mensajes viejos se descarta en una vuelta y la pantalla muestra siempre lo más reciente.
- **Notificación agrupada.** Al aplicar un snapshot, `_poner` anota qué propiedades cambiaron y `_emitir` emite una sola señal por propiedad al final de la vuelta (listado 8.4). Si en una vuelta llegaron tres snapshots, QML recalcula una vez, no tres.
- **Órdenes (REQ).** Un hilo aparte (`hmi-ordenes`) tiene su propio socket REQ y una cola. Las respuestas vuelven a la GUI con una señal conectada con `Qt.QueuedConnection`, que Qt ejecuta en el hilo de la GUI. Una orden que tarda 2 s no detiene ni un cuadro.
- **Tendencias.** El cronograma se muestrea a 10 Hz y, junto con la serie de latencia (240 muestras), se publica a QML a 5 Hz. Los `Canvas` se redibujan como mucho cinco veces por segundo.
- **Vigilancia.** Si en 2 s no llega ningún snapshot, `Planta` marca la planta sin conexión y lo anota una sola vez.

Listing 8.3: Vaciado del SUB sin bloquear (`ui/cliente.py:589-604`)

```
        try:
            while True:
                partes = self._sub.recv_multipart(zmq.NOBLOCK)
                if len(partes) != 2:
                    continue
                topic, carga = partes
                try:
                    obj = protocolo.desempaquetar(carga)
                except Exception:
                    continue
                if topic == protocolo.TOPIC_SNAP and isinstance(obj, dict):
                    ultimo = obj
                elif topic == protocolo.TOPIC_EVT and isinstance(obj, dict):
                    eventos.append(obj)
        except zmq.Again:
            pass
```

Listing 8.4: Notificación agrupada de propiedades (`ui/cliente.py:513-521`)

```
    def _poner(self, nombre, valor):
        if self._v[nombre] != valor:
            self._v[nombre] = valor
            self._sucio.add(nombre)

    def _emitir(self):
        sucio, self._sucio = self._sucio, set()
        for nombre in sucio:
            getattr(self, nombre + "Changed").emit()
```

8.2.2 Lógica que vive en el cliente

- **Posición de los vástagos**, según el modo:
  - SIM y EMULATE: la da el gateway;
  - REAL: la calcula `EstimadorCarrera` con la calibración de `datos/calibracion.json`;
  - modo suave: la da `Suavizador`.

  El estimador se alimenta también en SIM, para mostrar la calibración medida («ext 0,75 s · ret 0,74 s» junto a cada cilindro).
- **Registro.** Los cambios de etapa («X2 → X3») y de ciclo se anotan comparando contra la etapa *leída*, nunca contra la prevista.
- **Confirmación de parámetros.** Un parámetro escrito se marca confirmado cuando la relectura del PRE coincide (`CONFIRMACION_PARAMETRO` de `core/tags.py`).
- **Preferencias.** `modoSuave` y `tema` se guardan en `datos/preferencias_hmi.json`.

8.3 La pantalla

8.3.1 Ventana y pantalla separadas

`Main.qml` es solo la `ApplicationWindow`: tamaño, pantalla completa, la ventana de conexión y un temporizador que la abre a los 2 s si no hay gateway. Toda la pantalla está en `PantallaHmi.qml`, un `Item` de 1920×1080. Se separó así (sesión 12) para que Qt Design Studio, que edita `Item` pero no ventanas, pudiera abrirla (capítulo 11).

8.3.2 Rejilla de 12 columnas

La distribución sigue el sistema de diseño del autor: rejilla de 12 columnas con huecos de 16 px a 1920×1080 y modo compacto (huecos de 10 px) por debajo de 1600×940. Las fórmulas viven en `RejillaHmi.qml`:

Listing 8.5: Rejilla de la pantalla (`ui/qml/componentes/RejillaHmi.qml:18-29`)

```
    readonly property int g: compacto ? 10 : UI.Theme.espL
    readonly property real col: (width - 2 * g - 11 * g) / 12
    function ancho(n) { return n * col + (n - 1) * g }
    function xCol(c) { return g + c * (col + g) }

    readonly property real util: height - 2 * g - 2 * g
    readonly property real h1: Math.round(util * (compacto ? 0.57 : 0.53))
    readonly property real h2: Math.round(util * (compacto ? 0.225 : 0.25))
    readonly property real h3: util - h1 - h2
    readonly property real y1: g
    readonly property real y2: y1 + h1 + g
    readonly property real y3: y2 + h2 + g
```

A 1920 px de ancho, cada columna mide $(1920 - 32 - 176)/12 = 142{,}7$ px. Un panel de n columnas mide $n \cdot \text{col} + (n-1) \cdot g$. La figura 8.1 muestra qué archivo QML dibuja cada zona.

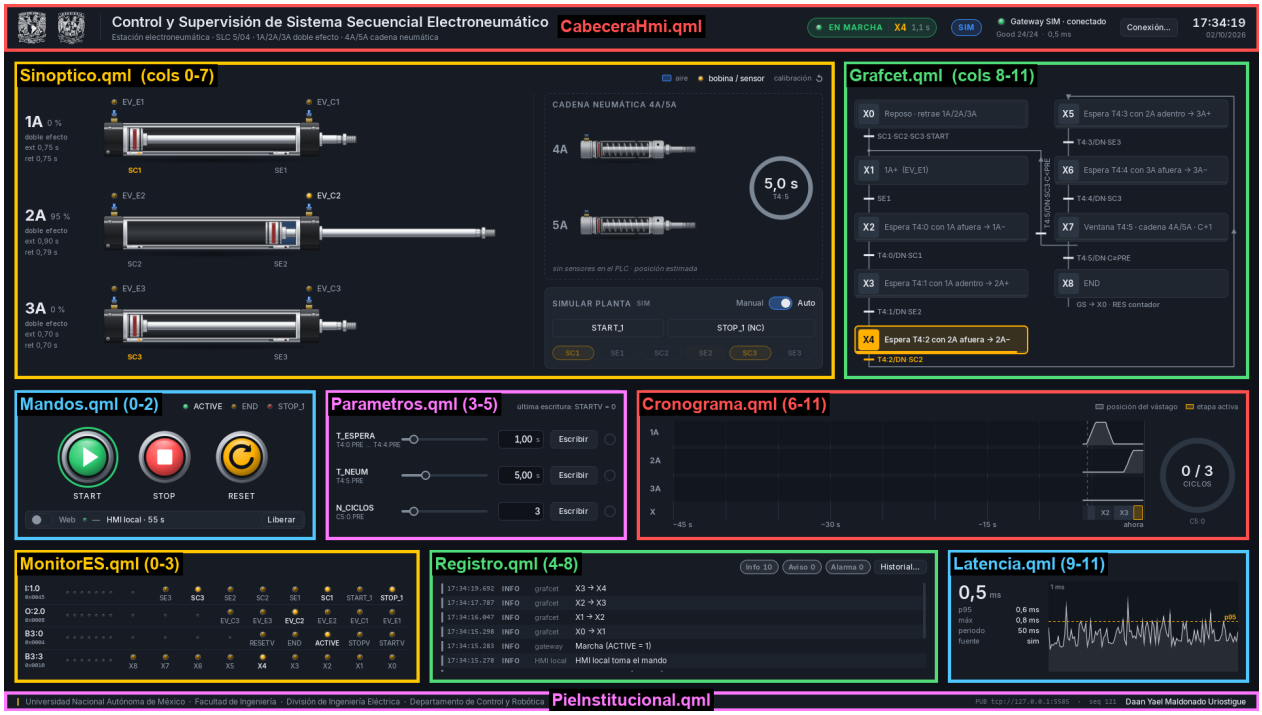


Figura 8.1: Mapa de componentes de la pantalla: cada panel es un archivo QML de `ui/qml/componentes/`, con las columnas que ocupa en la rejilla de 12.

8.3.3 Mandos y confirmación

Los tres botones llaman a `planta.pulso("STARTV" | "STOPV" | "RESETV")`. El botón no se da por «hecho» al pulsarlo, sino cuando el PLC lo confirma:

- START, cuando `ACTIVE` sube;
- STOP, cuando `ACTIVE` baja;
- RESET, cuando la etapa vuelve a X0.

Entonces se enciende el anillo verde del botón. Es la diferencia entre «envié la orden» y «la planta la ejecutó», y con un enlace de 1 s de latencia es la única forma honesta de mostrarlo.

8.3.4 Temas oscuro y gris

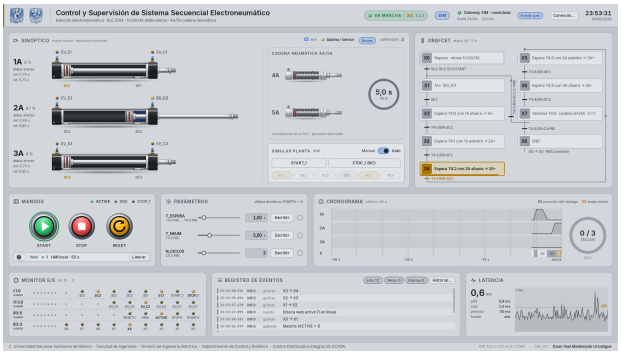
El sistema de diseño original es un tema oscuro industrial. En la sesión 14, a petición del autor («los fondos oscuros son malos»), se añadió una variante **gris** al estilo de ISA-101, conmutable en caliente desde la cabecera (figura 8.2). Para no modificar el sistema de diseño, que vive en `assets/` y es de solo lectura para el código, `ui/app.py` instala un *interceptor de URL*. Cuando el motor pide el `Theme.qml` de `assets/`, recibe `ui/qml/tema/Theme.qml`, que tiene las mismas propiedades más `variante`, `gris`, `sufijoIcono` y `sufijoLogo`. Un `Binding` en `Main.qml` enlaza `UI.Theme.variante` con `planta.tema`.

Componente	Columnas	Contenido
`CabeceraHmi.qml`	72 px	Escudos UNAM y FI, título, píldora EN MARCHA / DETENIDA / FIN / SIN DATOS con la etapa y su tiempo, chip de modo (SIM azul, EMULATE ámbar, REAL rojo), LED de conexión con calidad y latencia, chip «Fondo gris», botón «Conexión…», reloj.
`BandaAlarmas.qml`	0/30 px	Banda roja (o ámbar si solo hay avisos) bajo la cabecera, visible solo con alarmas activas.
`Sinoptico.qml`	0–7	1A/2A/3A con bobinas, sensores, porcentaje y carreras calibradas; cadena 4A/5A con cuenta regresiva; panel «Simular planta» (solo SIM/EMULATE); botones «Suave» y calibración.
`Grafcet.qml`	8–11	Etapas X0–X8 en dos columnas, receptividades, barra del TON, divergencia X7→X1/X8 y GS.
`Mandos.qml`	0–2	Botones industriales START/STOP/RESET con anillo de confirmación; LEDs `ACTIVE`/`END`/`STOP_1`; franja de mando web.
`Parametros.qml`	3–5	`T_ESPERA`, `T_NEUM` (segundos con dos decimales, centésimas internas) y `N_CICLOS`: deslizador, campo, botón «Escribir» y marca de confirmación.
`Cronograma.qml`	6–11	Gantt en vivo de 45 s con la posición de los tres vástagos y la banda de etapas; anillo `C5:0.ACC` / `C5:0.PRE`.
`MonitorES.qml`	0–3	`I:1.0`, `O:2.0`, `B3:0`, `B3:3` bit a bit (15…0) con LED y nombre, y el valor hexadecimal de la palabra.
`Registro.qml`	4–8	Eventos con hora, nivel, origen y mensaje; filtro por nivel; botón «Historial…».
`Latencia.qml`	9–11	Lectura actual, p95, máximo, periodo, fuente y tendencia con línea de p95.
`PieInstitucional.qml`	28 px	Universidad, facultad, división, departamento, materia, endpoint, secuencia y autor.

Tabla 8.2: Componentes de la pantalla principal.



(a) Tema oscuro (sistema de diseño original).



(b) Tema gris ISA-101 (sesión 14).

Figura 8.2: Las dos variantes del tema, con la planta en marcha en modo SIM.

8.4 Ventana de conexión

En v6.1 la conexión se configuraba desde la propia ventana: modo, servidor, topic y un selector OPC con cuatro listas. El nuevo HMI lo repone con `VentanaConexion.qml` y el objeto `Conexion` (`ui/conexion.py`), sin romper la arquitectura. El HMI sigue sin hablar COM: *lanza y vigila subprocesos* con `QProcess` y pide el descubrimiento OPC a `gateway/diagnostico_opc.py`.

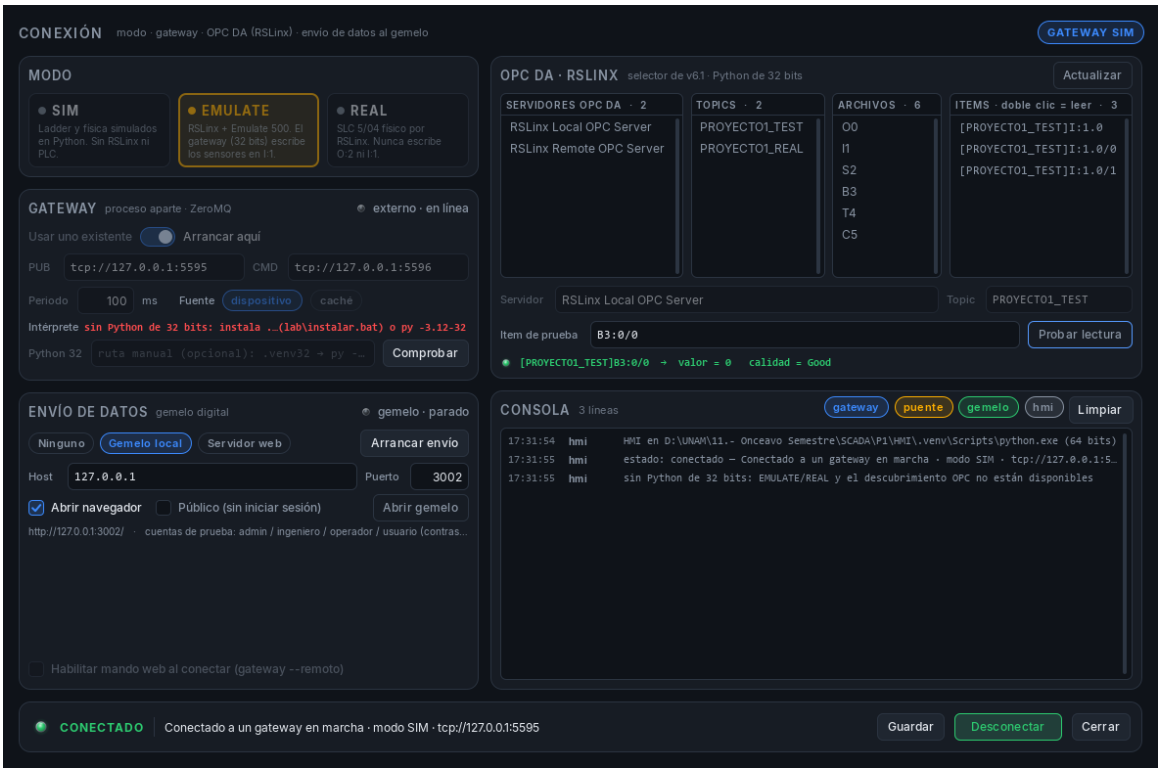


Figura 8.3: Ventana de conexión: modo, gateway, selector OPC de cuatro listas, envío al gemelo y consola de subprocesos.

- **Modo.** Tres tarjetas: SIM (azul), EMULATE (ámbar) y REAL (rojo). En REAL, conectar abre un diálogo de confirmación.
- **Gateway.** «Arrancar aquí» o «Usar uno existente», endpoints, periodo, fuente OPC. Para EMULATE y REAL busca un Python de 32 bits en este orden:

  1. la ruta manual;
  2. `.venv32\Scripts\python.exe`;
  3. `py -3.12-32`.

  Cada candidato se comprueba con `sys.maxsize <= 2**32`. Si el puerto PUB ya está ocupado por un gateway en marcha, se conecta a él en lugar de lanzar otro.
- **OPC DA.** Cuatro listas encadenadas (servidores → topics → archivos → items). Doble clic en un item lo lee y muestra valor y calidad. Cada consulta es un `QProcess` de `diagnostico_opc --json`.

- **Envío de datos.**
  - Ninguno.
  - Gemelo local: `python -m digital_twin.local`.
  - Servidor web: `python -m puente --url …`, con MQTT opcional.

  Incluye la vinculación con el servidor: el usuario admin escribe su contraseña, el HMI pide la clave del puente por HTTPS (`POST /api/puente/clave`) y la guarda sin mostrarla.
- **Consola.** Las últimas 300 líneas de los subprocesos, con filtro por origen.

8.5 Visor de historial

`VentanaHistorial.qml` con el objeto `Historial` (`ui/historial.py`) consulta en solo lectura la base del gateway. Tiene tres pestañas (eventos, alarmas, escrituras), filtros (1 h, 8 h, 24 h, todo; niveles; texto; tag) y exportación a CSV en `datos/exportes/`. No usa diálogo de archivo porque `QtQuick.Dialogs` difiere entre Qt 5 y Qt 6.

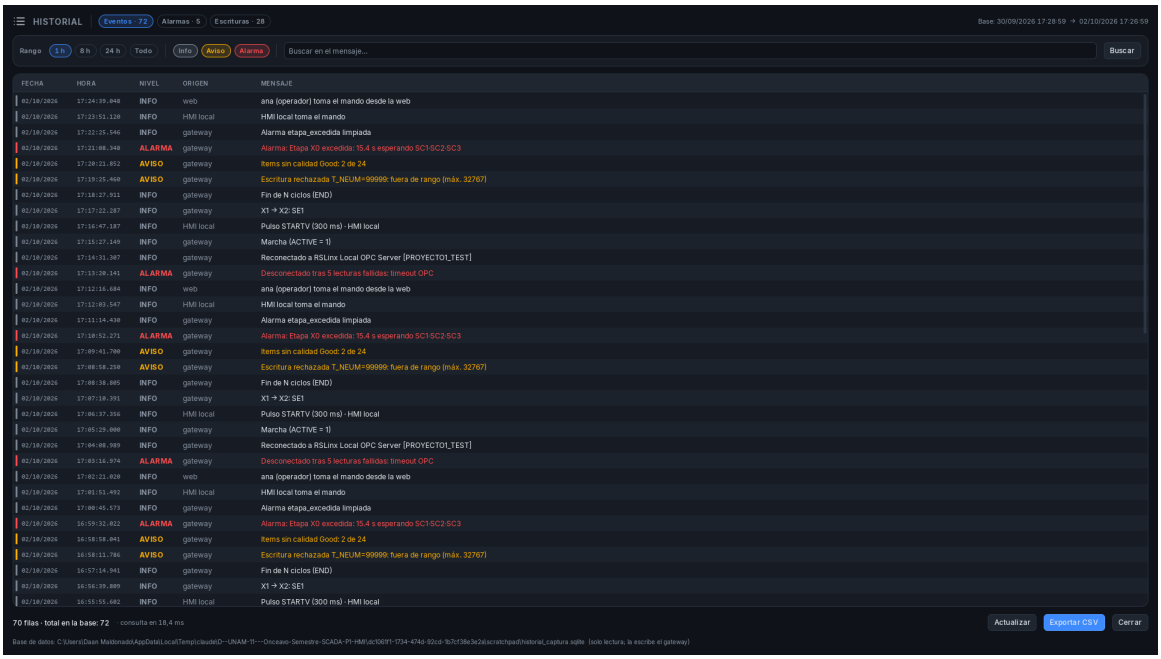


Figura 8.4: Visor del historial SQLite.

8.6 Herramientas de la interfaz

`ui/herramientas/captura.py`
: Arranca un gateway (real en SIM o falso) y el HMI sin ventana visible (plataforma `offscreen` en Qt 6), espera, captura con `grabWindow()` a 1920×1080 y 1366×768 y guarda PNG. Las capturas de este documento se hicieron así.

`ui/herramientas/gateway_falso.py`
Publicador sintético que cumple el protocolo. Lleva un Ladder simplificado, física, latencia con distribución lognormal y picos, y alarmas de demostración. Sirve para diseñar sin el gateway real.

`ui/herramientas/generar_dummydata.py`
Genera los datos de prueba de Qt Design Studio (capítulo 11).

CAPÍTULO 9

Assets y sistema de diseño

Los gráficos del HMI no son imágenes sueltas: forman un *sistema de diseño* preparado por el autor en `assets/assets_cilindros/`. Tiene cilindros vectoriales por capas, botones, LEDs, iconos, tokens de color y tipografía, y componentes QML base. El código lo trata como **solo lectura**: lo importa en su sitio, sin copiarlo, y si un componente necesita cambios crea una versión extendida en `ui/qml/componentes/`. Este capítulo explica cómo está hecho cada elemento y cómo lo usa el HMI.

9.1 Organización

Listing 9.1: Contenido de `assets/assets_cilindros/`

```
assets_cilindros/
|-- iso15552_d32/   cilindro perfilado ISO 15552 ø32 × 200, doble efecto, magnético
|   |-- camisa.svg, camisa_corte.svg, interior.svg, vastago.svg, sensor_on/off.svg
|   |-- geometria.json   coordenadas en mm (caras, tubo, sensores, carrera)
|   `-- png/             las mismas capas rasterizadas a 4 px/mm
|-- iso6432_d20/    minicilindro ISO 6432 ø20 × 50, simple efecto, retorno por muelle
|   `-- ... + muelle.svg (dibujado relajado); png/ a 8 px/mm
|-- ui/             botones/ (20), leds/ (10), iconos/ (54), qml/ (8 componentes + qmldir),
|                   tokens.json, DISENO.md, diseno.html
|-- logos/          escudos UNAM y FI, azul (fondo claro) y blanco (fondo oscuro)
|-- fuentes/        Inter Regular, Medium, SemiBold, Bold (SIL Open Font License 1.1)
|-- herramientas/   gen_cilindros.py, gen_ui.py, preview.py (generadores)
|-- Cilindro.qml    componente de ejemplo que apila las capas
|-- preview.html    prueba en navegador con deslizador
`-- lamina_preview.png
```

9.2 Cilindros vectoriales por capas

9.2.1 Un milímetro, una unidad

Cada cilindro está dibujado en vista lateral realista: aluminio anodizado, tapas negras, vástago cromado, racores *push-in* con tubo azul y sensores magnéticos con LED. La convención clave es que **una unidad SVG equivale a un milímetro** y que **todas las capas de un cilindro comparten el mismo `viewBox`**:

- 580×78 para el ø32;
- 215×40 para el ø20.

Ese ancho deja sitio para el vástago completamente extendido. Así las capas se apilan sin transformaciones y se escalan con un único factor, los píxeles por milímetro:

$$\text{ppm} = \frac{\text{ancho en pantalla (px)}}{\text{ancho del viewBox (mm)}}. \tag{9.1}$$

El vástago está dibujado *retraído*. Para una posición normalizada $p \in [0, 1]$, basta desplazar su capa

$$\Delta x = p \cdot s \cdot \text{ppm}, \qquad s = \text{carrera en mm (200 o 50)}. \tag{9.2}$$

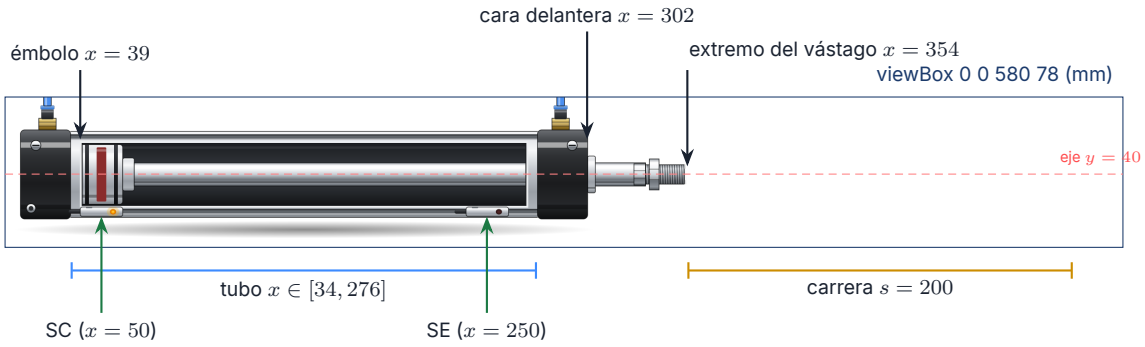


Figura 9.1: Sistema de coordenadas del cilindro ISO 15552 ø32 según `geometria.json`. Todas las capas comparten este rectángulo; el vástago se dibuja retraído y se desplaza $p \cdot 200$ mm.

9.2.2 Las capas y su orden

El orden de apilado, que reproduce `ui/qml/componentes/CapasCilindro.qml`, es:

1. `interior.svg`, el fondo oscuro del tubo, visible a través de la ventana de corte;
2. la **cámara presurizada**, un rectángulo azul (`UI.Theme.aire`) al 28 % de opacidad:
   - con `EV_E` energizada, entre la tapa trasera y el émbolo;
   - con `EV_C` energizada, entre el émbolo y la tapa delantera;
3. `muelle.svg` (solo el minicilindro), escalado en x;
4. `vastago.svg`, desplazado Δx;
5. `camisa_corte.svg`, que tapa los bordes del émbolo y deja ver el interior por la ventana;
6. `sensor_on.svg` o `sensor_off.svg` en las posiciones de `SC` y `SE`. El sensor mide 30×8 mm y tiene el cuerpo centrado en $(17, 4)$, así que se coloca en $x = c_x - 17$, $y = c_y - 4$, con ancho $30 \cdot$ ppm.

Listing 9.2: Vástago desplazado y muelle comprimido (`CapasCilindro.qml:56-72`)

```
1    Image {   // muelle: se comprime alrededor de su apoyo delantero
```

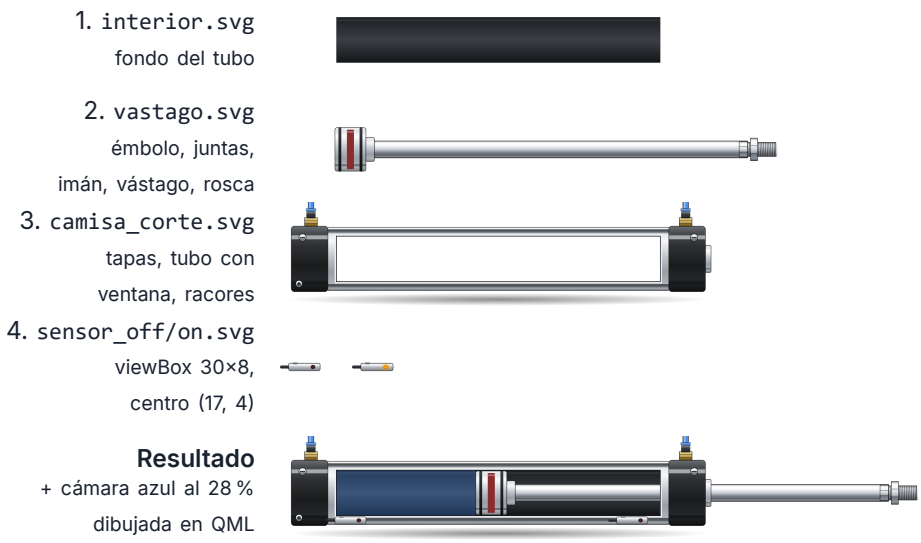


Figura 9.2: Capas del cilindro, de abajo arriba, y su composición. La cámara presurizada no está en ningún SVG porque su ancho depende de la posición: es un `Rectangle` de QML.

```
        anchors.fill: parent; visible: root.corte && root.muelleL > 0
        source: root.muelleL > 0 ? root.carpeta + "/muelle.svg" : ""
        sourceSize: Qt.size(Math.max(1, root.width), Math.max(1, root.height))
        smooth: true
        transform: Scale {
            origin.x: root.muelleApoyoX * root.ppm
            xScale: root.muelleL > 0 ? Math.max(0.08, (root.muelleL - root.posicion * root.carreraMm) /
    root.muelleL) : 1
        }
    }
    Image {   // vástago + pistón, dibujado retraído
        width: root.width; height: root.height
        x: root.dx
        source: root.carpeta + "/vastago.svg"
        sourceSize: Qt.size(Math.max(1, root.width), Math.max(1, root.height))
        smooth: true
    }
```

9.2.3 El muelle

El muelle del minicilindro está dibujado relajado, con longitud $L = 60$ mm y apoyo delantero en $x_{\text{apoyo}} = 95{,}5$ mm. Al extenderse el vástago, el émbolo avanza $p \cdot s$ y el muelle se comprime contra la tapa delantera. Se representa con una transformación de escala en x con origen en el apoyo:

$$\text{xScale} = \max\left(0{,}08,\ \frac{L - p\,s}{L}\right) = \max\left(0{,}08,\ 1 - \frac{50\,p}{60}\right). \tag{9.3}$$

Con el vástago fuera ($p = 1$) el muelle queda al $16{,}7$ % de su longitud. El 0,08 es un tope de seguridad para que la escala nunca llegue a cero. Es una aproximación visual: un muelle real se comprime

reduciendo el paso entre espiras, no el grosor del alambre, pero a la escala del HMI la diferencia no se aprecia.

9.2.4 Por qué SVG por capas

Ventajas del enfoque por capas frente a sprites o 3D

- **Nítido a cualquier tamaño.** La pantalla se adapta de 1366×768 a 1920×1080. Un SVG se rasteriza al tamaño exacto en pantalla (`sourceSize`) y nunca se ve borroso.
- **Posición continua.** Con sprites (fotogramas prerenderizados) habría que elegir cuántas posiciones dibujar. Con capas, la posición es un número real y cualquier valor es válido.
- **Fiel a la geometría.** Las cotas en milímetros salen de `geometria.json`. El sensor está donde estaría en el cilindro real y la carrera dibujada es la carrera real.
- **Ligero.** Cada capa pesa entre 8 y 24 KB. Sin motor 3D ni GPU dedicada, corre también en la VM de Windows 7, donde el README recomienda usar las capas PNG si el SVG pesa.

9.2.5 Restricciones de QtSvg

QtSvg implementa el perfil *SVG Tiny 1.2*, más limitado que un navegador. Los SVG del proyecto usan solo degradados lineales y radiales, `rect`, `circle`, `path`, `text` y `transform`. No usan filtros, patrones, máscaras ni CSS. El README advierte que no se «optimicen» con herramientas que conviertan estilos a clases CSS, porque QtSvg las ignora y la imagen sale sin colores.

9.2.6 Cómo se generaron

Los cilindros no se dibujaron a mano en Inkscape: los genera un programa. `herramientas/gen_cilindros.py` (26 KB, Python puro con `math` y `json`) escribe las cadenas SVG a partir de dos diccionarios de parámetros:

Listing 9.3: Parámetros de los generadores (`gen_cilindros.py:18-21`, resumidos)

```
BIG  = dict(bore=32, stroke=200, rod=12, thread_d=10, thread_len=20, wh=26, cap_len=26, prof=46)
MINI = dict(bore=20, stroke=50,  rod=8,  thread_d=8,  thread_len=12, wh=20, cap_len=16, tube_d=22,
    cap_d=24)
```

Los degradados predefinidos (`alu`, `blk`, `chrome`, `thread`, `inox`, `brass`, `plast`, `bore`, `goma`, `spring`…) dan el aspecto metálico. Cambiar la carrera o el diámetro y volver a ejecutar regenera todas las capas y `geometria.json`. `herramientas/gen_ui.py` genera botones, LEDs e iconos a partir de los colores de `tokens.json`, y `herramientas/preview.py` compone la lámina de comprobación con cairosvg y PIL (figura 9.3). Las herramientas tienen rutas de salida de Linux fijas, porque se escribieron fuera de esta máquina. Para este documento, las capas se compusieron con QtSvg mediante `docs/documentacion_tecnica/figuras/generar_figuras.py`.

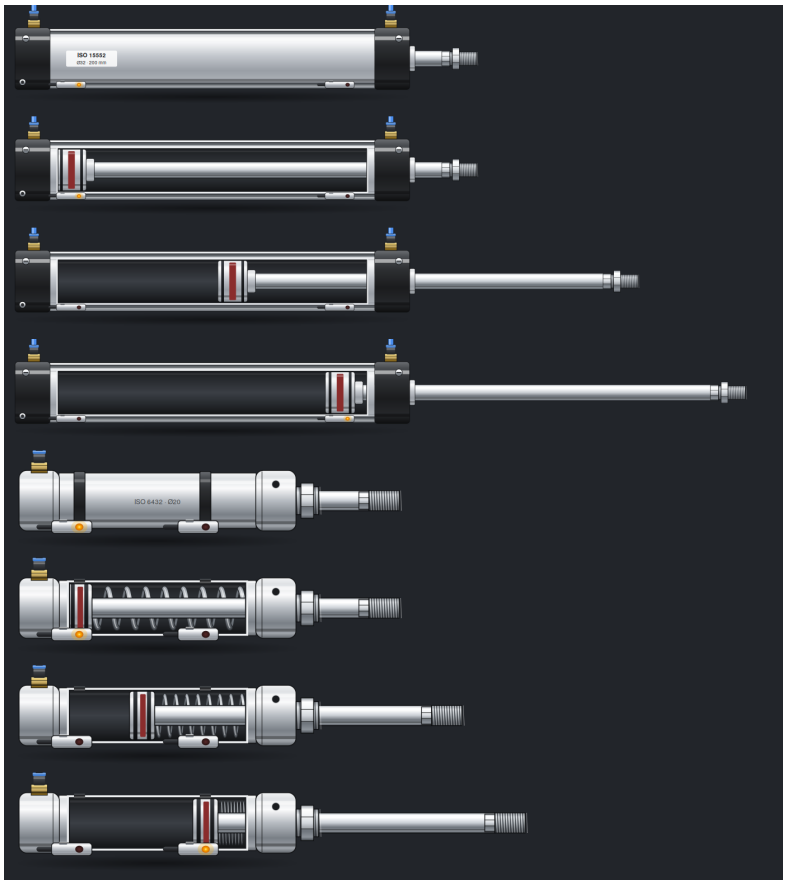


Figura 9.3: Lámina de comprobación del autor: los dos cilindros en varias posiciones, compuestos a partir de las capas.

9.3 Tokens de color y significado

`ui/tokens.json` y el singleton `ui/qml/Theme.qml` tienen el mismo contenido. La regla, anotada en el propio archivo, es: «color solo para estado; en reposo todo es gris (ISA-101)». En QML no se escriben colores hexadecimales sueltos: todo sale de `UI.Theme`.

	Token	Oscuro	Gris	Significado
	`fondo`	`#0F1319`	`#BFC4CA`	Fondo de la pantalla
	`surface`	`#171C24`	`#D3D7DB`	Paneles
	`borde`	`#2C3542`	`#99A0A9`	Bordes, etapas inactivas
	`texto`	`#E6EAF0`	`#1B2129`	Texto principal
	`neutro`	`#7B8794`	`#6B7480`	Inactivo, deshabilitado, sin dato
	`activo`	`#FFB000`	`#B07200`	Etapa activa, bobina energizada, sensor activo, advertencia
	`ok`	`#2ECC71`	`#178A47`	En marcha (`ACTIVE`), calidad Good, escritura confirmada
	`alarma`	`#FF4D4F`	`#CC2B2B`	STOP, alarma, desconectado, modo REAL
	`aire`	`#3D8BFF`	`#1B5FD1`	Aire presurizado en las cámaras, acciones del sistema, modo SIM
	`unamOro`	`#C9A227`	`#9C7A12`	Solo la franja de la cabecera
	`unamAzul`	`#1B3A6B`	`#1B3A6B`	Identidad institucional

Tabla 9.1: Tokens de color del sistema de diseño (variante oscura original y variante gris de `ui/qml/tema/Theme.qml`).

Otros tokens:

- **Tipografía:** título 22 px (peso 600), sección 14 px en mayúsculas con espaciado de 0,08, cuerpo 14 px, etiqueta 12 px, valor 28 px con cifras tabulares.
- **Espaciado:** 4, 8, 12, 16, 24 y 32 px.
- **Radios:** 12 px en paneles y 8 px en controles.
- **Animación:** rápida 120 ms, normal 220 ms, vástago 150 ms, pulso de botón 300 ms.

9.4 Botones, LEDs e iconos

- **Botón industrial.** Bisel metálico con degradado radial, anillo, casquete de color con brillo superior y halo. El color sigue el token: START = `ok`, STOP = `alarma`, RESET = `activo`, POWER = `aire`. El componente `BotonIndustrial.qml` cambia de SVG según el estado del ratón, escala la imagen a 0,97 al pulsar (animado en 120 ms) y muestra un anillo de confirmación con opacidad animada (220 ms).
- **Botón plano** (160×44) para acciones secundarias, en los mismos cuatro estados.

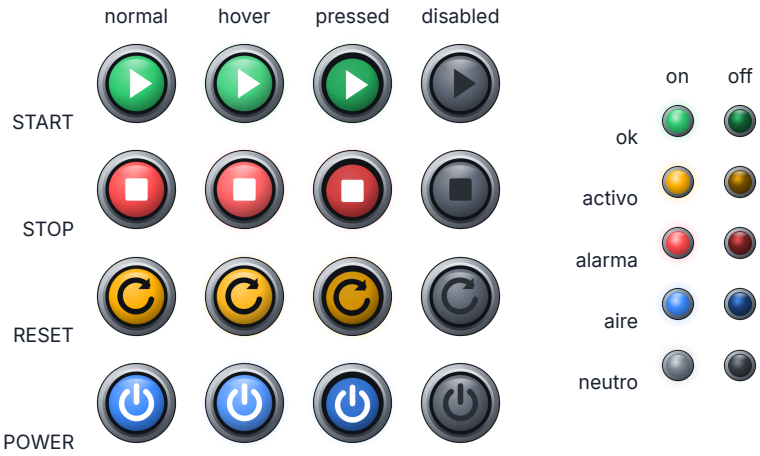


Figura 9.4: Botones industriales (viewBox 100×100) en sus cuatro estados y LEDs con bisel (40×40) encendidos y apagados. POWER no se usa en la pantalla actual.

- **LED con bisel**, en cinco colores. `MonitorES.qml` usa `activo` y `neutro` para los dieciséis bits de cada palabra; `Led.qml`, el resto.
- **Iconos de línea** de 24×24 en tres variantes: `_claro` sobre fondo oscuro, `_oscuro` sobre fondo claro (tema gris) y `_acento` en azul. El tema elige la variante con `UI.Theme.sufijoIcono`.



Figura 9.5: Los 18 iconos del sistema de diseño (variante `_claro` sobre el color `surface`). La primera fila son los que usan las cabeceras de los paneles.

9.5 Logos y tipografía

Los escudos de la UNAM y de la Facultad de Ingeniería están en PNG de 331×392 px, en azul con fondo transparente y en blanco para la cabecera oscura. Según `ui/DISENO.md` provienen del repositorio público `unamfi/Tesis`. Para una versión oficial en vector hay que pedirlos a Comunicación Social de la FI. La cabecera elige la variante con `UI.Theme.sufijoLogo` y los dibuja con `sourceSize.height = 104` y *mipmap* para que se vean suaves al reducirlos.

La tipografía es **Inter** (versión 4.001, © The Inter Project Authors, licencia SIL Open Font License 1.1, incluida en `fuentes/`). Es una sans serif diseñada para pantallas, con cifras tabulares (todas del mismo ancho, para que los valores numéricos no «bailen» al cambiar) y buena legibilidad a tamaños pequeños. El HMI la registra desde Python; el respaldo es Segoe UI. Este documento usa la misma

familia:

Regular 0123456789 Medium **SemiBold** **Bold**

9.6 Cómo carga el HMI los assets

- **Imágenes de la pantalla**: `ui/app.py` pone en el contexto `rutaAssets` con la URL `file:///.../assets/assets_cilindros/`, y cada componente le concatena la subruta, por ejemplo `rutaAssets + "iso15552_d32"` o `rutaAssets + "logos/unam_blanco.png"`.
- **Componentes del sistema de diseño** (`Led`, `BotonIndustrial`, `EtapaGrafcet`, `AnilloProgreso`…): se importan en su sitio con `import "../../assets/assets_cilindros/ui/qml" as UI`. Sus imágenes se resuelven respecto al propio archivo del componente con `UI.Theme.rutaUi = "../"`.
- **Nitidez**: toda `Image` de SVG fija `sourceSize` igual a su tamaño en pantalla. Así QtSvg rasteriza una sola vez al tamaño justo, sin reescalar un mapa de bits ni volver a rasterizar en cada cuadro.
- **Lo que no se usa**: las capas PNG (pensadas para máquinas sin GPU), `camisa.svg` sin corte, los botones planos y el botón POWER.

CAPÍTULO 10

Animaciones

10.1 Principios

Las animaciones del HMI no son decoración. Cada una responde a un problema de lectura de la planta y sigue cuatro reglas:

1. **Animar entre muestras, no inventar muestras.** El gateway entrega el estado de 3 a 20 veces por segundo. Sin animación, un vástago saltaría de posición en posición; con ella, se desliza entre dos posiciones conocidas.
2. **La duración la dicta la física.** En REAL, la carrera se anima con el tiempo de carrera *medido* en esa misma planta (mediana de las últimas cinco carreras), no con un valor estético.
3. **Nunca adelantar lo que depende de un sensor.** Una transición del GRAFCET o la llegada a un extremo no se muestra hasta que se lee. Si la animación se adelantó, se corrige con un ajuste visual corto y visible.
4. **Movimiento y color solo para el estado.** Siguiendo ISA-101, en reposo no se mueve nada. El parpadeo se reserva para las alarmas activas.

10.2 Las herramientas de Qt Quick

QML ofrece animaciones declarativas que el motor ejecuta en su propio reloj, sincronizado con el refresco de la pantalla (*vsync*), sin código Python:

`Behavior on <propiedad>`
: Intercepta cualquier cambio de la propiedad y lo convierte en una animación. Es la herramienta principal: el binding asigna el valor nuevo y el `Behavior` lo interpola.

`NumberAnimation`, `ColorAnimation`
: Interpolan un número o un color en una duración dada con una curva de suavizado (*easing*).

`SequentialAnimation`, `ParallelAnimation`
: Componen animaciones; con `loops: Animation.Infinite` se repiten.

`Timer`
: Dispara acciones periódicas o diferidas en el hilo de la GUI (reloj de la cabecera, avisos que se borran solos).

`Canvas`
: Dibujo imperativo en 2D con la API de `CanvasRenderingContext2D` de HTML5. Se usa para el GRAFCET, el cronograma, la tendencia de latencia y los anillos de progreso.

`Shape` / `ShapePath`
: Trazos vectoriales acelerados: el marco punteado de la cadena neumática y la palomita de confirmación.

`transform: Scale`
: Escala con origen arbitrario: la compresión del muelle.

10.2.1 Curvas de suavizado

Una curva de suavizado $f:[0,1]\to\mathbb{R}$ convierte el tiempo normalizado $\tau = t/T$ en avance normalizado. El valor animado es $v(t) = v_0 + (v_1 - v_0)\, f(t/T)$. El HMI usa cuatro (figura 10.1):

$$\text{Linear:}\quad f(\tau) = \tau \tag{10.1}$$

$$\text{OutQuad:}\quad f(\tau) = 1 - (1-\tau)^2 \tag{10.2}$$

$$\text{OutBack}(s):\quad f(\tau) = 1 + (s+1)(\tau-1)^3 + s\,(\tau-1)^2 \tag{10.3}$$

$$\text{InOutSine:}\quad f(\tau) = \tfrac{1}{2}\big(1 - \cos(\pi\tau)\big) \tag{10.4}$$

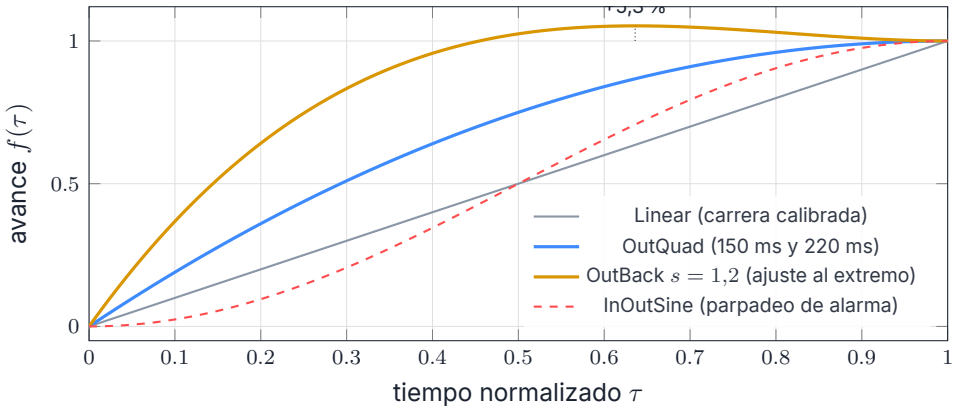


Figura 10.1: Curvas de suavizado usadas por el HMI.

OutQuad. empieza rápido y frena al llegar: la derivada en $\tau = 1$ es cero. Es la curva de los cambios que «alisan» el muestreo (vástago en SIM, minicilindros, banda de alarmas), porque el valor llega al destino sin brusquedad.

OutBack. se pasa un poco del destino y regresa. El sobrepaso máximo se obtiene derivando $g(u) = (s+1)u^3 + s\,u^2$, con $u = \tau - 1$:

$$g'(u) = 0 \;\Rightarrow\; u^{*} = -\frac{2s}{3(s+1)}, \qquad g(u^{*}) = \frac{4s^3}{27(s+1)^2}. \tag{10.5}$$

Con el valor por defecto de Qt, $s = 1{,}70158$, el sobrepaso es del 10 %. El HMI usa `easing.overshoot: 1.2`, que da $4(1{,}2)^3/(27\cdot 2{,}2^2) = 5{,}3\,\%$ en $\tau = 0{,}64$. Aplicado al tramo final de una carrera (por ejemplo, de 0,9 a 1,0), el vástago rebasa el extremo medio milímetro a escala y vuelve. Sugiere el golpe del émbolo contra la tapa sin parecer un error.

InOutSine. es simétrica y suave en ambos extremos: el parpadeo de la banda de alarmas «respira» en lugar de encenderse y apagarse de golpe.

10.3 El vástago de 1A, 2A y 3A

`Cilindro.qml` anima una propiedad `posVista` que `CapasCilindro.qml` convierte en desplazamiento de capas ($\Delta x = \texttt{posVista} \cdot s \cdot \text{ppm}$). Hay tres mecanismos, excluyentes entre sí, según de dónde venga la posición:

Listing 10.1: Las tres animaciones del vástago (`Cilindro.qml:50-57`)

```
Behavior on posVista { enabled: !root.posEstimada && !root.suave; NumberAnimation { duration: 150; easing.type: Easing.OutQuad } }
onPosicionChanged: if (!posEstimada || suave) posVista = posicion
onPosEstimadaChanged: { animCal.stop(); posVista = posicion }
onSuaveChanged: { animCal.stop(); animSalto.stop(); posVista = posicion }
Component.onCompleted: posVista = posicion

NumberAnimation { id: animCal; target: root; property: "posVista"; easing.type: Easing.Linear }
NumberAnimation { id: animSalto; target: root; property: "posVista"; duration: 140; easing.type: Easing.OutBack; easing.overshoot: 1.2 }
```

SIM y EMULATE: `Behavior` de 150 ms
La posición llega del modelo físico del gateway cada 50–100 ms. Un `Behavior` con `OutQuad` de 150 ms (el token `vastago_ms`) solo alisa el muestreo: la velocidad real la impone el gateway.

REAL: carrera calibrada
Al empezar una carrera (flanco de `moviendo` del estimador o cambio de objetivo), `animCal` anima linealmente desde la posición actual hasta el objetivo con una duración proporcional a lo que falta:

$$T_{\text{anim}} = \text{máx}\left(1,\ T_{\text{cal}} \cdot |p^{*} - p_{\text{vista}}|\right) \text{ ms.} \tag{10.6}$$

Cuando el estimador deja de moverse (llegó el sensor o venció el plazo) y la vista no está en el extremo, `animSalto` hace el ajuste en 140 ms con `OutBack`.

Modo suave: sin animación propia
`Suavizador` ya entrega una posición continua a velocidad finita en cada cuadro de 15 ms (sección 7.6). Animarla otra vez añadiría retraso, así que se asigna tal cual y las otras dos animaciones se detienen.

Listing 10.2: Carrera calibrada y ajuste al extremo en REAL (`Cilindro.qml:65-76`)

```
        if (c.moviendo) {
            if (!_movAnt || c.objetivo !== _objAnt) {      // nueva carrera: animación calibrada
                animSalto.stop(); animCal.stop()
                animCal.from = posVista; animCal.to = c.objetivo
                animCal.duration = Math.max(1, c.duracion_ms * Math.abs(c.objetivo - posVista))
                animCal.start()
            }
```

```
        } else if (Math.abs(posVista - c.pos) > 0.002 && !animSalto.running) {
            animCal.stop()                                  // llegó el sensor (o timeout): ajuste visual
            animSalto.from = posVista; animSalto.to = c.pos; animSalto.start()
        }
        _movAnt = c.moviendo; _objAnt = c.objetivo
```

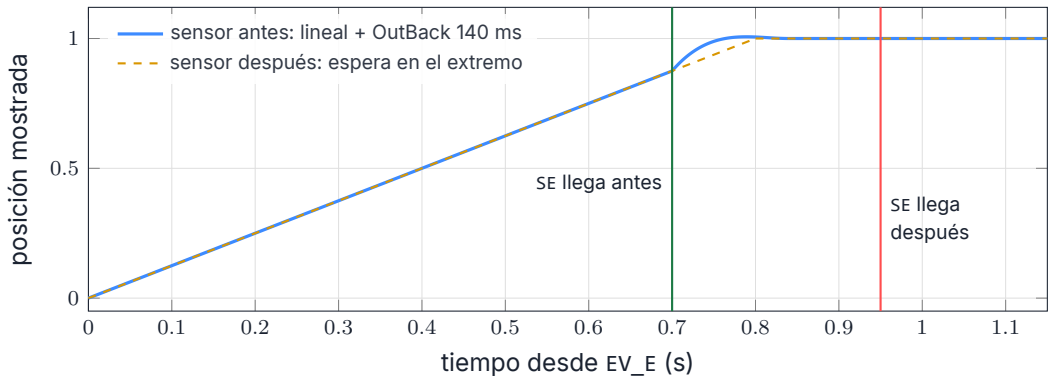


Figura 10.2: Animación de una carrera en modo REAL con $T_{\text{cal}} = 0{,}8$ s. Si el sensor de destino llega antes de lo previsto, la vista salta al extremo con un pequeño rebote; si llega después, el vástago espera en el extremo. Con cada carrera medida la calibración converge y el salto desaparece.

10.4 Cámaras, bobinas y sensores

Los estados discretos cambian *sin* animación: la cámara azul aparece en cuanto se lee la bobina, el LED de `EV_E1` se enciende y la etiqueta de `SC1` pasa a ámbar. Es deliberado. Un estado discreto es verdadero o falso, y un fundido de 200 ms haría dudar en qué momento cambió. El ancho de la cámara sí sigue al émbolo, porque depende de `posVista`:

$$w_{\text{trasera}} = (x_{\text{émbolo},0} - x_{\text{tubo},0}) \cdot \text{ppm} + \Delta x, \qquad w_{\text{delantera}} = x_{\text{tubo},1} \cdot \text{ppm} - (x_{\text{émbolo},1} \cdot \text{ppm} + \Delta x). \quad (10.7)$$

10.5 Cadena neumática 4A/5A

`CadenaNeumatica.qml` dibuja los dos minicilindros con las capas del ø20 y anima su posición con un `Behavior` de 150 ms `OutQuad`. A diferencia de `Cilindro`, ese `Behavior` sigue activo en modo suave. La cámara trasera se enciende mientras el vástago sube:

$$\text{subiendo} = (p > 0{,}005) \wedge (p \geq p_{\text{anterior}} - 0{,}0005), \quad (10.8)$$

porque durante el retorno por muelle no hay aire en la cámara. Todo el panel está al 75 % de opacidad, con un marco punteado (`ShapePath` con `dashPattern [4, 3]`) y el rótulo «sin sensores en el PLC · posición estimada»: el diseño comunica que esa posición no es una medición. A la derecha, un `AnilloProgreso` muestra la cuenta regresiva de la ventana, $(\text{PRE} - \text{ACC})/100$ segundos de `T4:5`, con el arco en ámbar durante X7.

10.6 GRAFCET

- **Etapa activa.** `EtapaGrafcet.qml` anima el color de fondo y del borde con `ColorAnimation` de 220 ms al activarse o desactivarse. Con el recorrido de la secuencia, el ámbar parece «viajar» de etapa en etapa.
- **Barra del temporizador.** Dentro de la etapa activa, una barra de ancho ACC/PRE con `Behavior on width` de 150 ms. Muestra cuánto falta para que se cumpla la espera.
- **Enlaces y transiciones.** Un `Canvas` dibuja las líneas, las flechas y las barras de transición, y solo se repinta cuando cambia la etapa activa o el tamaño. La transición de salida de la etapa activa se dibuja en ámbar y con trazo de 3 px: es la condición que la planta está esperando.

10.7 Alarmas

Listing 10.3: Banda de alarmas: aparición y parpadeo (`BandaAlarmas.qml:14-16, 26-30`)

```
    implicitHeight: hay ? 30 : 0
    clip: true
    Behavior on implicitHeight { NumberAnimation { duration: UI.Theme.animNormal; easing.type: Easing.
     OutQuad } }
        SequentialAnimation on opacity {
            running: root.hay; loops: Animation.Infinite
            NumberAnimation { from: 1.0; to: 0.45; duration: 900; easing.type: Easing.InOutSine }
            NumberAnimation { from: 0.45; to: 1.0; duration: 900; easing.type: Easing.InOutSine }
        }
```

La banda despliega su altura de 0 a 30 px en 220 ms y su fondo respira entre opacidad 1 y 0,45 con un periodo de 1,8 s (0,56 Hz), *solo* mientras haya alarmas activas. La frecuencia es baja a propósito: atrae la mirada sin cansar y queda muy por debajo del umbral de tres destellos por segundo que las pautas de accesibilidad (WCAG 2.x) consideran riesgoso para personas con fotosensibilidad. Con la banda presente, el resto de la pantalla se desplaza hacia abajo y no queda nada tapado.

10.8 Botones y controles

10.9 Tendencias

El **cronograma** es un diagrama de Gantt en vivo de los últimos 45 s. Tres bandas de área con la posición de cada vástago y una banda inferior con la etapa activa, alternando tonos; el tramo actual va en ámbar. La **latencia** es una tendencia de las últimas 240 lecturas con una línea punteada en el p95, y su escala vertical se ajusta sola: máx$(1,\ 1{,}15 \cdot \text{máx},\ 1{,}3 \cdot p_{95})$. Ambas son `Canvas` que se repintan cuando `Planta` publica datos nuevos, a 5 Hz. Repintar más seguido no aporta nada visible en una ventana de 45 s y costaría CPU en el hilo de la GUI.

Componente	Qué se anima	Duración / curva
`BotonIndustrial`	Escala de la imagen a 0,97 al pulsar; opacidad del anillo de confirmación	120 ms; 220 ms
`Interruptor`	Posición de la perilla; color de la pista	120 ms OutQuad; 120 ms
`Chip`, `BotonMini`, casillas	Color de fondo y borde	120 ms
`Check`	Color de la palomita (`Shape` con extremos redondeados)	220 ms
`PanelHmi`	Opacidad del contenido a 0,55 cuando no hay datos	220 ms
`CabeceraHmi`	Reloj (`Timer` de 1 s)	—
`VentanaConexion`	Texto «buscando…» que parpadea mientras dura un descubrimiento OPC	500 + 500 ms
`Mandos`	Aviso de RESET pedido, borrado por un `Timer`	2,5 s

Tabla 10.1: Animaciones de los controles.

10.10 Lo que no se usó, y por qué

`SpringAnimation`
: El sistema de diseño (`ui/DISENO.md`) propone un resorte (`spring: 3`, `damping: 0.3`) al llegar al final de carrera. Un resorte amortiguado sigue la ecuación $\ddot{x} = k\,(x^{*} - x) - c\,\dot{x}$. Su tiempo de asentamiento depende de los parámetros y de la distancia, y con amortiguamiento bajo oscila varias veces. La implementación usa en su lugar una `NumberAnimation` con `OutBack` de duración fija (140 ms). El código no documenta el motivo, pero la diferencia práctica es que el rebote es acotado y determinista: termina siempre a los 140 ms, con un sobrepaso del 5,3 %, y no compite con la carrera siguiente.

`MultiEffect`, sombras, resplandores y partículas
: El plan inicial mencionaba `MultiEffect` y `ParticleSystem`. `MultiEffect` solo existe en Qt 6.5+, y el QML debe correr también con PyQt5/Qt 5.15 (sección 5.2). Además, ISA-101 desaconseja los efectos decorativos en una pantalla de operación. La sombra de panel que define `tokens.json` tampoco se implementó.

10.11 Rendimiento

Las animaciones de QML corren en el hilo de la GUI, pero el dibujado lo hace el *scene graph* en el hilo de render, normalmente con la GPU. Python solo interviene una vez por vuelta de 15 ms para aplicar el snapshot, y una vez cada 200 ms para publicar las tendencias. Las SVG se rasterizan una sola vez a su tamaño (`sourceSize`), y los `Canvas` se repintan solo cuando cambian sus datos. Por eso la pantalla mantiene el refresco del monitor aunque el enlace con el PLC tarde segundos.

CAPÍTULO 11

Integración con Qt Design Studio

11.1 Qué es y por qué se incorporó

Qt Design Studio es el editor visual de The Qt Company para interfaces QML. Tiene un lienzo (*Form Editor*) donde se arrastran y colocan componentes, un árbol de elementos (*Navigator*), un inspector de propiedades, un editor de estados y línea de tiempo, y una vista previa en vivo. A diferencia de los diseñadores de formularios de las herramientas RAD, no genera un archivo binario aparte: lee y escribe directamente el mismo `.qml` que ejecuta la aplicación.

El proyecto se desarrolló primero solo desde VS Code, con recarga en caliente. El autor pidió Design Studio el 9 de octubre de 2026 (sesión 12), y se instaló la versión 4.8.3 en `E:\Qt`. Ese mismo día otra sesión escribió una guía de uso en Word (`docs/GUIA_QT_DESIGN_STUDIO.docx`). Este capítulo explica qué hubo que cambiar en el código para que el editor pueda abrir la pantalla y cómo conviven el diseño visual y la lógica en Python.

11.2 El problema: el diseñador no ejecuta Python

En ejecución, la pantalla depende de seis objetos que pone `ui/app.py` en el contexto del motor: `planta`, `conexion`, `historial`, `datosInstitucion`, `rutaAssets` y `configVentana`. Design Studio no ejecuta Python. Abierta tal cual, la pantalla mostraría errores por todas partes (`planta is not defined`), sin cilindros, sin GRAFCET y sin datos. Hubo que resolver cuatro cosas.

11.2.1 1. Separar la pantalla de la ventana

Design Studio edita un `Item`, no una `ApplicationWindow`. La pantalla se movió de `Main.qml` a `PantallaHmi.qml`: un `Item` de 1920×1080 con las propiedades `compacto` y la señal `conexionPedida()`. `Main.qml` quedó como una ventana que solo contiene esa pantalla, la ventana de conexión y el temporizador que la abre. Se comprobó píxel a píxel que el HMI se veía igual antes y después.

11.2.2 2. Datos de prueba generados, no escritos

Design Studio carga automáticamente cada archivo `dummydata/<nombre>.qml` como propiedad de contexto `<nombre>`. Por eso existen `ui/qml/dummydata/planta.qml`, `conexion.qml`, `historial.qml`, `datosInstitucion.qml`, `rutaAssets.qml` y `configVentana.qml`.

Escribirlos a mano habría sido frágil: `Planta` tiene unas cuarenta propiedades y cambia con frecuencia. En su lugar, `ui/herramientas/generar_dummydata.py` los *genera*:

1. arranca el gateway SIM en puertos propios, crea un `Planta` real, pulsa START y espera unos segundos a que la secuencia esté en marcha;
2. recorre el `staticMetaObject` de `Planta`, `Conexion` e `Historial` por introspección de Qt y escribe un `QtObject` con:
   - cada propiedad con su tipo y su valor real;
   - cada slot como función vacía;
   - cada modelo como `ListModel` con sus filas;
3. usa configuración, calibración e historial vacíos en una carpeta temporal, y escribe las rutas como relativas, para que los archivos sirvan en cualquier máquina.

El resultado es que en Design Studio la pantalla aparece *con la planta en marcha*: cilindros en posición, etapa activa en el GRAFCET, eventos en el registro y latencia en la gráfica. La prueba `tests/test_dummydata.py` falla si alguien añade una propiedad, un slot o una señal a esas clases sin regenerar los archivos.

Listing 11.1: Fragmentos del archivo generado `ui/qml/dummydata/planta.qml` (líneas 1–21, 49–54, 84 y 91)

```
// Generado por ui/herramientas/generar_dummydata.py: no editar a mano.
QtObject {
    property bool conectado: true
    property string modo: "SIM"
    property int etapa: 7
    property var pos: ([0.0, 0.0, 0.0])
    property var posNeum: ([1.0, 0.566])
    ...
    property ListModel eventos: ListModel {
        property var filtro: (["info", "aviso", "alarma"])
        property var conteos: ({"alarma": 0, "aviso": 0, "info": 14})
        function alternarNivel(a0) {}
        function limpiar() {}
        ListElement { hora: "23:55:48.482"; nivel: "info"; msg: "Ciclo 1 de 3"; origen: "grafcet"; t:
 1791611748.482 }
        ...
    function escribir(a0, a1) {}
    function pulso(a0) {}
```

11.2.3 3. La ruta de los assets

En el HMI, `rutaAssets` es una cadena con una URL `file:///…`. En Design Studio tiene que salir de la ubicación del proyecto. `dummydata/rutaAssets.qml` es un objeto cuyo método `toString(): string` devuelve `Qt.resolvedUrl("../../../assets/assets_cilindros/")`. La anotación de tipo `: string` es imprescindible: sin ella, la concatenación `rutaAssets + "logos/…"` produce el texto «QVariant()logos/...» y ninguna imagen carga.

11.2.4 4. La rejilla en un componente aparte

Design Studio reconstruye nodo por nodo el archivo que se edita y no evalúa las funciones ni las propiedades propias declaradas en él. Con las fórmulas de la rejilla dentro de `PantallaHmi.qml`, todos los paneles aparecían amontonados en la esquina superior izquierda. Las fórmulas se movieron a `RejillaHmi.qml` (listado 8.5), un componente aparte cuyas funciones sí evalúa. `PantallaHmi.qml` solo instancia la rejilla y coloca los paneles con `x: g.xCol(8)` o `width: g.ancho(4)`.

11.3 El proyecto `HMI.qmlproject`

Listing 11.2: Proyecto de Qt Design Studio (`ui/qml/HMI.qmlproject`, resumido)

```
Project {
    mainFile: "PantallaHmi.qml"
    mainUiFile: "PantallaHmi.qml"
    QmlFiles { directory: "." }
    JavaScriptFiles { directory: "." }
    ImageFiles { directory: "." }
    Files { filter: "qmldir"; directory: "." }
    Files { filter: "*.ttf;*.otf"; directory: "../../assets/assets_cilindros/fuentes" }
    importPaths: [ "." ]
    qt6Project: true
}
```

El archivo de preferencias del usuario que crea Design Studio (`HMI.qmlproject.qtds`) se ignora en Git. `HMI.qrc`, que apareció sin confirmar en el área de trabajo de otra sesión, lista los QML y las fuentes como recursos de Qt, pero ningún archivo de Python ni QML lo referencia todavía: el HMI carga el QML desde archivos.

11.4 Cómo conviven el diseño visual y el código

La guía del autor establece reglas de edición segura:

- **Propiedades por el inspector, posiciones por texto.** Colores, textos, tamaños de fuente y márgenes se cambian sin riesgo en el inspector. Arrastrar un panel en el lienzo sustituye su fórmula de la rejilla (`x: g.xCol(8)`) por un número fijo, y la pantalla deja de adaptarse entre 1366×768 y 1920×1080.
- **Colores solo desde `UI.Theme`.** Escribir un hexadecimal en el inspector rompe el cambio de tema oscuro/gris.
- **Compatibilidad con Qt 5.** Design Studio ofrece componentes y propiedades de Qt 6 (`MultiEffect`, `topLeftRadius`) que romperían la variante PyQt5. No deben usarse.
- **Las propiedades del interceptor se protegen.** En Design Studio se carga el `Theme.qml` original de `assets/`, sin `sufijoIcono` ni `textoSobreAcento`. Los componentes escriben `(UI.Theme.sufijoIcono || "_claro")` para funcionar en ambos entornos.

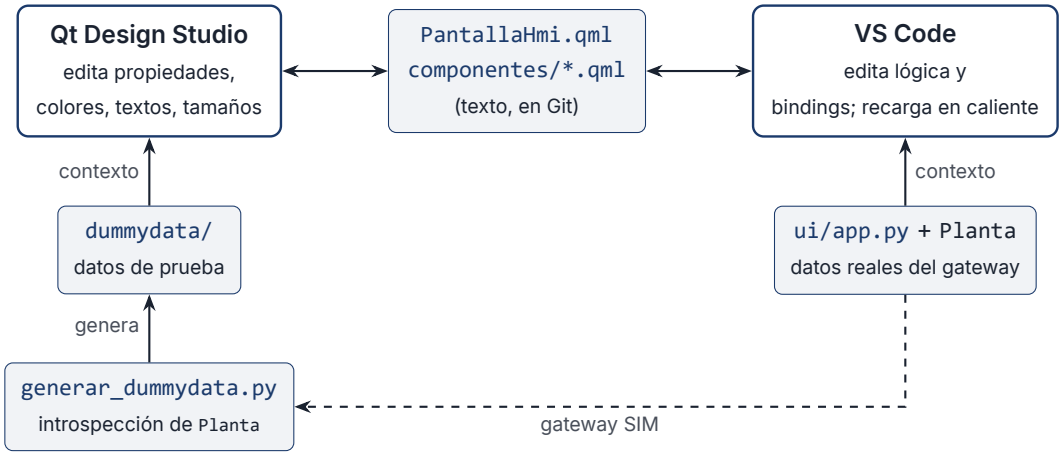


Figura 11.1: Flujo de trabajo: el mismo QML se edita en Design Studio (con datos de prueba) y en VS Code (con datos reales). No hay archivo intermedio ni código generado.

- **`dummydata/` no se edita a mano**: se regenera.

11.5 Comparación con el diseñador de formularios de Delphi

La guía de Design Studio incluye una tabla de equivalencias con Delphi RAD Studio (VCL), útil para quien viene de esa herramienta:

La diferencia de fondo es el modelo de programación. En Delphi, un temporizador lee el dato y lo asigna a un control: `lblEtapa.Caption := 'X' + IntToStr(etapa);` dentro de `Timer1Timer`. En QML se *declara* la relación una vez, `Text { text: "X" + planta.etapa }`, y el motor la mantiene. No hay código que olvide actualizar un control cuando cambia el dato.

11.6 Limitaciones conocidas

- El diseñador no ejecuta la lógica: lo que se ve son los datos congelados de `dummydata/`, no una simulación en vivo.
- Un doble clic sobre un botón no crea un manejador como en Delphi; las conexiones se escriben en el QML.
- La guía deja pendiente verificar que `PantallaHmi.qml` abra limpia en la vista de diseño. La captura de una primera prueba (`docs/img/qt.png`) mostró elementos superpuestos, y por eso se creó `RejillaHmi.qml`.

Delphi RAD Studio (VCL)	Qt Design Studio / QML en este proyecto
`Unit1.pas` + `Unit1.dfm`	Un solo `PantallaHmi.qml` (estructura y comportamiento declarativo)
Form Designer	Form Editor
Tool Palette	Components
Structure View	Navigator
Object Inspector (propiedades)	Properties
Object Inspector (eventos)	Connections / `onClicked: planta.pulso("STARTV")`
Align, Anchors	Anchors, Layouts y la rejilla de 12 columnas
Frames	Componentes `.qml` reutilizables
Componentes *data-aware*	Bindings a propiedades de `Planta`
Dataset activo en diseño	`dummydata/`
`TTimer`	`Timer`
Componente OPC dentro del `.exe`	Proceso aparte: gateway de 32 bits que publica por ZeroMQ
F9 (compilar y ejecutar)	Live Preview, o `python ui/app.py --dev`
Object Pascal en manejadores de eventos	QML declarativo; la lógica de la planta en Python

Tabla 11.1: Equivalencias entre Delphi VCL y Qt Design Studio (de `docs/GUIA_QT_DESIGN_STUDIO.docx`).

CAPÍTULO 12

Normas aplicables

Este capítulo reúne las normas que orientan la planta, el programa y la interfaz, y dice con franqueza hasta dónde las sigue el código. Ninguna se ha auditado formalmente: se usan como *criterio de diseño*, no como certificación. El repositorio cita explícitamente ISA-101, IEC 60073, IEC 60204-1, ISA-18.2, ISO 15552 e ISO 6432; la evaluación de colores está en la bitácora de la sesión 02. Las demás (IEC 60848, IEC 61131-3, ISO 1219, IEC 62443, WCAG) se añaden aquí porque describen lo que el código ya hace o lo que convendría hacer.

Norma	Qué regula	Dónde aparece	Grado
ISA-101.01	Diseño y ciclo de vida de HMI para automatización de procesos	Sistema de diseño, tema gris, banda de alarmas	Criterio rector
IEC 60073	Codificación por colores de indicadores y actuadores	Significado de los tokens de color	Parcial
IEC 60204-1	Equipo eléctrico de máquinas: colores de pulsadores, paros	Botones START/STOP/RESET	Parcial (desvío en RESET)
ISA-18.2 / IEC 62682	Gestión de alarmas	`core/alarmas.py`, historial	Parcial
IEC 60848	Lenguaje GRAFCET	Diseño de la secuencia; dibujo del GRAFCET	Aplicada
IEC 61131-3	Lenguajes de programación de PLC	Ladder del SLC 500 (dialecto propio)	Referencia
ISO 1219	Símbolos de circuitos neumáticos	Diagramas de FluidSIM (no en el HMI)	Fuera del HMI
ISO 15552, ISO 6432	Cotas de cilindros neumáticos	Geometría de los assets	Referencia
ISO 6358	Características de caudal de componentes neumáticos	Modelo dinámico (no implementado)	Referencia
IEC 62443	Ciberseguridad de automatización industrial	Enlace web, roles, firma de órdenes	Principios
WCAG 2.x	Contraste y accesibilidad visual	Evaluación de la paleta (este capítulo)	Evaluada

Tabla 12.1: Normas aplicables y grado de aplicación.

12.1 ISA-101: interfaces hombre-máquina

12.1.1 Qué dice

ISA-101.01 (2015) trata el ciclo de vida completo de un HMI de proceso: filosofía, guía de estilo, diseño, implementación, operación y mejora continua. Sus ideas centrales, y las de la práctica de *HMI de alto desempeño* que la acompaña, son:

- la pantalla debe mostrar *información*, no solo datos: tendencias, contexto y desviaciones respecto a lo normal;
- el color se reserva para transmitir *estado o condición anormal*; lo normal se ve en tonos neutros, y por eso son frecuentes los fondos grises;
- consistencia: el mismo color y el mismo símbolo significan siempre lo mismo;
- se evitan las decoraciones que no informan (relieves 3D, animaciones gratuitas, imágenes fotográficas);
- las alarmas deben destacar justamente porque el resto de la pantalla es sobrio.

12.1.2 Cómo se refleja

- **Regla de color escrita en los tokens**: «color solo para estado; en reposo todo es gris (ISA-101)» (`tokens.json`).
- **Tema gris** conmutable (sesión 14), con fondo `#BFC4CA` y superficies más claras (figura 8.2).
- **Banda de alarmas** visible *solo* con alarmas activas, roja con alarmas y ámbar si solo hay avisos.
- **Sin dato, gris**: un panel sin conexión baja su opacidad a 0,55; los chips del registro solo toman color si hay eventos de ese nivel.
- **Movimiento con significado**: los vástagos se mueven porque la planta se mueve; nada parpadea salvo las alarmas.
- **Información y no solo datos**: el cronograma de 45 s, la barra del temporizador en cada etapa, la cuenta regresiva de `T4:5` y la calibración junto a cada cilindro dan contexto temporal.

12.1.3 Desvíos señalados

La evaluación de la sesión 02 encontró tres desvíos, todos heredados del sistema de diseño del autor. La decisión de corregirlos sigue abierta:

1. **El ámbar se usa para estados normales** (etapa activa, bobina energizada, sensor activo) y también para advertencias. Una lectura estricta de ISA-101 reservaría los colores saturados para lo anormal y mostraría lo normal con variaciones de gris o de brillo.
2. **Los botones tienen acabado 3D brillante** (bisel, degradados, brillo), que ISA-101 considera decorativo.

3. El tercer desvío es de IEC 60204-1 (sección 12.3).

12.2 IEC 60073: codificación por colores

IEC 60073 establece los principios de codificación de indicadores y actuadores para la seguridad de la interfaz hombre-máquina. Para indicadores de condición asigna:

Color	Significado (IEC 60073)	Uso en el HMI	Coherencia
Rojo	Emergencia, condición peligrosa	STOP, alarma, desconexión, modo REAL	Coherente
Amarillo/ámbar	Anormal, condición crítica inminente	Advertencias *y* estados normales activos (etapa, bobina, sensor)	Parcial: doble uso
Verde	Normal	En marcha, calidad Good, escritura confirmada	Coherente
Azul	Obligatorio, acción requerida	Aire presurizado, acciones del sistema, modo SIM	Desvío: significado propio del sinóptico
Gris/blanco	Neutro, sin significado especial	Inactivo, sin dato, deshabilitado	Coherente

Tabla 12.2: Colores de IEC 60073 frente a los tokens del HMI.

12.3 IEC 60204-1: pulsadores y paros

IEC 60204-1 regula el equipo eléctrico de las máquinas, entre otras cosas los colores de los pulsadores físicos:

- **arranque**: preferentemente blanco; también gris, negro o verde; nunca rojo;
- **paro**: preferentemente negro; también gris o blanco; el rojo se admite si no está junto a un paro de emergencia; nunca verde;
- **rearme** (*reset*): azul, blanco, gris o negro;
- **paro de emergencia**: rojo sobre fondo amarillo.

En el HMI, START es verde y STOP rojo: ambos están permitidos aunque no sean los preferidos. RESET es ámbar, y eso es el **tercer desvío**. Tres matices importan:

- los botones del HMI son *virtuales*, y la norma se escribió para actuadores físicos;
- los pulsadores físicos de la planta son `START_1` y `STOP_1`, este último NC, como corresponde a una función de paro;

- el STOP del HMI es un **paro funcional**, no un paro de emergencia: congela la secuencia, pero las salidas siguen la lógica del Ladder.

> **El HMI no es un dispositivo de paro de emergencia** La parada de emergencia de la máquina debe ser un dispositivo cableado, independiente del PLC de secuencia, del HMI y de cualquier red, conforme a IEC 60204-1 e ISO 13850. Nada en este software sustituye esa función. Por eso el árbitro de mando deja pasar siempre el `STOPV` de un operador: es lo mínimo que un HMI puede hacer, no un paro de seguridad.

Corrección posible: cambiar el token del botón RESET a azul o gris en `Theme.qml` y en los SVG de `ui/botones/reset_*`. Es una decisión del autor, que se registró como pendiente en las sesiones 02, 05, 06 y 09.

12.4 ISA-18.2 / IEC 62682: gestión de alarmas

12.4.1 Qué dice

ISA-18.2 (y su equivalente internacional IEC 62682) define la gestión de alarmas como un ciclo de vida: filosofía, identificación, *racionalización* (justificar cada alarma, su prioridad, su causa y la respuesta esperada), diseño, implementación, operación, mantenimiento, monitoreo y auditoría. Define estados de alarma (normal, activa no reconocida, activa reconocida, retorno a la normalidad no reconocido, *shelved* o archivada temporalmente, suprimida por diseño, fuera de servicio) y métricas de desempeño, como la tasa de alarmas por operador y las inundaciones de alarmas.

12.4.2 Qué implementa el código

- **Prioridades distinguibles**: dos niveles, `alarma` (rojo) y `aviso` (ámbar), además de los eventos `info`.
- **Activación y normalización** con eventos fechados («Normalizada: ...»).
- **Historial** con inicio y fin de cada alarma en SQLite, consultable y exportable.
- **Supresión lógica**: con el gateway desconectado, las alarmas que dependen de datos no se evalúan, lo que evita una inundación de falsas alarmas.
- **Presentación**: banda dedicada, visible solo con alarmas activas.

12.4.3 Qué falta

- **Reconocimiento (ACK)**: no hay estado «no reconocida/reconocida» ni botón para reconocer.
- **Archivado temporal** (*shelving*) y fuera de servicio.
- **Racionalización documentada**: un documento de filosofía con la causa, la consecuencia, la respuesta y el tiempo disponible de cada una de las cinco alarmas.

- **Métricas**: tasa de alarmas, alarmas más frecuentes, alarmas permanentes.

Con cinco alarmas, un solo operador y una planta de laboratorio, lo implementado es razonable. Para una planta real, el reconocimiento y la racionalización serían lo primero que habría que añadir.

12.5 IEC 60848: GRAFCET

IEC 60848 define el GRAFCET como lenguaje de especificación de automatismos secuenciales: etapas (la inicial con doble cuadro), transiciones con su receptividad, acciones asociadas a etapas (continuas, condicionadas, memorizadas), reglas de evolución, divergencias y convergencias en Y y en O, y estructuración en macroetapas y GRAFCET parciales con forzado. El proyecto la aplica en tres lugares:

- en el diseño de la secuencia: GRAFCET de nivel 2, con un GRAFCET de marcha (GM) y uno de seguridad (GS) que fuerza la etapa inicial;
- en la traducción a Ladder por el método de etapas como memorias;
- en el dibujo del HMI, que conserva los símbolos: etapas numeradas, barras de transición con receptividad, divergencia en O tras X7 y forzado GS.

12.6 IEC 61131-3: lenguajes de programación

IEC 61131-3 normaliza los lenguajes de los autómatas programables: diagrama de contactos (LD), diagrama de bloques de función (FBD), texto estructurado (ST), diagrama funcional secuencial (SFC, derivado del GRAFCET) y, hasta su tercera edición, lista de instrucciones (IL). También normaliza los temporizadores (`TON`, `TOF`, `TP`) y contadores (`CTU`, `CTD`). El RSLogix 500 del SLC 500 usa un dialecto propio de Allen-Bradley, anterior a la norma: archivos de datos (`B3`, `T4`, `C5`) en lugar de variables declaradas, e instrucciones `XIC`/`XIO`/`OTE`/`OTL`/`OTU`/`OSR`. Conceptualmente equivale al LD de la norma, y el simulador de Python reproduce su semántica: TON que deja de acumular en el PRE, CTU por flanco y barrido secuencial de escalones.

12.7 ISO 1219: símbolos neumáticos

ISO 1219 define los símbolos gráficos de los circuitos de fluidos (válvulas por número de vías y posiciones, actuadores, accionamientos). Los diagramas neumáticos del proyecto, hechos en FluidSIM, la siguen. El HMI, en cambio, usa a propósito una *ilustración realista* de los cilindros en lugar de símbolos ISO 1219. Para un operador, un vástago que sale es más inmediato que un símbolo de válvula 5/2 cambiando de casilla. Una mejora posible sería añadir junto a cada cilindro el símbolo de su válvula con la casilla activa resaltada.

12.8 ISO 15552 e ISO 6432: cilindros

ISO 15552 normaliza las cotas básicas y de montaje de los cilindros neumáticos con fijaciones desmontables de la serie de 10 bar (1000 kPa), de 32 a 320 mm de diámetro. ISO 6432 hace lo mismo para minicilindros de vástago simple de 8 a 25 mm. Su propósito es la intercambiabilidad entre fabricantes. Los assets se nombran por ellas (`iso15552_d32`, `iso6432_d20`), y las referencias comerciales del README son Parker P1D-S ø32×200 y P1A ø20. Los parámetros del generador coinciden con las cotas habituales de catálogo:

- ø32: vástago de 12 mm, rosca M10 y saliente WH de 26 mm;
- ø20: vástago de 8 mm, rosca M8 y cuello M22×1,5.

PENDIENTE: la carrera de 50 mm del minicilindro es una suposición del README. Hay que medir la de los cilindros 4A/5A del tablero.

12.9 IEC 62443 y seguridad de las comunicaciones

IEC 62443 es la serie de normas de ciberseguridad para sistemas de automatización industrial. Propone dividir el sistema en zonas y conductos, aplicar defensa en profundidad y asignar el mínimo privilegio. Sin pretender cumplirla, el diseño del enlace web sigue esos principios:

- **Ningún puerto entrante** en la red del laboratorio: el puente abre la conexión hacia fuera.
- **Órdenes firmadas** con HMAC-SHA256 (RFC 2104 sobre SHA-256, FIPS 180-4), con ventana de tiempo y protección contra repetición.
- **Contraseñas** con PBKDF2 (RFC 8018) de 240 000 iteraciones; sesiones con cookies `HttpOnly`, `SameSite=Strict` y `Secure`.
- **Mínimo privilegio**: cuatro roles con permisos crecientes y el mando remoto deshabilitado por defecto.
- **Autoridad local**: el HMI local siempre manda; las salidas son del Ladder; las reglas de escritura se verifican en el gateway aunque la orden venga autorizada.
- **Trazabilidad**: auditoría de inicios de sesión, órdenes, cambios de cuentas y conexiones del puente.

Los protocolos usados se apoyan en sus normas respectivas: OPC DA (OPC Foundation), WebSocket (RFC 6455), MQTT 3.1.1 (estándar OASIS, también ISO/IEC 20922) y HTTP sobre TLS.

12.10 Contraste y accesibilidad (WCAG 2.x)

Las pautas WCAG del W3C definen el contraste entre dos colores a partir de su luminancia relativa L:

$$L = 0{,}2126\,R + 0{,}7152\,G + 0{,}0722\,B, \qquad C_{\text{lin}} = \begin{cases} C/12{,}92 & C \le 0{,}04045 \\ \left(\frac{C+0{,}055}{1{,}055}\right)^{2{,}4} & C > 0{,}04045 \end{cases} \tag{12.1}$$

$$\text{contraste} = \frac{L_{\text{claro}} + 0{,}05}{L_{\text{oscuro}} + 0{,}05}. \tag{12.2}$$

Piden al menos 4,5:1 para texto normal y 3:1 para texto grande y para elementos gráficos de la interfaz. La tabla 12.3 aplica la fórmula a la paleta sobre el color de los paneles (`surface`).

Token sobre `surface`	Oscuro	Gris	Lectura
`texto`	14,16	11,20	ambos muy por encima de 4,5
`textoSuave`	5,70	5,31	ambos cumplen 4,5
`activo` (ámbar)	9,33	2,76	gris: por debajo de 3
`ok` (verde)	8,14	3,05	gris: justo en 3
`alarma` (rojo)	5,23	3,68	gris: solo gráficos o texto grande
`aire` (azul)	5,16	4,03	gris: solo gráficos o texto grande
`neutro`	4,67	3,27	—
`unamOro`	7,07	2,79	franja decorativa

Tabla 12.3: Contraste WCAG de la paleta sobre `surface` (calculado para este documento).

El tema oscuro cumple con holgura en todos los colores. El tema gris tiene un texto principal excelente, pero sus acentos, oscurecidos para el fondo claro, quedan cerca o por debajo de 3:1. En particular, el ámbar de etapa activa y de sensor (2,76:1) se usa también como color de texto en las etiquetas `SC`/`SE`. **Recomendación**: oscurecer los acentos del tema gris (por ejemplo `activo` hacia `#8A5A00`, que da unos 4,1:1 sobre `surface`) o acompañarlos siempre de un cambio de peso o de fondo, de modo que el estado no dependa solo del matiz.

12.11 Resumen de desvíos y recomendaciones

Norma	Desvío	Corrección propuesta
IEC 60204-1	RESET en ámbar.	Token del RESET en azul o gris y regenerar `reset_*.svg`.
ISA-101 / IEC 60073	Ámbar para estados normales y para advertencias.	Estados normales activos en gris claro o blanco con más peso; ámbar solo para advertencias.
ISA-101	Botones 3D brillantes.	Variante plana de los botones (el sistema ya tiene `plano_*`).
ISA-18.2	Sin reconocimiento ni archivado temporal; sin racionalización escrita.	Estado ACK en `core/alarmas.py`, botón «Reconocer» y documento de filosofía de alarmas.
WCAG	Acentos del tema gris por debajo de 3:1.	Oscurecer `activo`, `ok` y `unamOro` en la variante gris.

Tabla 12.4: Desvíos normativos y correcciones propuestas (ninguna aplicada; requieren decisión del autor).

CAPÍTULO 13

Gemelo digital y enlace web

El gemelo digital es una réplica 3D de la estación que corre en cualquier navegador, desde una PC o un celular. Se mueve con los datos reales de la planta y, con los permisos adecuados, también manda órdenes. Este capítulo describe sus tres piezas, el puente, el servidor y el frontend, además del despliegue. Los protocolos que las unen se explicaron en el capítulo 4.

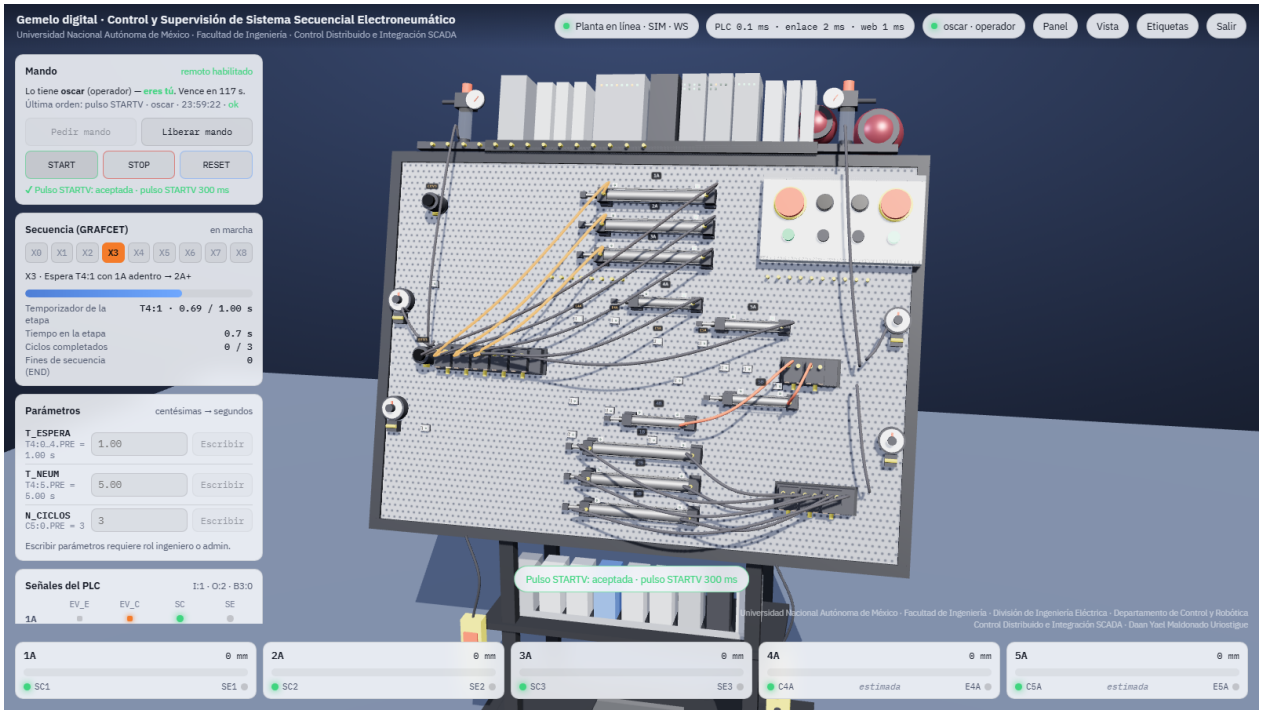


Figura 13.1: Gemelo digital en el navegador, con la planta en marcha en SIM. A la izquierda, paneles de mando (con el titular del mando y su vencimiento), GRAFCET, parámetros y señales; abajo, los cinco cilindros en milímetros; arriba, la píldora de latencias «PLC · enlace · web».

13.1 Arquitectura

13.2 El puente

`puente/enlace.py` se suscribe al gateway como un HMI más (SUB y REQ con `zmq.asyncio`) y mantiene la conexión con el servidor por WebSocket o MQTT. Hace cuatro cosas:

1. **Reenviar el estado.** Manda cada snapshot en cuanto llega, como máximo a `--hz` (20 por defecto) y como mínimo uno por segundo. Si llega antes de tiempo, espera lo justo y manda el

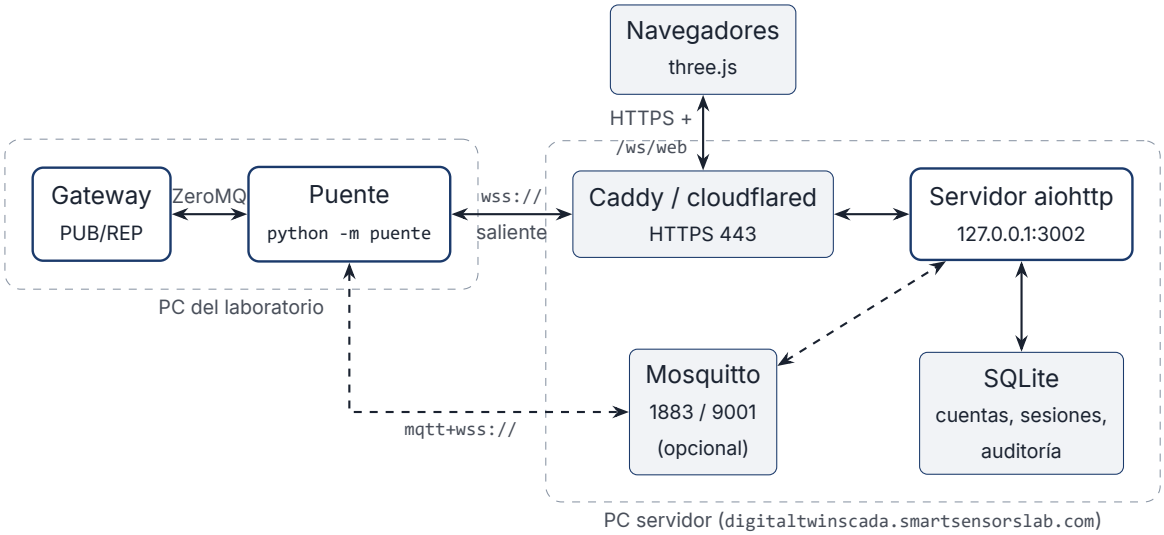


Figura 13.2: Despliegue del gemelo: el laboratorio solo abre conexiones salientes.

más nuevo, no el que lo provocó. En REAL completa `pos` con el estimador de carrera y, con `--suave auto|si|no`, añade el bloque `suave` para el seguidor del navegador.

2. **Verificar las órdenes.** Cada orden firmada pasa por `core.remoto.Verificador` (firma, hora, id no repetido, origen web). Solo entonces se añade `quien` y se envía al REP del gateway, con un plazo de 2 s. El resultado vuelve al servidor.
3. **Mantener el latido.** Hace `ping` cada 3 s y publica el tiempo de ida y vuelta como `lat_web_ms`. Reenvía al gateway la presencia de los usuarios web cada 5 s; si deja de hacerlo durante 15 s, el gateway revoca el mando web.
4. **Reconectar** con espera creciente (1, 2, 4, 8, 15 s).

Listing 13.1: Envío por evento limitado por `hz` (`puente/enlace.py:271-282`)

```
            ahora = time.monotonic()
            desde_envio = ahora - self._t_envio
            if s.get("seq") == self._seq_enviado and desde_envio < 1.0:
                continue
            if desde_envio < self.periodo_s:
                # Llegó antes de lo que permite `hz`: se espera lo justo y se manda el más nuevo.
                await asyncio.sleep(self.periodo_s - desde_envio)
                s = self.snapshot
                ahora = time.monotonic()
            if self.rtt_ms is not None:
                s["lat_web_ms"] = round(self.rtt_ms, 1)
            await canal.enviar({"tipo": "snap", "s": self.preparar_envio(s)})
```

`puente/vincular.py` resuelve cómo llega la clave compartida al laboratorio sin copiarla a mano. Con una cuenta admin del servidor, `python -m puente.vincular --usuario daan` hace `POST /api/puente/clave` por HTTPS y guarda la respuesta de forma atómica en `datos/puente.clave`. Usa solo la biblioteca estándar, no sigue redirecciones (para no enviar la contraseña a otro sitio) y nunca imprime la clave completa. La ventana de conexión del HMI ofrece lo mismo con un botón.

13.3 El servidor

`digital_twin/servidor_web/app.py` es una aplicación aiohttp que sirve el frontend, la API REST (sección 4.5) y dos WebSocket:

- **`/ws/planta`**: un solo puente a la vez, autenticado con la clave.
- **`/ws/web`**: un canal por navegador. Requiere sesión, salvo que el servidor arranque con `--publico`, en cuyo caso se entra como invitado de solo lectura. Al conectar, el navegador recibe una `bienvenida` con su usuario, el último snapshot, los últimos 50 eventos y los roles.

13.3.1 Una cola por navegador

Un navegador lento (un celular con mala señal) no debe frenar a los demás. Cada conexión tiene una `Salida` con dos compartimentos. Los snapshots se *sustituyen*: solo importa el último, y si llega uno nuevo antes de enviar el anterior, el viejo se descarta. Los demás mensajes (eventos, actividad, resultados) se encolan en orden y no se pierden:

Listing 13.2: Cola de salida por navegador (`servidor_web/app.py:94-112`)

```
    def poner(self, tipo, texto):
        if tipo == "snap":
            if self.snap is not None:
                self.descartados += 1
            self.snap = texto
        else:
            self.cola.append(texto)
        self.aviso.set()

    async def bombear(self):
        try:
            while not self.ws.closed:
                await self.aviso.wait()
                self.aviso.clear()
                while self.cola:
                    await self.ws.send_str(self.cola.popleft())
                if self.snap is not None:
                    texto, self.snap = self.snap, None
                    await self.ws.send_str(texto)
```

13.3.2 Órdenes desde la web

Cuando un navegador manda una orden, el servidor la procesa así:

1. filtra los campos con una lista blanca;
2. relee el rol del usuario en la base, por si un admin lo cambió;
3. comprueba el permiso por rol;
4. limita a 4 órdenes por segundo;

5. firma la orden y la manda al puente;
6. espera el resultado hasta 3 s;
7. audita, difunde la actividad a todos y responde al navegador que la envió.

Los permisos se filtran dos veces: el servidor rechaza pronto lo que el rol no permite, y la última palabra la tiene el gateway (árbitro de mando y reglas de escritura).

13.4 El frontend 3D

`digital_twin/index.html` es una sola página con three.js r128, cargado primero desde el propio servidor y, si falla, desde un CDN. Si tampoco carga, el tablero de mando funciona sin la escena.

13.4.1 La escena

- **Render**: `WebGLRenderer` con antialiasing, sombras suaves (`PCFSoftShadowMap`), espacio de color sRGB y *tone mapping* ACES filmic. La exposición baja de 1,05 a 0,55 cuando la planta está fuera de línea: el gemelo «se apaga» visualmente.
- **Luces**: hemisférica de ambiente, un «sol» direccional con sombras de 2048^2 (1024^2 en móvil) y una luz de relleno azulada.
- **Tablero**: placa perforada de 140×100 cm inclinada, con la textura de perforaciones generada en un `CanvasTexture`; rack del PLC con LEDs de E/S ligados a los bits de `I:1` y `O:2`; reguladores de presión; panel de mando 3D cuyos botones se hunden según `START_1`, `STOP_1` y los bits de pulso, y que además mandan órdenes con un clic.
- **Neumática**: tres bloques de electroválvulas con bobinas que se iluminan, y tubos (`TubeGeometry` sobre curvas Catmull-Rom) que se tiñen de naranja según la memoria de la válvula 5/2.
- **Cilindros**: cuerpo, tapas, cuatro tirantes, vástago, punta, racores y sensores, construidos con primitivas. El vástago se mueve con $x = -\frac{L}{2} - \frac{L_v}{2} + s\,(1-p)$. 1A, 2A y 3A siguen `pos`; 4A y 5A siguen `pos_neum` y encienden los finales de carrera C4A, E4A, C5A y E5A según la posición estimada.

13.4.2 Cámara y controles

La cámara orbital es propia, sin OrbitControls, en coordenadas esféricas alrededor de un punto de mira:

$$\vec{c} = \vec{m} + r\,(\sin\varphi\,\sin\theta,\ \cos\varphi,\ \sin\varphi\,\cos\theta), \tag{13.1}$$

con $\theta \in [-1{,}45;\ 1{,}45]$, $\varphi \in [0{,}35;\ 1{,}62]$ y $r \in [40;\ 500]$. El zoom es multiplicativo, $r \leftarrow r\,e^{0{,}0012\,\Delta y}$, también con el gesto de pellizco. Un `Raycaster` permite seleccionar cilindros y pulsar los botones del panel 3D.

13.4.3 Móvil

Con `matchMedia('(max-width:900px), (pointer:coarse)')` el frontend pasa a modo móvil:

- en vertical conserva el ángulo horizontal de visión ajustando el vertical, fov $=$ mín$(72°,\ 2\arctan(\tan(0{,}55)/\text{aspecto}))$;
- baja la resolución de sombras y limita la densidad de píxeles a 1,5;
- reorganiza el tablero en pestañas (Mando, GRAFCET, Parámetros, Señales, En línea, Actividad, Eventos).

Un `manifest.webmanifest` permite instalarlo como aplicación de pantalla completa.
El dibujo usa `requestAnimationFrame` con un Δt limitado a 50 ms, y el texto de los paneles se refresca cada 80 ms. La posición de los vástagos sale del `Seguidor` o del `Interp` descritos en la sección 7.6.



Figura 13.3: El gemelo en un celular.

13.5 Gemelo local

`digital_twin/local.py` levanta servidor y puente en el *mismo proceso* asyncio, con la misma clave y sin internet:

Listing 13.3: Gemelo local

```
python -m digital_twin.local --gateway SIM          # + gateway SIM propio, abre el navegador
python -m digital_twin.local --host 0.0.0.0         # visible en la red local
```

Crea cuentas de prueba (`admin`, `ingeniero`, `operador` y `usuario`, con contraseña `<nombre>1234`) en `datos/gemelo_local/`. Sirve para demostraciones en el laboratorio y para las pruebas automáticas.

13.6 Despliegue en la PC servidor

La carpeta `digital_twin/despliegue/` trae lo necesario para una PC Windows siempre encendida:

- **`instalar_servidor.bat`**: crea `.venv-web`, instala aiohttp y paho, muestra la clave del puente, configura el broker y crea la cuenta admin. **`servidor.bat`** arranca aiohttp en `127.0.0.1:3002`.
- **Caddy** (si hay IP pública): proxy inverso con HTTPS automático de Let's Encrypt, cabeceras `X-Forwarded-*`, compresión, HSTS, y el subdominio del broker hacia el WebSocket de Mosquitto.
- **cloudflared** (sin IP pública): un túnel saliente de Cloudflare con reglas de *ingress* `digitaltwinscada… → 127.0.0.1:3002` y `broker… → 127.0.0.1:9001`. Por el túnel solo pasa HTTP/WebSocket; por eso el puente usa `mqtt+wss://`.
- **Mosquitto** (opcional): sin anónimos, con archivo de contraseñas y ACL (el usuario `puente` solo publica `snap`, `evt`, `estado`, `resultado` y `ping`, y solo lee `orden`, `presencia` y `pong`), escuchando únicamente en `127.0.0.1`.
- **`clave_puente.bat`**: diagnóstico de la clave del puente; busca qué proceso escucha en 3002 y prueba la clave.

CAPÍTULO 14

Pruebas y latencia

14.1 Pruebas automáticas

14.1.1 Estrategia

La regla del proyecto es que todo lo que no necesite COM se prueba con pytest en la máquina de desarrollo. Eso es posible por tres decisiones de diseño:

- el backend SIM implementa la misma interfaz que el OPC;
- `core/` no depende de Qt ni de COM;
- OpenOPC se importa dentro de `conectar()`, de modo que se puede sustituir por un módulo falso inyectado en `sys.modules`.

Las pruebas de `ui/` crean un `QCoreApplication` con la plataforma `offscreen` y avanzan el bucle de eventos con `processEvents()` dentro de un lazo con plazo, nunca con `exec()`. Las de red usan puertos efímeros, para no chocar con un gateway real en marcha.

14.1.2 Resultado

El 10 de octubre de 2026, con Python 3.13.7 de 64 bits y PySide6 6.11.2, la orden

```
.venv\Scripts\python.exe -m pytest tests -q
```

dio **229 pruebas en verde en 210,6 s**, sin fallos ni pruebas saltadas.

14.1.3 Prueba de humo

Además de pytest, hay dos pruebas de humo:

- `python -m gateway --modo SIM --ciclos-demo 3`: completa tres ciclos y sale con código 0 solo si nunca hubo dos bobinas de una válvula activas a la vez;
- `ui/herramientas/captura.py`: levanta el HMI sin ventana contra un gateway, captura la pantalla y reporta cuántos avisos y errores de QML hubo. Las capturas de este documento salieron con «0 avisos y 0 errores de Qt/QML».

Archivo	Pruebas	Qué verifica
`test_tags.py`	10	Direcciones, palabras, reglas de escritura (REAL bloquea `I:`, nunca `O:`), `T_ESPERA` → cinco direcciones.
`test_ladder_sim.py`	16	Secuencia X0…X8 y END en N ciclos; STOP congela; RESET vuelve a X0; nunca dos bobinas de una válvula a la vez; válvula biestable; cadena 4A/5A.
`test_servidor_sim.py`	9	Gateway SIM por ZeroMQ real: snapshots, pulsos, parámetros, rechazo de entradas en REAL.
`test_backend_opc.py`	8	Backend OPC con un OpenOPC falso: validación de direcciones, contaminación del paquete DF1.
`test_diagnostico_json.py`	12	Contrato JSON del diagnóstico OPC.
`test_alarmas.py`	10	Activación y limpieza de cada regla; latencia; historial.
`test_historial_lector.py`	15	Lector del historial: filtros, orden, límite, CSV, archivo roto.
`test_calibracion.py`	9	Mediana móvil, persistencia atómica, JSON corrupto, descartes.
`test_estimador.py`	12	Estimador con bits sintéticos: sensor antes y después, inversiones, tiempo límite.
`test_suavizado.py`	15	Modo suave contra la planta SIM con enlace lento: sin saltos, todas las carreras visibles, DN previsto, RESET, ciclo nuevo, discrepancia.
`test_mando.py`	12	Árbitro de mando y firma de órdenes.
`test_enlace_web.py`	1	Gateway SIM + puente + servidor + cuatro navegadores con roles distintos.
`test_enlace_mqtt.py`	3	Transporte MQTT: topics y sobre sellado.
`test_broker_mqtt_minimo.py`	9	Broker MQTT 3.1.1 mínimo de pruebas con clientes paho reales.
`test_gemelo_local.py`	1	Gemelo local: `pong`, `lat_web_ms`, frontend servido.
`test_frontend_web.py`	2	Frontend 3D en Chromium sin ventana (Playwright).
`test_vincular.py`	27	Vinculación de la clave del puente con una cuenta admin.
`test_clave_servidor.py`	8	Diagnóstico de la clave en la PC servidor y en el laboratorio.
`test_ui_planta.py`	13	`Planta` contra un gateway SIM real.
`test_ui_conexion.py`	21	Ventana de conexión: funciones puras y ciclo conectar/desconectar.
`test_ui_historial.py`	9	Visor de historial sin ventana.
`test_ui_tema.py`	2	Temas oscuro y gris y el interceptor de `Theme.qml`.
`test_dummydata.py`	5	Los datos de Qt Design Studio siguen al día con las clases de Python.
Total	**229**	

Tabla 14.1: Pruebas automáticas (recuento de pytest, con parametrizaciones).

14.2 Latencia medida en el laboratorio

El 3 de octubre de 2026 se conectó el HMI al SLC 5/04 real a través del 1761-NET-ENI. La tabla 14.2 resume lo medido (`mds/LATENCIA.md`, `docs/DIAGNOSTICO_CONEXION_PLC.md`).

Medición	Resultado	Consecuencia en el código
Lectura de grupo (15 items) desde el dispositivo	200–2238 ms	El gateway quedaba bloqueado: 19 snapshots en 16 s.
Lectura de grupo desde la caché de RSLinx	**6–8 ms**	Fuente `cache` por defecto: unos 9 snapshots/s (124 en 14 s).
Escritura OPC de un bit	73–98 ms sin grupos; 165–1177 ms con un grupo de caché activo	`TIMEOUT_CMD_MS` de 500 a 2000 ms.
Refresco real de la caché con *update* 100 / 50 / 20 ms	mediana 329–335 / 294–403 / 1212–1367 ms	Pedir más rápido *empeora*: se deja `--periodo 100`.
Lectura individual desde el dispositivo	404–2441 ms en escalones de ∼1 s	Reintentos DF1 por tiempo de espera del ACK: problema del enlace serie.

Tabla 14.2: Mediciones del 3 de octubre de 2026 contra el SLC real.

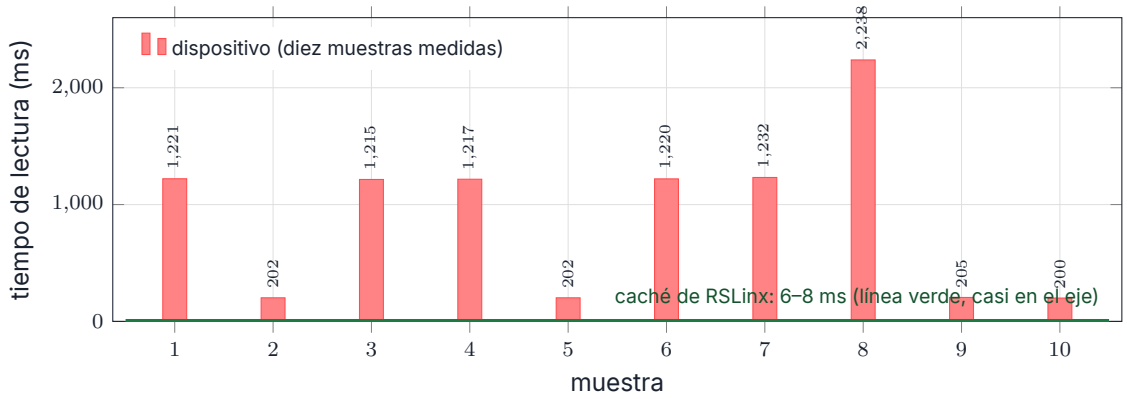


Figura 14.1: Lectura de grupo de 15 items desde el dispositivo (las diez muestras medidas el 3 de octubre) frente a la caché de RSLinx, de la que se registró el rango de 6–8 ms.

14.2.1 Conclusión

Con el código corregido, el gateway lee en 6 ms. La latencia que queda, unos 0,3 s típicos con picos de 1–2 s, está en el enlace serie ENI ↔ canal 0 del SLC, que repite paquetes. La base de unos 400 ms por transacción es varias veces lo esperable a 19 200 bps (30–50 ms). Si se eliminan los reintentos, el refresco debería bajar a 100–150 ms sin tocar el código.

El diagnóstico propone revisar, en este orden:

1. el *Channel Status* del canal 0 en línea (ACK timeouts, NAKs, retries);

2. la configuración del canal 0: DF1 full duplex, 19 200, CRC, respuestas embebidas en automático, detección de paquetes duplicados;
3. que la velocidad serie del ENI coincida con la del canal 0;
4. el cable 1761-CBL-AM00/HM02;
5. que ninguna otra PC tenga RSLinx contra el mismo ENI;
6. bajar las *Polled Messages* del topic de 1000 a 100 ms.

14.2.2 Lo que el software hace mientras tanto

- **Nunca congela la pantalla**: el gateway y la GUI están separados, y las órdenes tienen su propio hilo.
- **Mide y muestra la latencia**: lectura, p95, máximo, periodo y fuente en el panel «Latencia», con alarma si el p95 supera dos periodos durante diez ciclos.
- **Dibuja de forma honesta lo que llega tarde**: el estimador anima con la duración calibrada, y el modo suave predice sin adelantar transiciones (sección 7.6).
- **Permite reproducir el problema sin el PLC**: `--simular-enlace` imita la distribución de refrescos medida.

14.2.3 Mejoras de código y de Ladder propuestas

- `MOV I:1.0 B3:4` y `MOV O:2.0 B3:5` en el Ladder, para leer todo en un solo bloque `B3:0`…`B3:5`.
- Temporizadores en el grupo lento, con el avance de `T4:n.ACC` interpolado (el modo suave ya lo hace).
- Mandos de una sola escritura, con el Ladder haciendo OTU de `STARTV`, `STOPV` y `RESETV`.
- Cierre ordenado del gateway que escriba el 0 de los pulsos pendientes. Hoy la ventana de conexión lo termina con `kill` a los 3 s, y sus grupos quedan en RSLinx unos 6 minutos.
- Con autorización del profesor, mensajes no solicitados (instrucción MSG del SLC hacia RSLinx) al cambiar de etapa, que eliminarían el sondeo para los eventos discretos.

CAPÍTULO 15

Python frente a Delphi y las herramientas RAD

El enunciado del proyecto pide la comunicación OPC desde Delphi, y el uso de Python está pendiente del visto bueno del profesor. Este capítulo compara las dos opciones para construir un HMI *innovador*, con la experiencia concreta de este proyecto como evidencia. La comparación intenta ser justa: Delphi tiene ventajas reales, y se dicen.

15.1 Qué es cada opción

Delphi y el desarrollo RAD. Delphi (Embarcadero RAD Studio) es un entorno de *desarrollo rápido de aplicaciones* (RAD) con el lenguaje Object Pascal, compilado a código nativo. Su fortaleza histórica es el diseñador de formularios. Se arrastran componentes (botones, etiquetas, temporizadores, componentes de datos) a un formulario, se ajustan sus propiedades en el inspector y se escribe código en los manejadores de eventos. Ofrece dos bibliotecas visuales: VCL, solo Windows, sobre controles nativos, y FireMonkey (FMX), multiplataforma y dibujada con GPU. Otras herramientas de la misma familia son Visual Basic, C# con WinForms o WPF, y LabVIEW en su variante gráfica.

Python con Qt/QML. Python es un lenguaje interpretado de propósito general con un ecosistema de bibliotecas libres muy amplio. Aquí se combina con Qt mediante PySide6: la lógica está en Python y la interfaz en QML, un lenguaje declarativo con animaciones y gráficos acelerados. La interfaz se edita como texto (VS Code con recarga en caliente) o visualmente (Qt Design Studio).

15.2 Comparación por criterios

Tabla 15.1: Python + Qt/QML frente a Delphi para HMI.

Criterio	Python + PySide6/QML (este proyecto)	Delphi (VCL/FMX)
Acceso a OPC DA	OpenOPC + pywin32. Exige un proceso de 32 bits aparte, porque PySide6 es de 64 bits.	Compilación nativa de 32 bits y COM directo en el mismo ejecutable; existen componentes OPC para Delphi. **Ventaja Delphi.**

Criterio	Python + PySide6/QML	Delphi (VCL/FMX)
Comunicación moderna	ZeroMQ, msgpack, WebSocket, MQTT, HTTP, HMAC y PBKDF2 con bibliotecas libres de una línea de `pip`.	Disponible con componentes (Indy, bibliotecas propias o de terceros), con más trabajo de integración.
Diseño de la interfaz	QML declarativo: bindings, animación con una línea (`Behavior`), SVG nítido, temas por tokens. Editor visual con datos de prueba.	Diseñador de formularios muy maduro, de arrastrar y soltar, con los datos en vivo en diseño. FMX añade estilos y animaciones con GPU. VCL depende de controles nativos.
Modelo de programación	Declarativo: la pantalla *es función* del estado (`text: "X" + planta.etapa`).	Imperativo: un temporizador lee y asigna (`lblEtapa.Caption := …`); es fácil olvidar actualizar un control.
Arquitectura	Procesos separados (gateway, HMI, puente, servidor) que se pueden probar y reemplazar por separado.	Tiende a un solo ejecutable con todo dentro. Es más simple de distribuir, pero mezcla comunicación y pantalla.
Pruebas	229 pruebas pytest, planta simulada, Qt sin ventana, navegador sin ventana.	DUnitX existe; probar formularios es más difícil, y la práctica es menos habitual.
Versionado	Todo es texto (`.py`, `.qml`, `.json`); los cambios se leen en un *diff*.	`.pas` y `.dfm` en texto; el `.dfm` lo escribe el diseñador y sus *diffs* son menos legibles.
Web y gemelo digital	El mismo `core/` corre en el gateway, el HMI, el puente y el servidor web.	Exige otra tecnología para la web o productos específicos.
Plataformas	Windows, Linux y macOS; el modo SIM y las pruebas no necesitan Windows.	VCL: solo Windows. FMX: multiplataforma.
Rendimiento	Interpretado; lo pesado (dibujo, animación) lo hace Qt en C++ y la GPU; la GUI de Python solo aplica un snapshot cada 15 ms.	Código nativo compilado: arranque rápido, poca memoria. **Ventaja Delphi.**
Distribución	Requiere intérpretes (dos en el laboratorio: 32 y 64 bits) y un instalador (`lab/instalar.bat`).	Un solo `.exe`. **Ventaja Delphi.**

Criterio	Python + PySide6/QML	Delphi (VCL/FMX)
Errores de tipos	Tipado dinámico: algunos errores aparecen al ejecutar; se compensa con pruebas.	Tipado estático: el compilador atrapa muchos errores. **Ventaja Delphi.**
Costo y licencia	Python (PSF) y PySide6 (LGPLv3) libres y gratuitos.	Community Edition gratuita con restricciones de licencia; las ediciones profesionales son de pago.
Ecosistema y comunidad	Uno de los lenguajes más usados en ingeniería, ciencia de datos y automatización; abundante documentación y asistentes de código.	Comunidad más pequeña y especializada, fuerte en aplicaciones de escritorio heredadas.

15.3 Lo que este proyecto obtuvo de Python

Varias de las funciones que distinguen a este HMI de uno tradicional vinieron casi «gratis» del ecosistema de Python y de Qt:

- **Un gemelo digital web con cuentas y auditoría** en un solo proceso de aiohttp, reutilizando las mismas reglas de mando y firma (`core/mando.py`, `core/remoto.py`) que el gateway.
- **Transporte MQTT** con paho-mqtt y **seguridad criptográfica** con la biblioteca estándar (`hmac`, `hashlib`, `secrets`).
- **Historial con consulta y exportación** con `sqlite3`, sin servidor de base de datos.
- **Animación física y predictiva**: el estimador y el modo suave son algoritmos de unas centenas de líneas de Python puro, probados contra una planta simulada. QML dibuja el resultado con una línea por propiedad animada.
- **Simulación completa de la planta** (Ladder + física + enlace lento) para desarrollar y probar sin el PLC ni Windows.
- **Herramientas propias**: capturas sin ventana, generación de datos para el diseñador por introspección, figuras de este documento rasterizadas con QtSvg.

15.4 Lo que Delphi habría hecho mejor

- **Un solo ejecutable de 32 bits** con el cliente OPC dentro, sin gateway ni ZeroMQ. La separación en procesos tiene ventajas propias (sección 3.4), pero en Python fue *obligatoria* por la falta de ruedas de 32 bits de PySide6.

- **Distribución trivial**: copiar un `.exe`, frente a crear dos entornos virtuales e instalar dependencias.
- **Diseño visual inmediato**: el diseñador de Delphi muestra los datos en vivo y crea los manejadores con un doble clic. Qt Design Studio no ejecuta Python y trabaja con datos congelados.
- **Tipado estático y arranque rápido.**

15.5 Conclusión

Para un HMI *clásico* (pantalla de escritorio en Windows, un PLC, un ejecutable), Delphi sigue siendo una opción productiva, y su diseñador de formularios es difícil de igualar. Para un HMI *innovador* en el sentido de este proyecto, Python con Qt/QML resultó más adecuado:

- gráficos vectoriales animados con la física medida;
- predicción frente a un enlace lento;
- acceso web seguro desde cualquier dispositivo;
- historial, alarmas y arbitraje de mando;
- pruebas automáticas de todo sin hardware.

La razón no es que Delphi no pueda hacerlo. Es que cada una de esas piezas existe en Python como biblioteca libre, y QML permite expresar la interfaz como función del estado en lugar de como secuencia de actualizaciones.

Hay además un punto que hace la decisión menos arriesgada: **la arquitectura es independiente del lenguaje**. El servidor OPC, el topic, los ItemID y la tabla de variables son los mismos. Si el profesor exige Delphi, un cliente Delphi puede sustituir al HMI y conectarse al mismo gateway por ZeroMQ, o hablar OPC directamente con la misma tabla de `core/tags.py`.

CAPÍTULO 16

Conclusiones y trabajo futuro

16.1 Lo que se logró

- **Una arquitectura que separa la comunicación de la presentación**: un gateway de 32 bits es el único que habla OPC DA y publica el estado por ZeroMQ, y cualquier número de clientes lo consume. La pantalla nunca espera al enlace serie.
- **Una sola fuente de verdad** para las direcciones del PLC (`core/tags.py`) y para las reglas de seguridad heredadas de v6.1, que se verifican en el gateway en toda escritura, venga del HMI o de la web.
- **Lectura de la caché de RSLinx**, que bajó el tiempo de lectura de 200–2238 ms a 6–8 ms y dejó al descubierto el verdadero cuello de botella: los reintentos DF1 del enlace ENI ↔ SLC.
- **Una interfaz QML vectorial y animada** sobre el sistema de diseño del autor, con cilindros a escala real por capas, carrera calibrada con la mediana de las carreras medidas, modo suave predictivo, dos temas (oscuro y gris ISA-101) y un mismo QML para Qt 5 y Qt 6.
- **Edición visual en Qt Design Studio** sin duplicar la pantalla, gracias a datos de prueba generados por introspección.
- **Un gemelo digital web** con cuentas, roles, auditoría, árbitro de mando y órdenes firmadas, accesible desde un celular sin abrir puertos en el laboratorio.
- **229 pruebas automáticas** que corren sin PLC ni Windows.

16.2 Trabajo futuro

1. **Enlace con el PLC.** Revisar el canal 0 y el ENI según el diagnóstico (sección 14.2). Es la mejora de mayor efecto y no requiere tocar código.
2. **Programa del PLC.** Confirmar las correcciones de los escalones 0001 y 0002, cargar los preajustes y, con autorización, añadir `MOV I:1.0 B3:4` y `MOV O:2.0 B3:5` para leer todo en un bloque.
3. **Sensor de 4A.** Llevar una señal «4A retraído» a una entrada libre (por ejemplo `I:1/8`) para sustituir la ventana de tiempo de `T4:5` por un evento. Medir la cadena real y ajustar `T_NEUM` con un margen del 20–30 %.
4. **Alarmas según ISA-18.2.** Añadir reconocimiento, archivado temporal y un documento de racionalización.

5. **Ajustes normativos de color** (tabla 12.4): RESET en azul o gris, ámbar solo para advertencias, botones planos y acentos más oscuros en el tema gris.
6. **Cierre ordenado del gateway**, que escriba el 0 de los pulsos pendientes y elimine sus grupos de RSLinx.
7. **Empaquetado** del HMI con un instalador o un ejecutable autocontenido para el laboratorio.
8. **Verificar en Qt Design Studio** que `PantallaHmi.qml` abre limpia tras la separación de `RejillaHmi.qml`, y completar las capturas pendientes de la guía.

APÉNDICE A

Tabla de variables completa

Ladder del 18 de septiembre de 2026 (`docs/TABLA_DE_VARIABLES.md`, sección 3; `core/tags.py`). El ItemID completo en RSLinx es `[PROYECTO1_TEST]` seguido de la columna «ItemID».

Tabla A.1: Tabla de variables del PLC.

Símbolo	Dirección	ItemID	Descripción
Entradas, 1746-IB16 (ranura 1)			
`STOP_1`	`I:1/0`	`I:1.0/0`	Pulsador NC de paro (= 1 en reposo)
`START_1`	`I:1/1`	`I:1.0/1`	Pulsador NA de inicio
`SC1` / `SE1`	`I:1/2`, `I:1/3`	`I:1.0/2`, `/3`	1A retraído / extendido
`SC2` / `SE2`	`I:1/4`, `I:1/5`	`I:1.0/4`, `/5`	2A retraído / extendido
`SC3` / `SE3`	`I:1/6`, `I:1/7`	`I:1.0/6`, `/7`	3A retraído / extendido
Salidas, 1746-OB16 (ranura 2)			
`EV_E1` / `EV_C1`	`O:2/0`, `O:2/1`	`O:2.0/0`, `/1`	Bobina 1A+ / 1A−
`EV_E2` / `EV_C2`	`O:2/2`, `O:2/3`	`O:2.0/2`, `/3`	Bobina 2A+ / 2A−
`EV_E3` / `EV_C3`	`O:2/4`, `O:2/5`	`O:2.0/4`, `/5`	Bobina 3A+ / 3A−
Internas y de HMI			
First Pass	`S:1/15`	—	Primer escaneo tras RUN
`STARTV`	`B3:0/0`	`B3:0/0`	Arranque desde el HMI (pulso)
`STOPV`	`B3:0/1`	`B3:0/1`	Paro desde el HMI (pulso; XIO en el Ladder)
`ACTIVE`	`B3:0/2`	`B3:0/2`	Bit de marcha enclavado (lectura)
`END`	`B3:0/3`	`B3:0/3`	Fin de los N ciclos (lectura; dura un barrido)
`RESETV`	`B3:0/4`	`B3:0/4`	Reset desde el HMI (pulso)
`OSR_X7`	`B3:2/0`	—	Memoria del OSR del contador
`X0`…`X8`	`B3:3/0`…`/8`	`B3:3/n`	Etapas del GRAFCET (una sola activa)
`T_ESPERA`	`T4:0.PRE`…`T4:4.PRE`	ídem	Espera por etapa, centésimas (100 = 1 s); se escriben los cinco
`T_NEUM`	`T4:5.PRE`	ídem	Ventana neumática 4A/5A (500 = 5 s)
`N_CICLOS`	`C5:0.PRE`	ídem	Número de ciclos (3)
C	`C5:0.ACC`	ídem	Ciclos completados
(monitor)	`T4:n.ACC`, `T4:n/DN`	ídem	Avance y bit de hecho de cada espera

APÉNDICE B

Campos del snapshot

Tabla B.1: Campos del snapshot (protocolo versión 2).

Campo	Tipo	Contenido
`v`	int	Versión del protocolo (2).
`seq`	int	Contador de publicaciones del gateway.
`t`	float	Hora de pared al terminar la lectura (epoch, s).
`modo`	str	`SIM`, `EMULATE` o `REAL`.
`conectado`	bool	Hay lectura válida.
`fuente`	str	`sim`, `dispositivo` o `cache`.
`calidad`	mapa	`{good, total}` items con calidad Good.
`bits`	mapa	Tag → bool (entradas, salidas, `B3:0`, etapas, DN).
`ints`	mapa	Tag → int (ACC y PRE de los temporizadores, `C_ACC`, `C_PRE`).
`palabras`	mapa	`I:1.0`, `O:2.0`, `B3:0`, `B3:3` como enteros.
`etapa`	int / null	Etapa activa 0…8; null si no hay exactamente una.
`t_etapa`	float	Segundos en la etapa (reloj del gateway).
`pos`	lista / null	Posición de 1A, 2A, 3A en [0, 1]; null en REAL.
`pos_neum`	lista	Posición estimada de 4A y 5A.
`lat`	mapa	`lectura_ms`, `lectura_max_ms`, `ciclo_ms`, `periodo_ms`, `p95_ms`, `fuente`.
`alarmas`	lista	Alarmas activas `{id, nivel, msg, t_ini, activa}`.
`escrituras`	mapa	`{pendientes, ultima: {tag, valor, ok, msg, quien}}`.
`fines`	int	Veces que terminaron los N ciclos.
`mando`	mapa	`remoto_habilitado`, `titular`, `desde`, `vence_en_s`, `enlace_web`, `conectados`, `ultima_orden`.
`lat_web_ms`	float	(Añadido por el puente) ida y vuelta al servidor web.
`pos_estimada`, `suave`	bool, mapa	(Añadidos por el puente) posición estimada y bloque del modo suave.

APÉNDICE C

Discrepancias detectadas entre documentación y código

Al preparar este documento se leyó el código completo y se encontraron algunas diferencias con la documentación de contexto y detalles menores del código. No se corrigieron, porque la tarea era documentar. Se listan para que el autor decida.

Tabla C.1: Discrepancias y observaciones (10 de octubre de 2026).

Dónde	Observación
`mds/ARQUITECTURA.md` §4.6 y §6.4; `mds/ENLACE_WEB.md` §5.3	Los ejemplos usan `--topic PROYECTO1_REAL`; en el laboratorio ese topic no existe y el código usa `PROYECTO1_TEST` en todos los modos.
Notas del HMI v6.1 y reglas de trabajo	Mencionan `POLL_MS = 150`; esa constante no existe en el código nuevo (periodo de 50 ms en SIM y 100 ms en OPC, configurable).
`core/protocolo.py` (docstring)	`escrituras.ultima` lleva además el campo `quien`, que el docstring no menciona.
`core/remoto.py` (docstring)	Lista `ping` como mensaje del servidor al puente; en el código lo envía el puente y el servidor responde `pong`.
`mds/ENLACE_WEB.md` §3.2	Dice «∼10 Hz» para los snapshots web; el puente envía hasta 20 Hz por defecto.
Vigilante MQTT del servidor	El md habla de «10 s sin snapshots»; en el código cualquier mensaje del puente (incluido el `ping` de cada 3 s) renueva el plazo.
`ui/app.py` (docstring)	Menciona `--lanzar-puente URL`, que no existe en `crear_parser()`.
`mds/ARQUITECTURA.md` §6.3	Dice que `PantallaHmi.qml` tiene la propiedad `hueco`; ahora la rejilla está en `RejillaHmi.qml` (g).
`ui/DISENO.md`, `tokens.json`	Proponen `SpringAnimation` al final de carrera y una sombra de panel; no están implementadas (se usa `OutBack` de 140 ms).
`tokens.json`	Monoespaciada «JetBrains Mono, Consolas»; el HMI usa Consolas.

Dónde	Observación
`assets/assets_cilindros/README.md`	Dice que las capas PNG están a 4 px/mm; las del ø20 están a 8 px/mm.
`gen_cilindros.py` (docstring)	Menciona `clipPath`; ningún SVG generado lo usa.
`gateway/ladder_sim.py`	Franquea como mucho una transición por barrido y el GS reinicia explícitamente TON y END; en el PLC los escalones se evalúan en secuencia en el mismo barrido. Diferencia sin efecto visible a 5 ms por barrido.
Canvas del HMI	GRAFCET, Latencia, Cronograma y `AnilloProgreso` no se repintan al cambiar de tema; conservan los colores anteriores hasta que cambian sus datos o su tamaño.
`BandaAlarmas.qml`	El icono usa siempre `alarma_claro.svg` y no sigue `sufijoIcono`.
`EtapaGrafcet.qml`	Dos colores fijos (`Qt.rgba(1, 0.69, 0, 0.14)` y `#111417`) fuera de `UI.Theme`.
`FranjaMando.qml`	Un `join` con un salto de línea literal dentro de la cadena.
`ui/qml/HMI.qrc`	Sin confirmar en Git y sin referencias desde Python ni QML.
`docs/img/conexion_dialogo_real.png`	Muestra `PROYECTO1_REAL`; está desactualizada.
Servidor web	No comprueba el rango 0…32 767 de los parámetros (lo hacen el frontend y el gateway).

APÉNDICE D

Glosario

ACC / PRE / DN
Acumulador, preajuste y bit de hecho de un temporizador o contador del SLC.

Binding
Expresión de QML que se reevalúa sola cuando cambian sus dependencias.

Caché de RSLinx
Copia de los valores que RSLinx mantiene sondeando el PLC al ritmo del grupo; leer de ella no espera al PLC.

COM / DCOM
Modelo de objetos binarios de Windows sobre el que está construido OPC DA.

DF1
Protocolo serie de Allen-Bradley para la familia SLC/PLC-5.

Easing
Curva que convierte tiempo normalizado en avance de una animación.

Gateway
Proceso de 32 bits que habla OPC DA y publica el snapshot.

GRAFCET
Lenguaje gráfico de especificación de automatismos secuenciales (IEC 60848).

HMAC
Código de autenticación de mensajes basado en una función hash y una clave secreta.

Item / grupo / topic
Variable OPC, conjunto de variables leídas juntas y nombre que asocia un driver y un PLC en RSLinx.

msgpack
Formato de serialización binario con el modelo de datos de JSON.

Modo suave
Animación prevista con el cursor del ciclo y el temporizador extrapolado.

Puente
Proceso que reenvía el snapshot al servidor web y verifica las órdenes firmadas.

QML / Qt Quick
Lenguaje declarativo de interfaces de Qt y su biblioteca de elementos visuales.

Scene graph
Árbol de nodos que Qt Quick dibuja con la GPU en un hilo de render.

Snapshot
Diccionario con el estado completo de la planta que el gateway publica en cada ciclo.

ZeroMQ
Biblioteca de mensajería sin broker con patrones PUB/SUB y REQ/REP.

Bibliografía

[1] International Society of Automation. *ANSI/ISA-101.01-2015, Human Machine Interfaces for Process Automation Systems*. ISA, 2015.

[2] B. Hollifield, D. Oliver, I. Nimmo y E. Habibi. *The High Performance HMI Handbook*. PAS, 2008.

[3] International Society of Automation. *ANSI/ISA-18.2, Management of Alarm Systems for the Process Industries*. ISA, 2016.

[4] International Electrotechnical Commission. *IEC 62682, Management of alarm systems for the process industries*.

[5] IEC. *IEC 60073:2002, Basic and safety principles for man-machine interface, marking and identification – Coding principles for indicators and actuators*.

[6] IEC. *IEC 60204-1:2016, Safety of machinery – Electrical equipment of machines – Part 1: General requirements*.

[7] ISO. *ISO 13850:2015, Safety of machinery – Emergency stop function – Principles for design*.

[8] IEC. *IEC 60848:2013, GRAFCET specification language for sequential function charts*.

[9] IEC. *IEC 61131-3:2013, Programmable controllers – Part 3: Programming languages*.

[10] ISO. *ISO 1219-1:2012, Fluid power systems and components – Graphical symbols and circuit diagrams – Part 1: Graphical symbols for conventional use and data-processing applications*.

[11] ISO. *ISO 15552:2018, Pneumatic fluid power – Cylinders with detachable mountings, 1000 kPa (10 bar) series, bores from 32 mm to 320 mm – Basic, mounting and accessories dimensions*.

[12] ISO. *ISO 6432:2015, Pneumatic fluid power – Single rod cylinders, 1000 kPa (10 bar) series, bores from 8 mm to 25 mm – Basic and mounting dimensions*.

[13] ISO. *ISO 6358-1:2013, Pneumatic fluid power – Determination of flow-rate characteristics of components using compressible fluids*.

[14] IEC. *Serie IEC 62443, Security for industrial automation and control systems*.

[15] OPC Foundation. *OPC Data Access Custom Interface Specification*, versión 2.05a, 2002.

[16] I. Fette y A. Melnikov. *RFC 6455: The WebSocket Protocol*. IETF, 2011.

[17] OASIS. *MQTT Version 3.1.1*. OASIS Standard, 2014 (también ISO/IEC 20922:2016).

[18] H. Krawczyk, M. Bellare y R. Canetti. *RFC 2104: HMAC, Keyed-Hashing for Message Authentication*. IETF, 1997.

[19] NIST. *FIPS PUB 180-4, Secure Hash Standard (SHS)*, 2015.

[20] K. Moriarty, B. Kaliski y A. Rusch. *RFC 8018: PKCS #5: Password-Based Cryptography Specification Version 2.1*. IETF, 2017.

[21] W3C. *Web Content Accessibility Guidelines (WCAG) 2.2*, 2023.

[22] P. Hintjens. *ZeroMQ: Messaging for Many Applications*. O'Reilly, 2013. Guía en línea: `https://zguide.zeromq.org`.

[23] MessagePack. *MessagePack specification*. `https://msgpack.org`.

[24] The Qt Company. *Qt 6 Documentation: Qt Quick, QML, Animation and Transitions in Qt Quick, Qt SVG*. `https://doc.qt.io`.

[25] The Qt Company. *Qt for Python (PySide6) Documentation*. `https://doc.qt.io/qtforpython-6/`.

[26] The Qt Company. *Qt Design Studio Manual*. `https://doc.qt.io/qtdesignstudio/`.

[27] aiohttp contributors. *aiohttp Documentation*. `https://docs.aiohttp.org`.

[28] Eclipse Foundation. *Eclipse Paho MQTT Python Client*. `https://eclipse.dev/paho/`.

[29] three.js authors. *three.js Documentation*, r128. `https://threejs.org/docs/`.

[30] OpenOPC for Python 3 (`OpenOPC-Python3x`). `https://pypi.org/project/OpenOPC-Python3x/`.

[31] Rockwell Automation. *SLC 500 Instruction Set Reference Manual*, publicación 1747-RM001.

[32] Rockwell Automation. *DF1 Protocol and Command Set Reference Manual*, publicación 1770-6.5.16.

[33] Rockwell Automation. *EtherNet/IP Interface 1761-NET-ENI User Manual*, publicación 1761-UM006.

[34] R. Andersson y The Inter Project Authors. *Inter typeface*, versión 4.0. SIL Open Font License 1.1. `https://rsms.me/inter/`.

[35] Repositorio del HMI. `mds/ARQUITECTURA.md`: contrato entre módulos.

[36] Repositorio del HMI. `mds/LATENCIA.md` y `docs/DIAGNOSTICO_CONEXION_PLC.md`: mediciones del 3 de octubre de 2026.

[37] Repositorio del HMI. `mds/ENLACE_WEB.md`, `mds/GEMELO_DIGITAL.md` y `mds/DESPLIEGUE_SERVIDOR.md`.

[38] Repositorio del HMI. `docs/GUIA_QT_DESIGN_STUDIO.docx`: guía de Qt Design Studio y comparativa con Delphi.

[39] Repositorio del HMI. `assets/assets_cilindros/README.md` y `ui/DISENO.md`: sistema de diseño.

[40] Repositorio del HMI. `bitacora/`: bitácoras de las sesiones 01 a 15 (en particular la 02, evaluación normativa de colores).